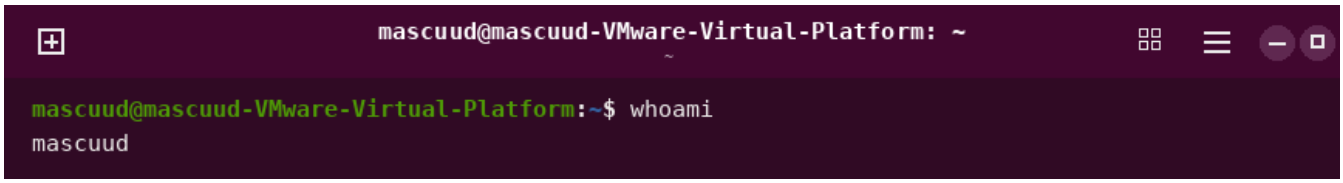
Chapter One

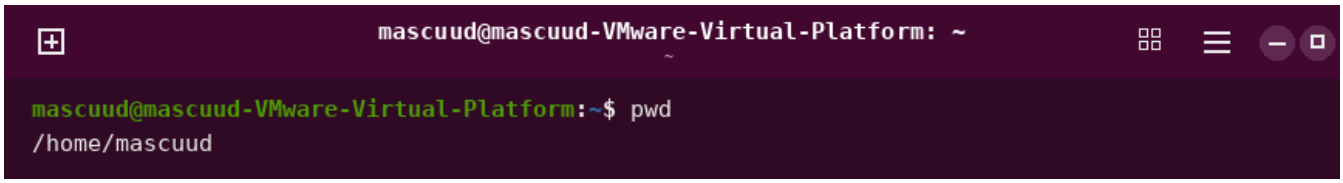
`whoami`

Shows the name of the user who is logged in.



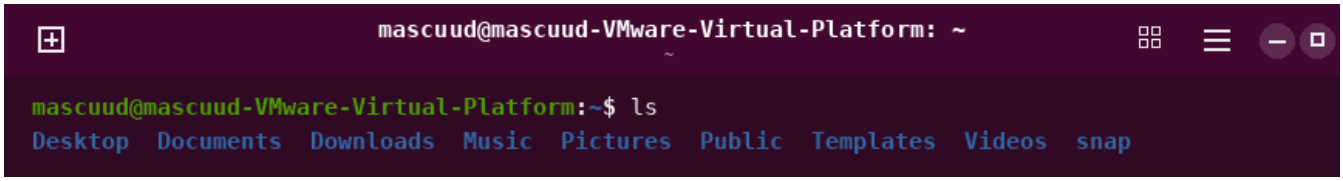
`pwd`

Shows the folder you are in now.



`ls`

Shows the files and folders in the current folder.



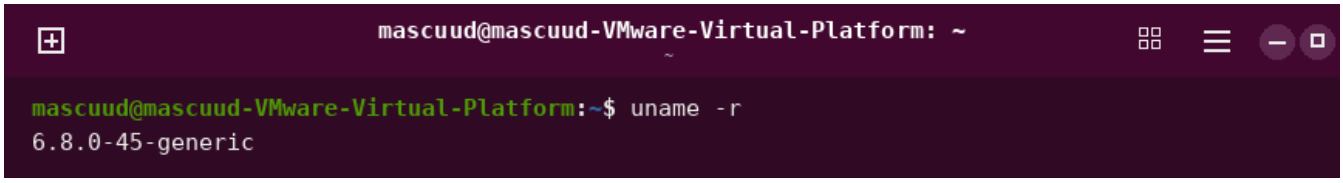
`cat /etc/os-release`

Shows the text inside a file. This file tells the Linux name and version.



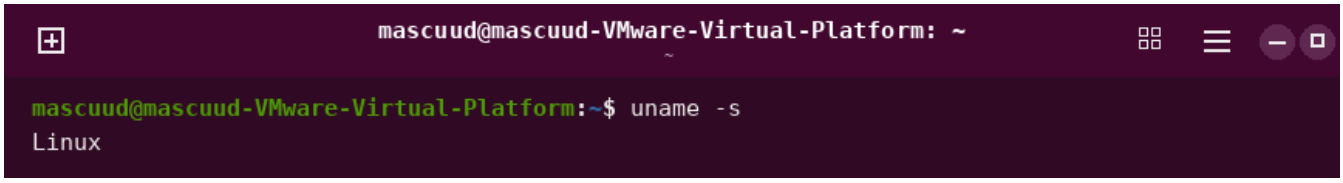
`uname -r`

Shows the version number of the Linux kernel.



`uname -s`

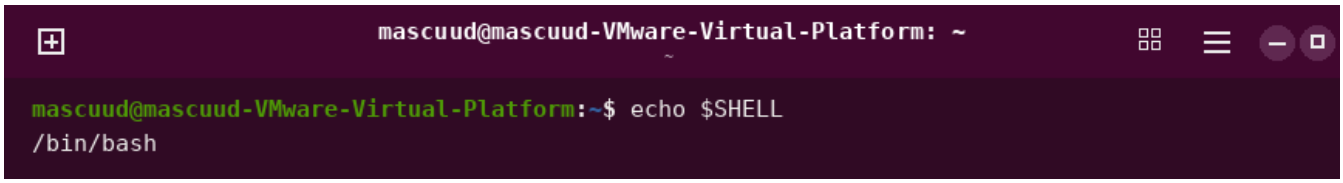
Shows the name of the operating system.



Chapter Two

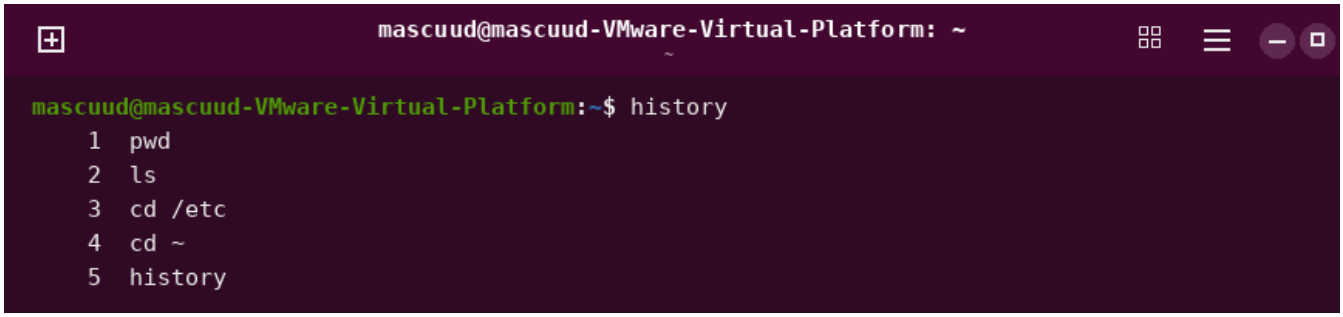
`echo $SHELL`

Shows the name of your shell program.



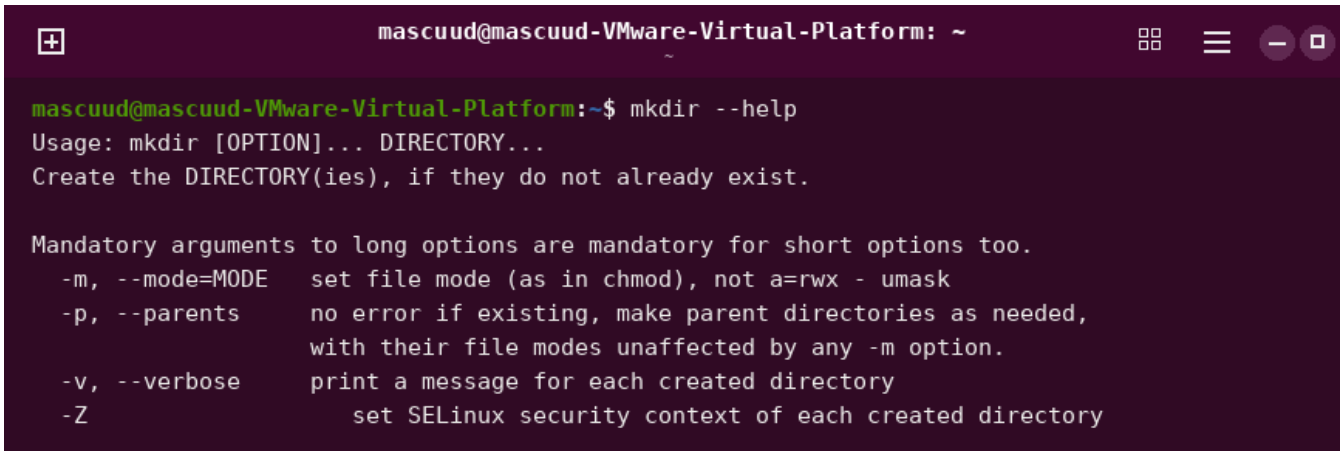
`history`

Shows the commands that were used before in the terminal.



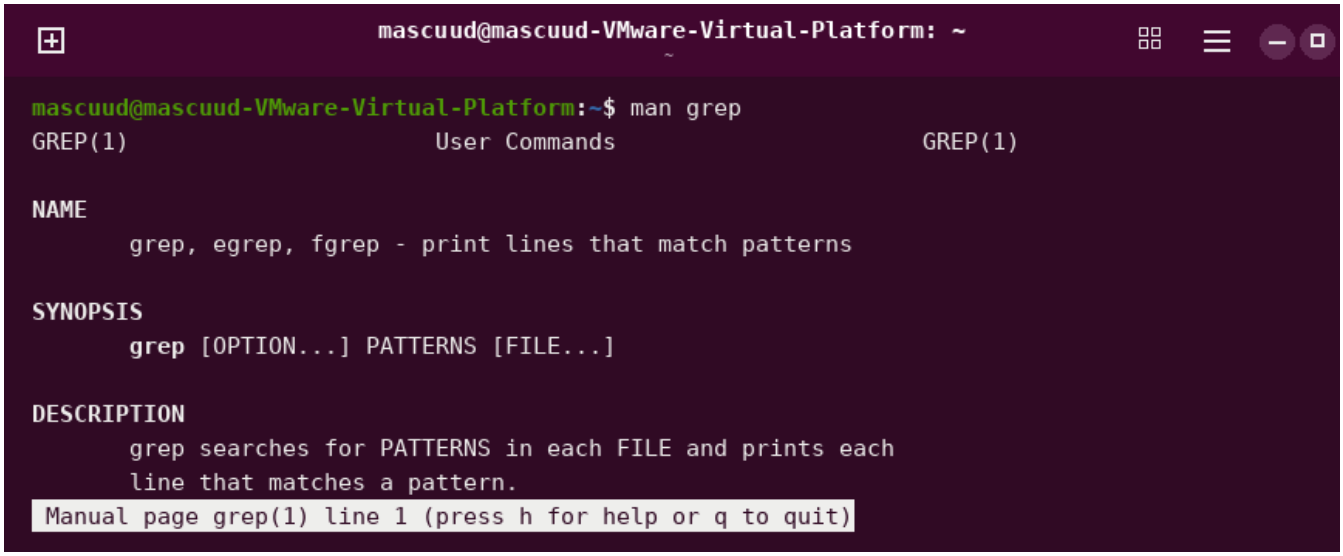
`mkdir --help`

Shows a short help text for the mkdir command.



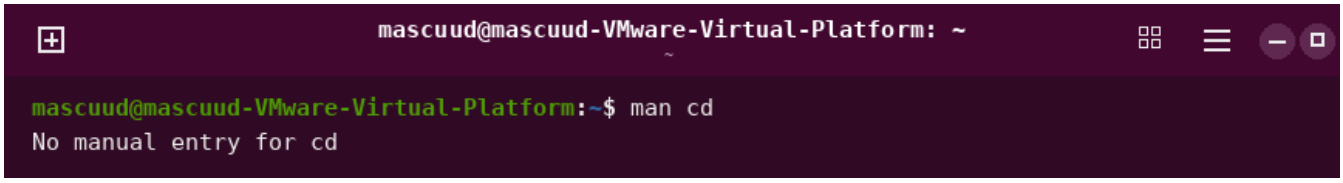
man grep

Opens the full manual of the grep command. Press q to quit.



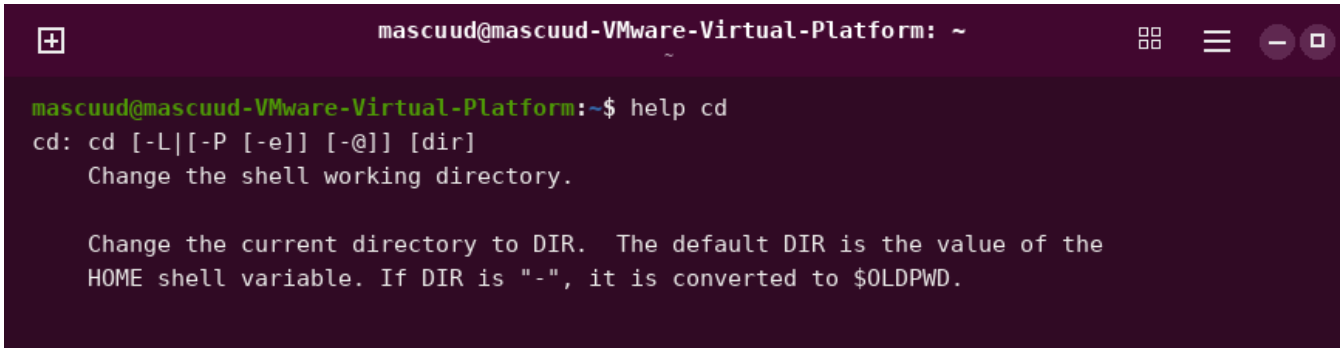
man cd

Shows that cd has no manual page, because it is built into Bash.



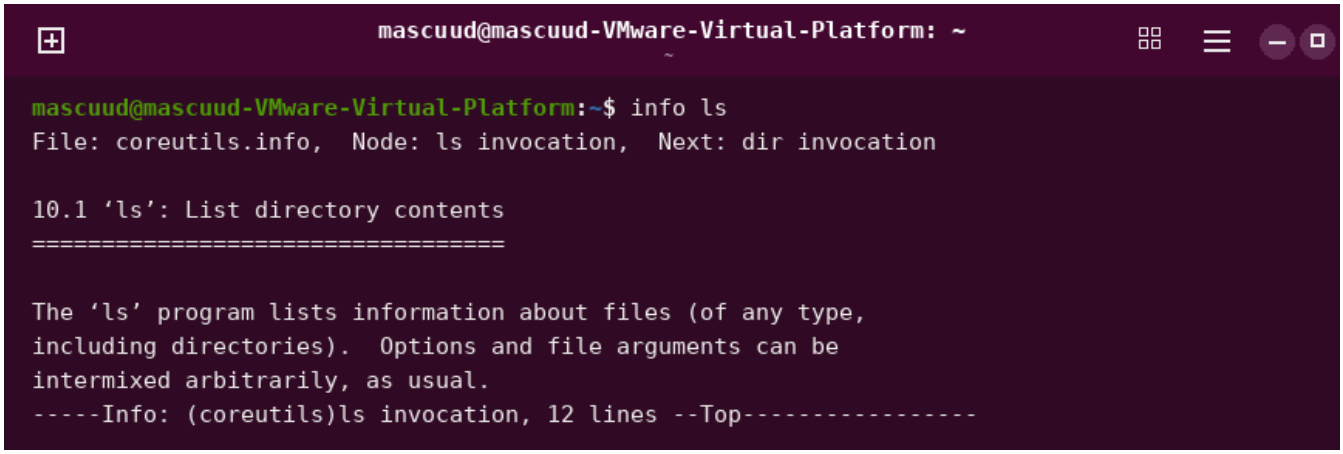
help cd

Shows help for the cd command.



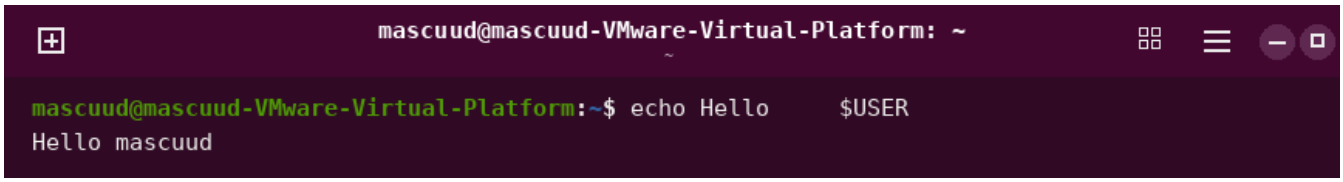
`info ls`

Opens a longer guide for the ls command. Press q to quit.



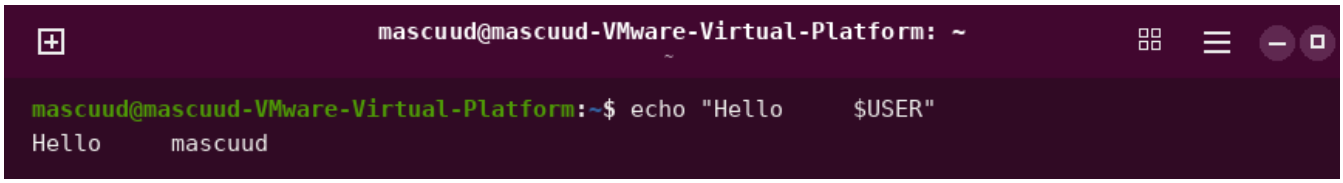
`echo Hello $USER`

Prints the words. Bash joins the spaces and puts in your user name.



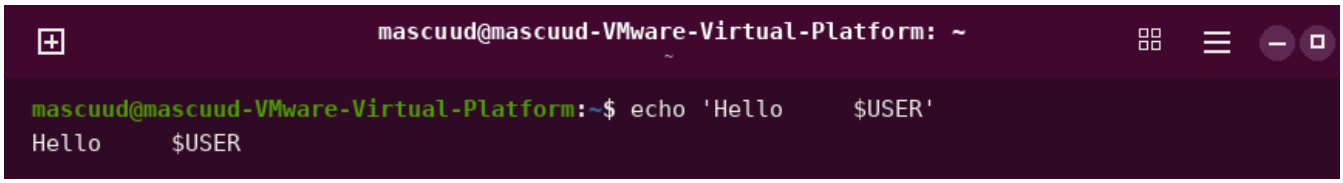
`echo "Hello $USER"`

Prints the words and keeps the spaces. Your user name is still put in.



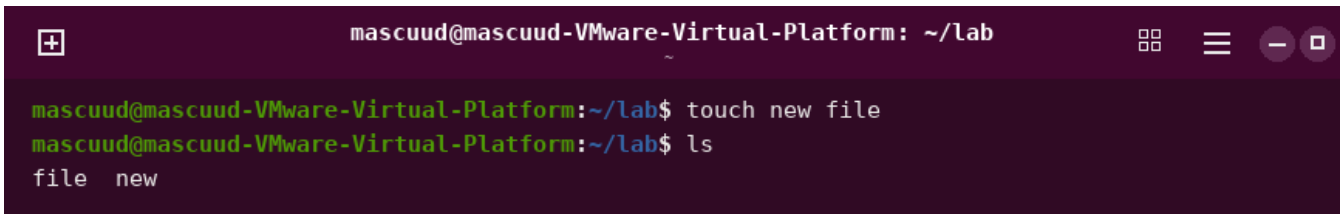
`echo 'Hello $USER'`

Prints the words exactly as you typed them. Nothing is changed.



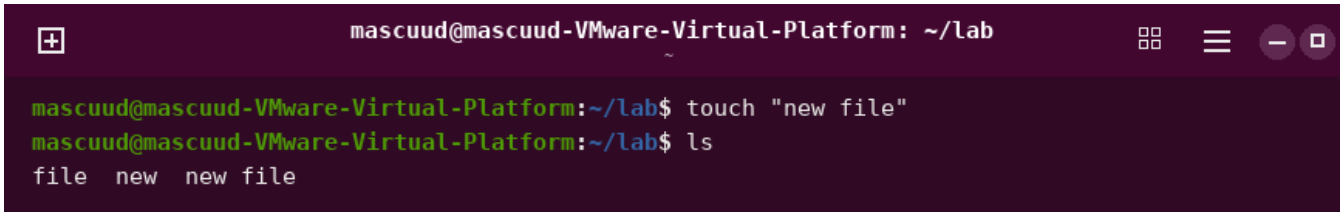
`touch new file`

Makes two files, called new and file, because of the space.



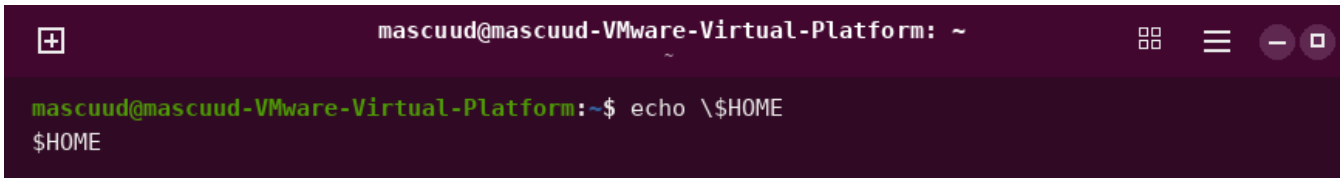
`touch "new file"`

Makes one file called "new file". The quotes keep the space.



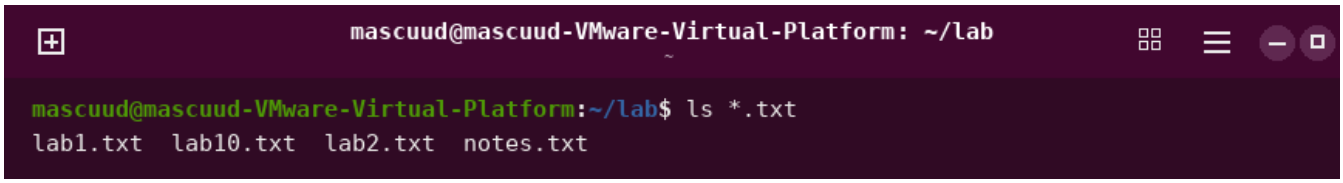
`echo \$HOME`

Prints $HOME as text. The backslash stops its special meaning.



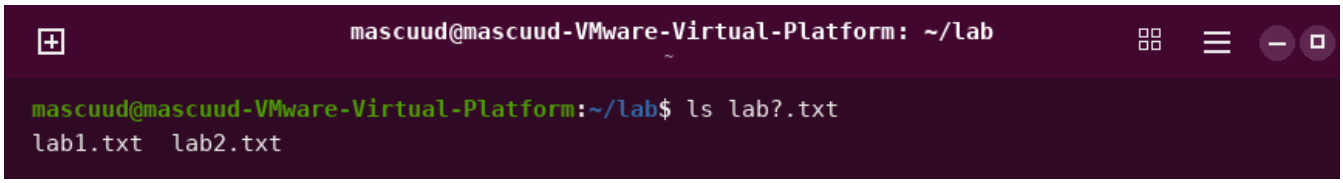
`ls *.txt`

Lists all files that end with .txt. The star means any letters.



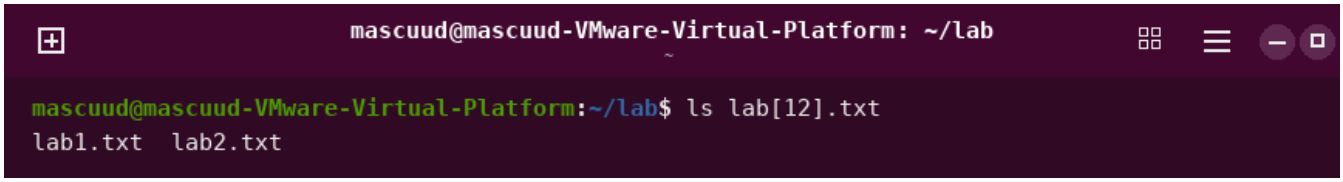
`ls lab?.txt`

Lists files that have exactly one letter where the ? is.



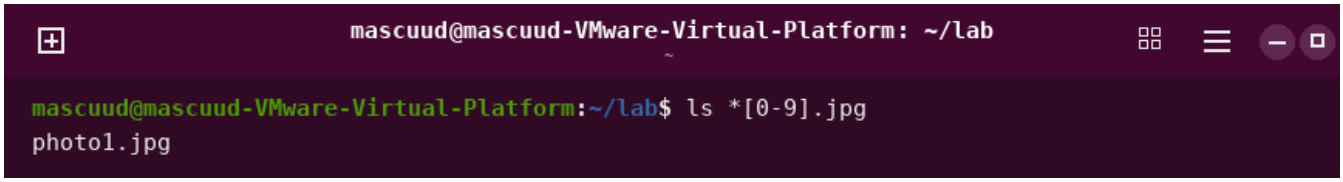
`ls lab[12].txt`

Lists files that have 1 or 2 where the brackets are.



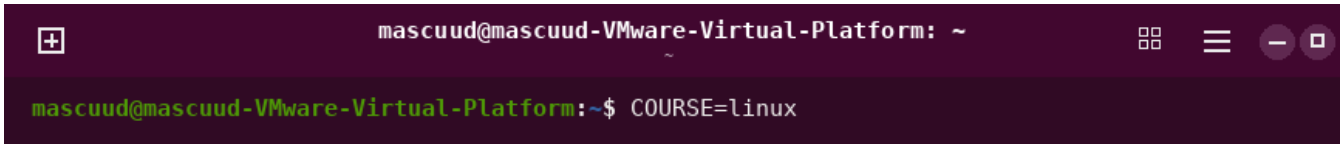
`ls *[0-9].jpg`

Lists .jpg files that end with one number before .jpg.



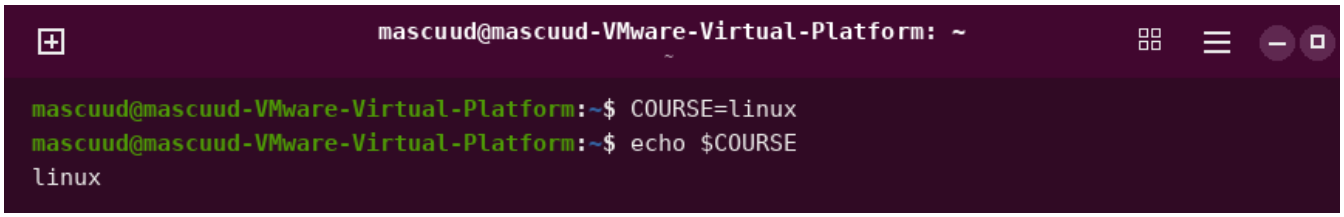
COURSE=linux

Makes a variable called COURSE with the value linux.



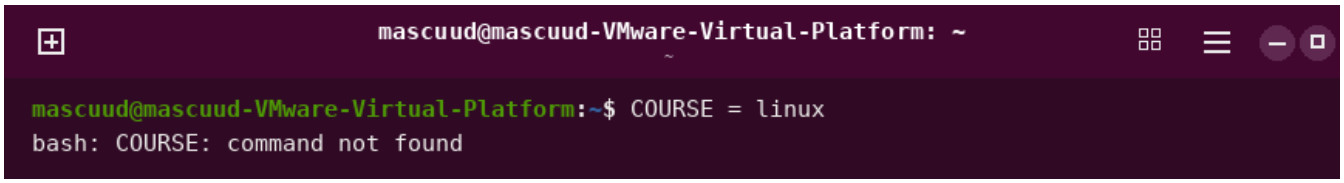
echo $COURSE

Shows the value of the variable COURSE.



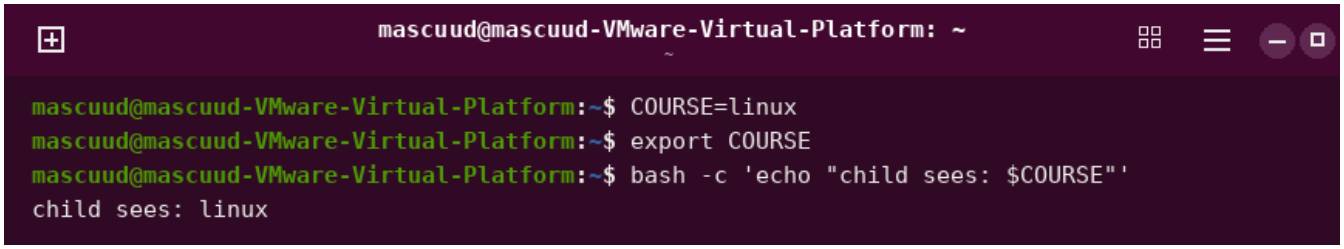
COURSE = linux

Does not work. Spaces around = give an error.



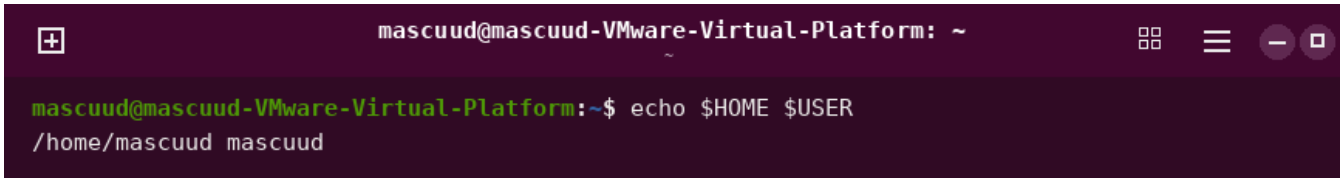
export COURSE

Shares the variable with other programs.



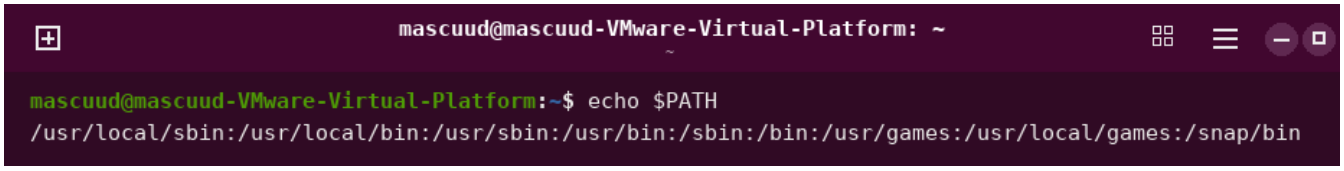
echo $HOME $USER

Shows your home folder and your user name.



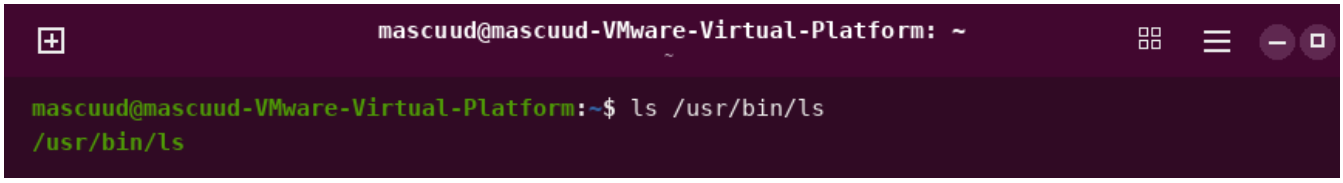
echo $PATH

Shows the list of folders where Bash looks for commands.



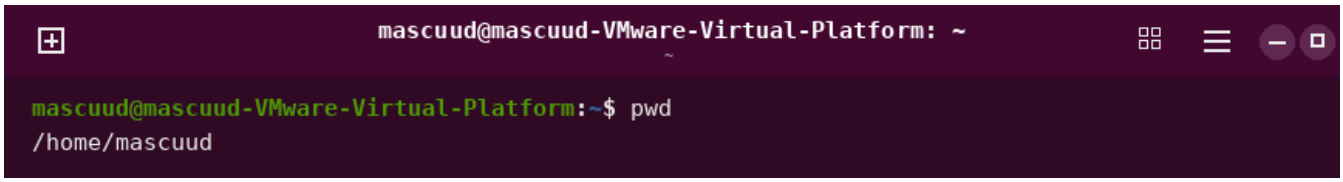
`ls /usr/bin/ls`

Shows the file of the ls command in /usr/bin.



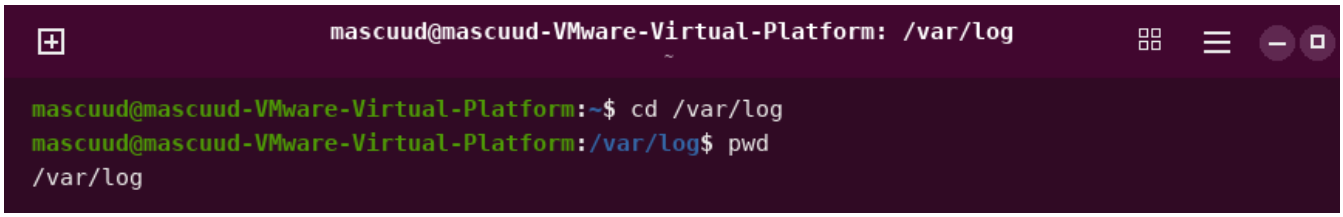
`pwd`

Shows the folder you are in now.



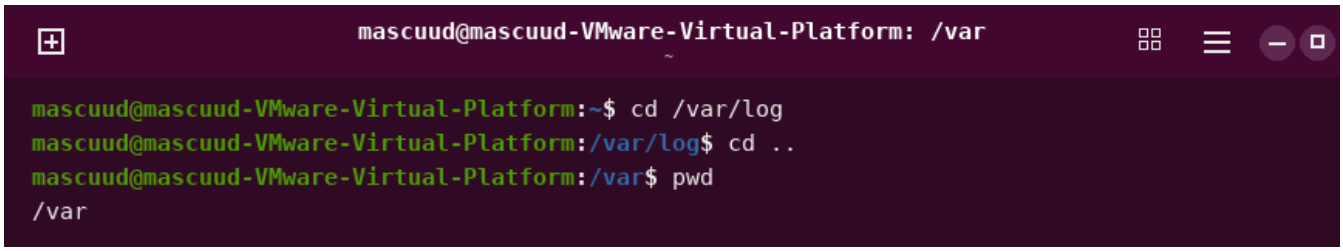
`cd /var/log`

Goes to the /var/log folder.



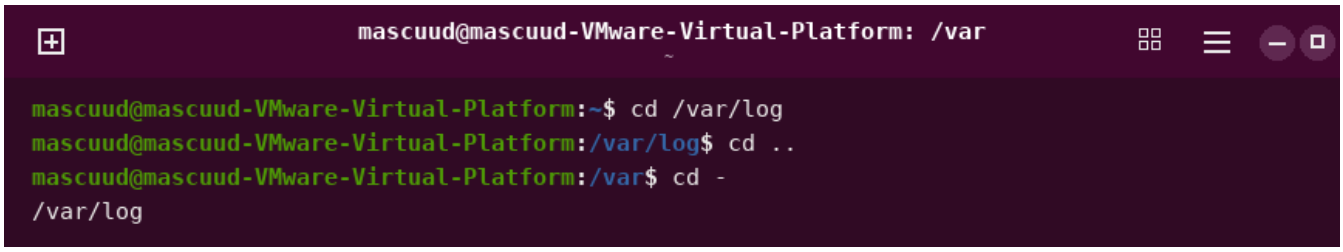
`cd ..`

Goes up to the parent folder.



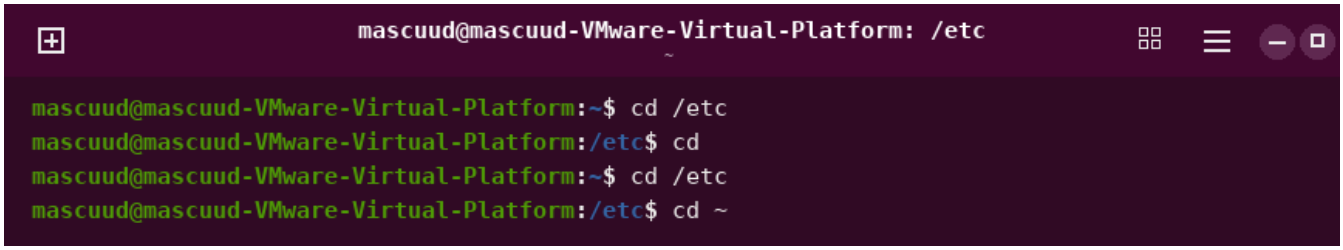
`cd -`

Goes back to the last folder you were in.



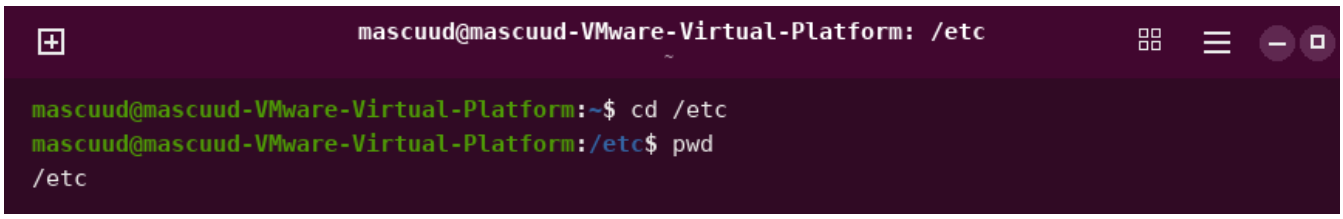
`cd (alone) and cd ~`

Goes to your home folder.



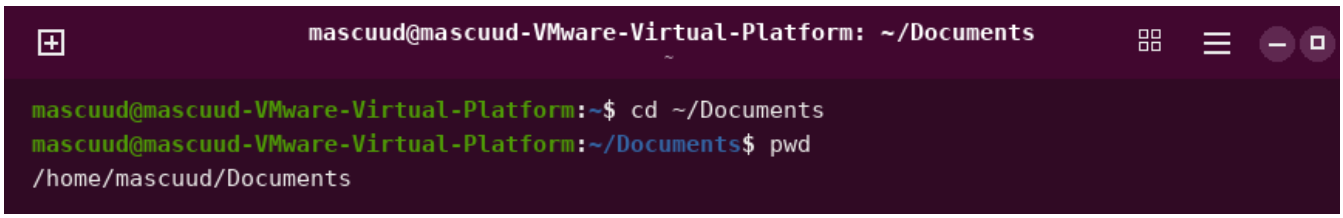
`cd /etc`

Goes to the /etc folder.



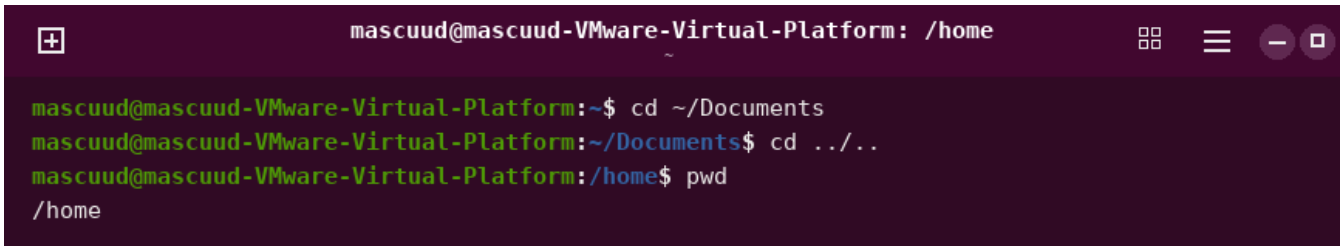
`cd ~/Documents`

Goes to the Documents folder inside your home.



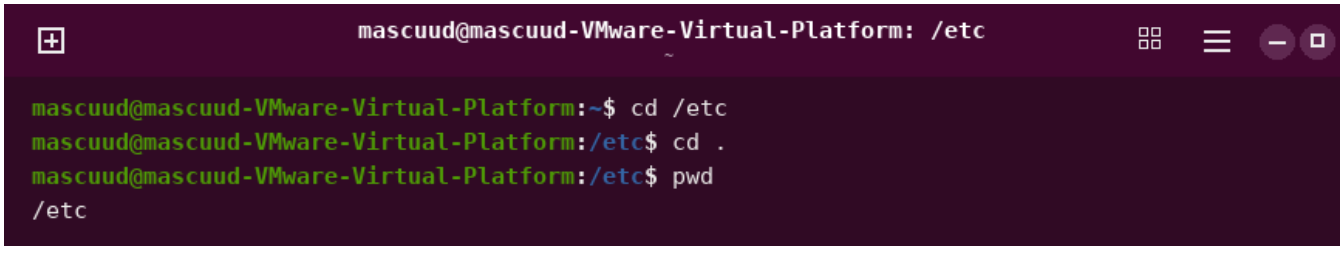
`cd ../..`

Goes up two folders.



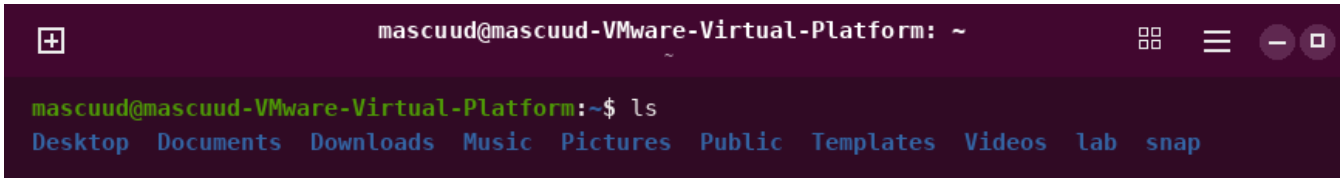
`cd .`

Stays in the same folder.



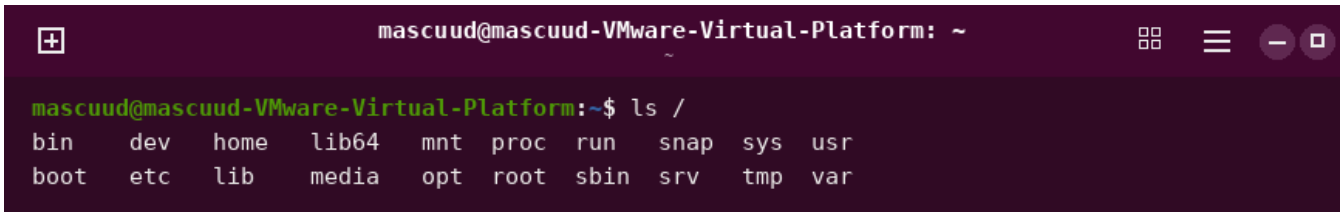
`ls`

Lists the files and folders here.



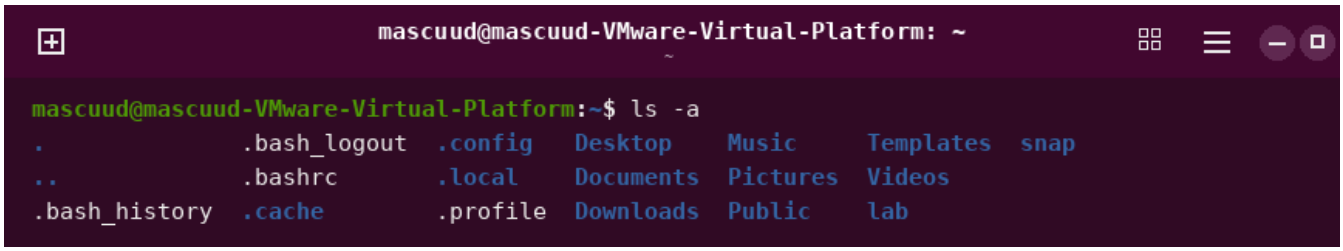
`ls /`

Lists the files and folders in the top folder.



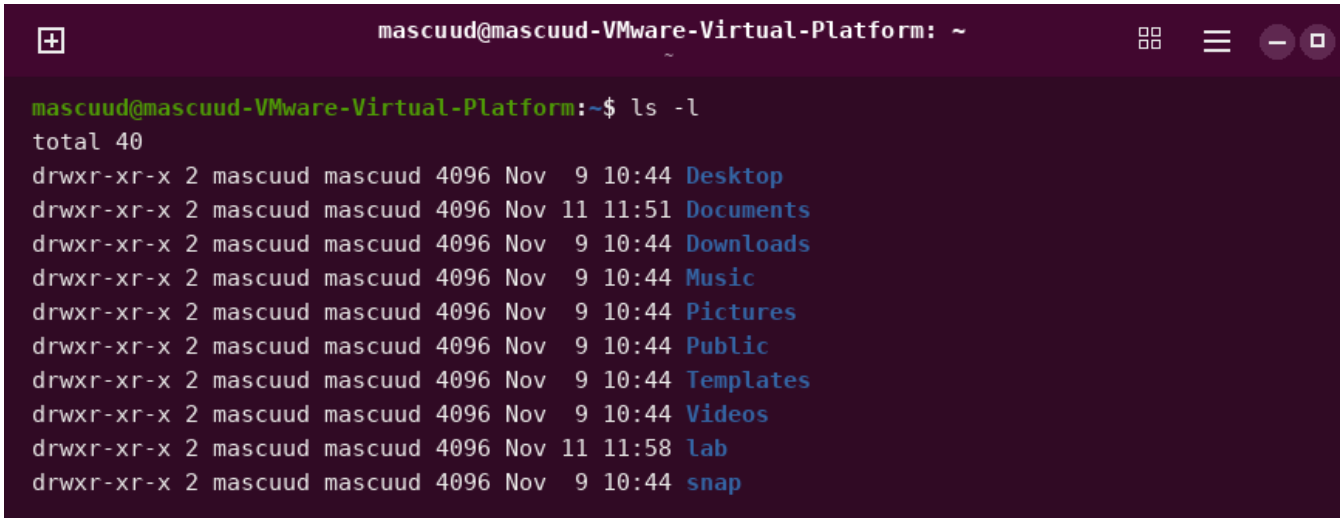
`ls -a / ls --all`

Shows hidden files too. Hidden names start with a dot.



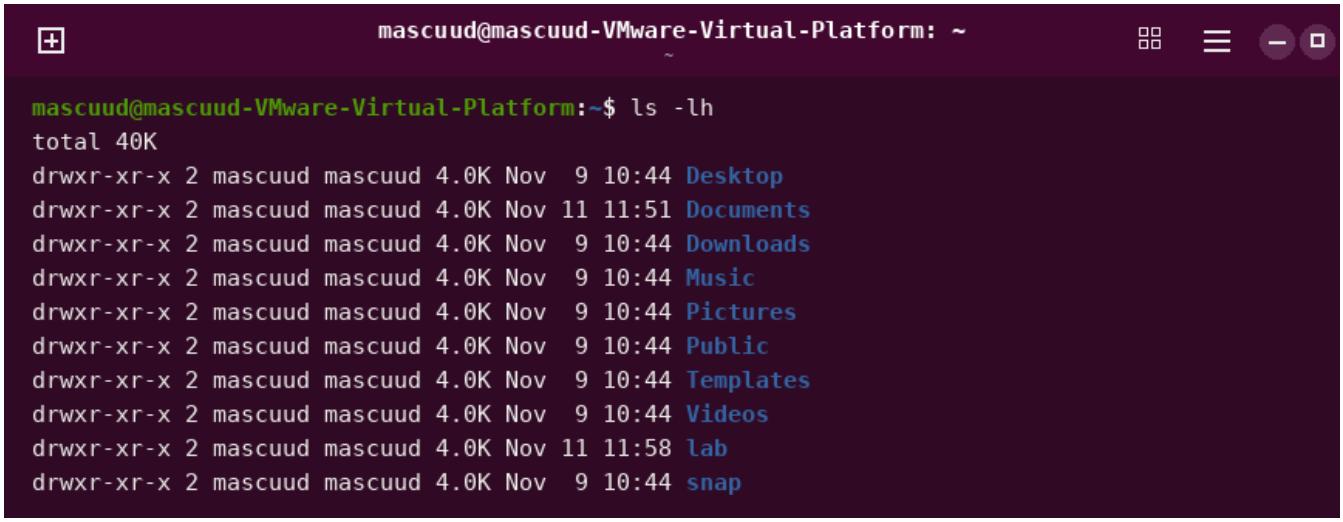
`ls -l`

Shows a long list with details about each file.



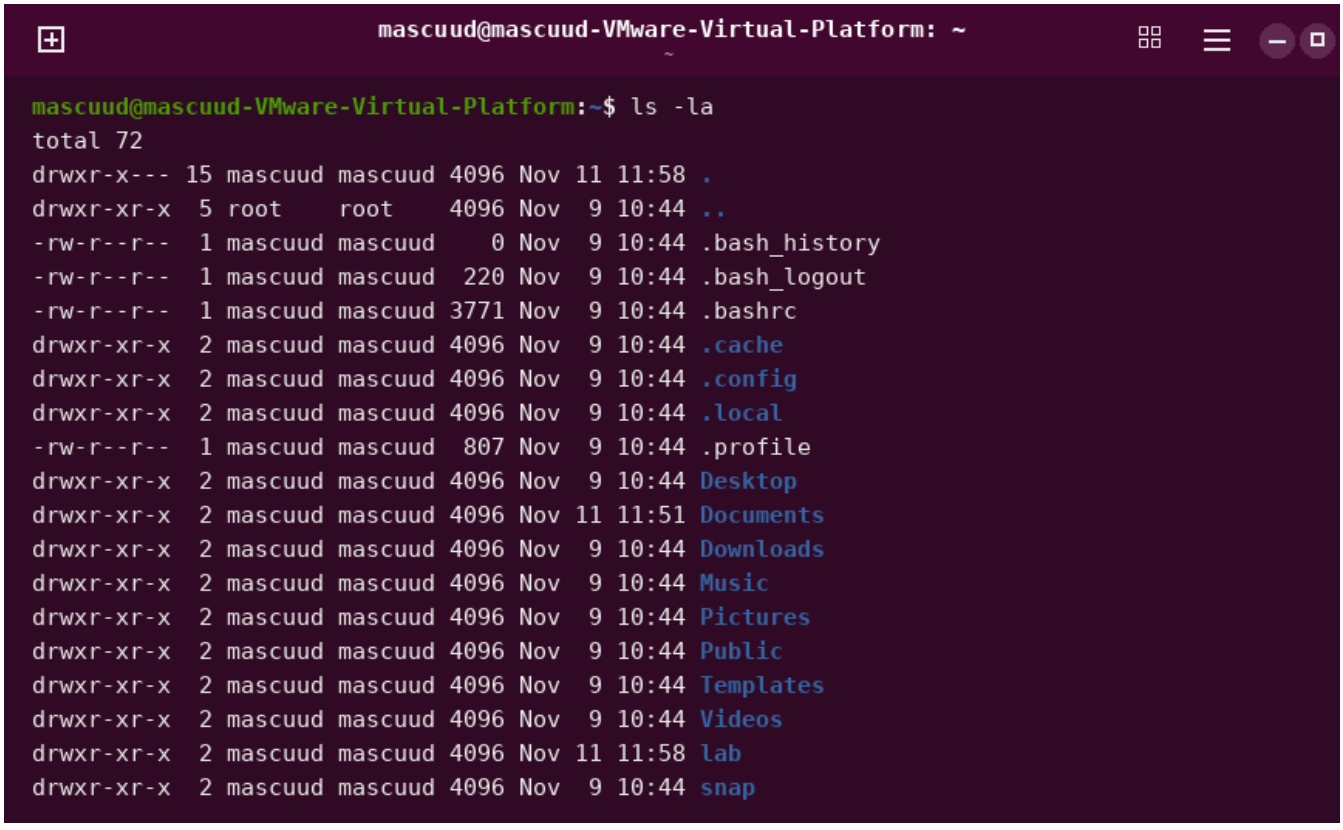
`ls -lh / ls --human-readable`

Shows file sizes in easy units like K and M.



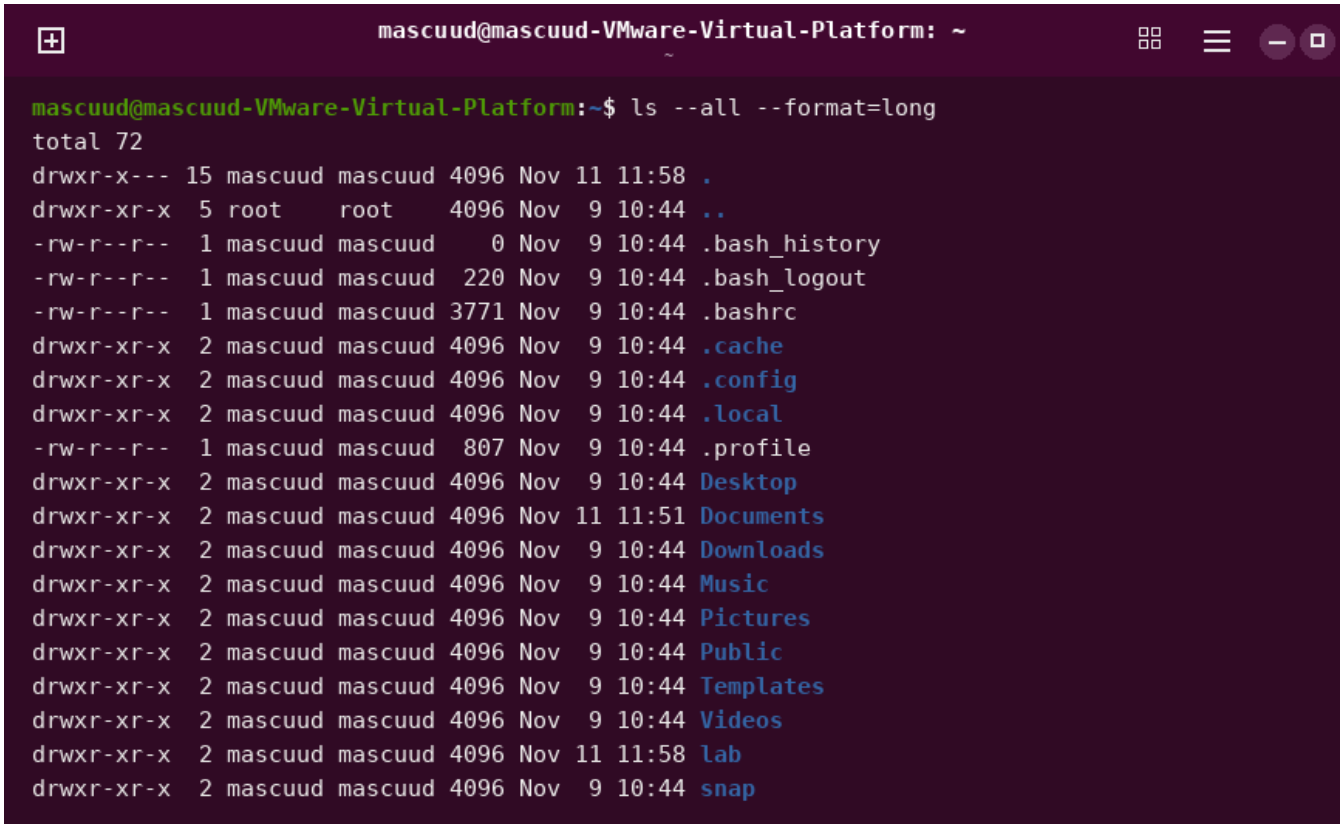
`ls -la`

Shows a long list with hidden files too.



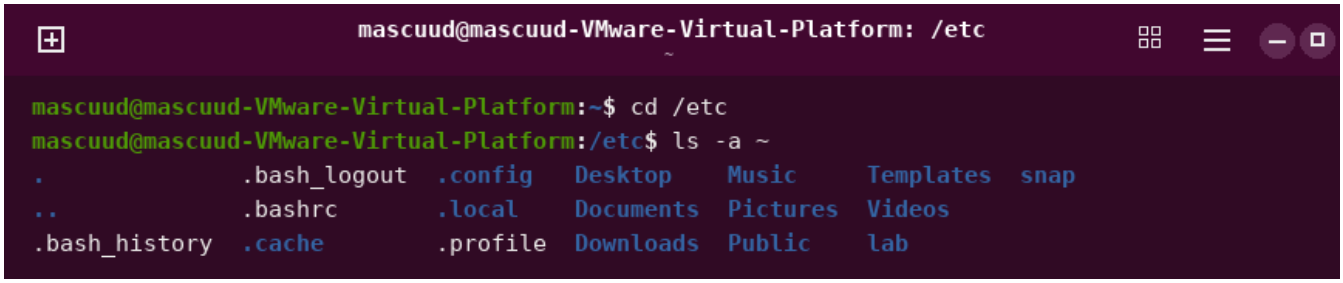
`ls --all --format=long`

Same as ls -la. These are the long option names.



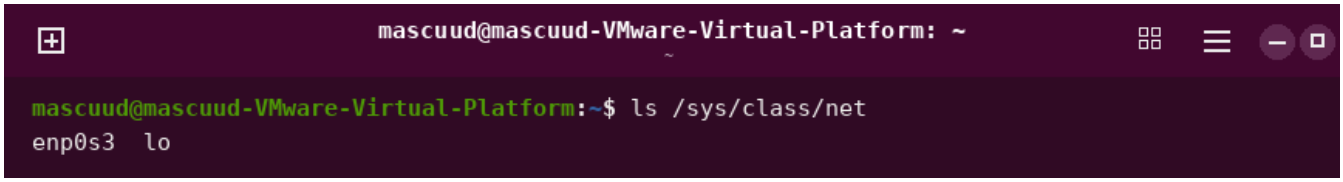
`ls -a ~`

Shows the hidden files in your home folder.



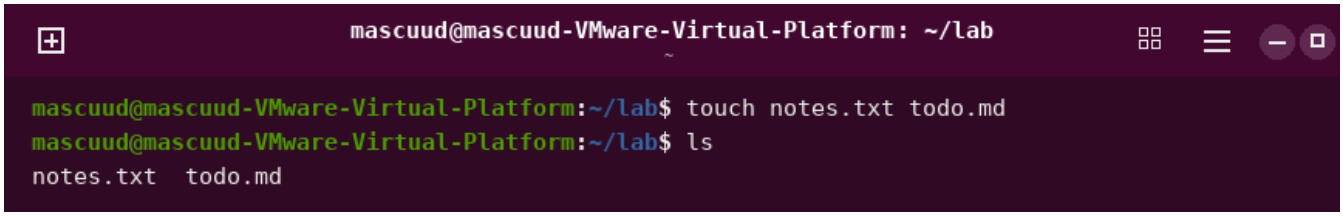
`ls /sys/class/net`

Lists the network cards of the computer.



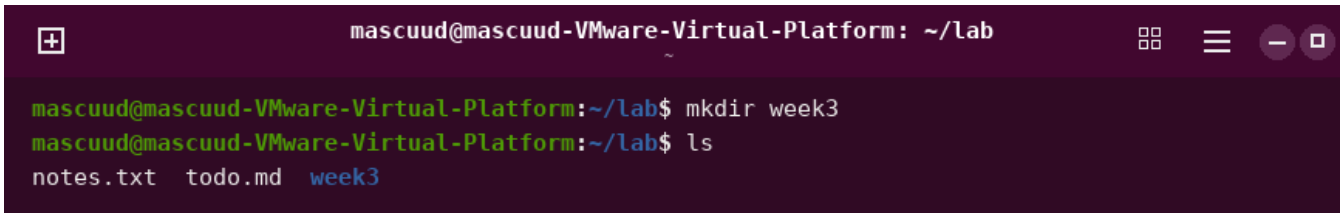
`touch notes.txt todo.md`

Makes two empty files.



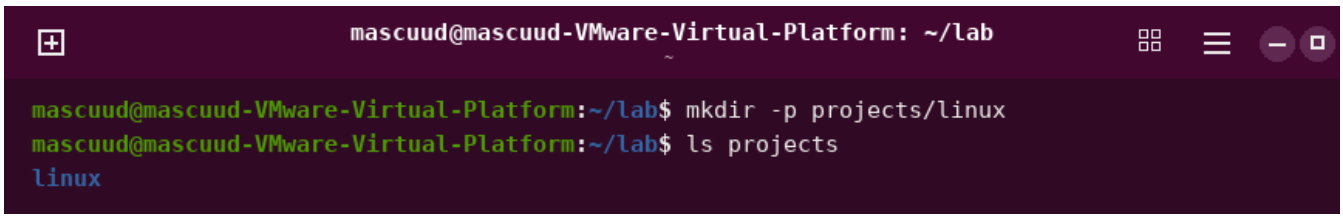
`mkdir week3`

Makes a new folder called week3.



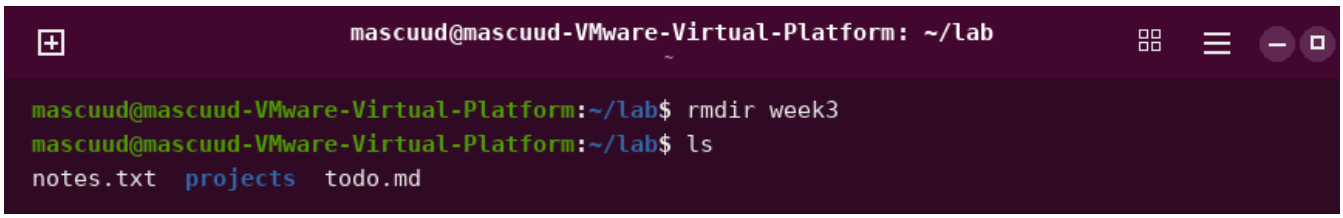
`mkdir -p projects/linux`

Makes a folder and the folder inside it.



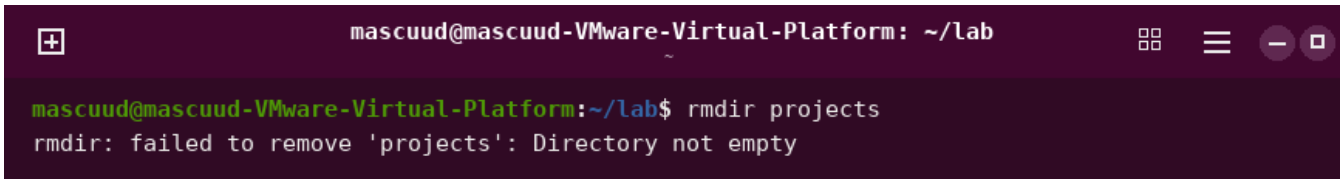
`rmdir week3`

Removes an empty folder.



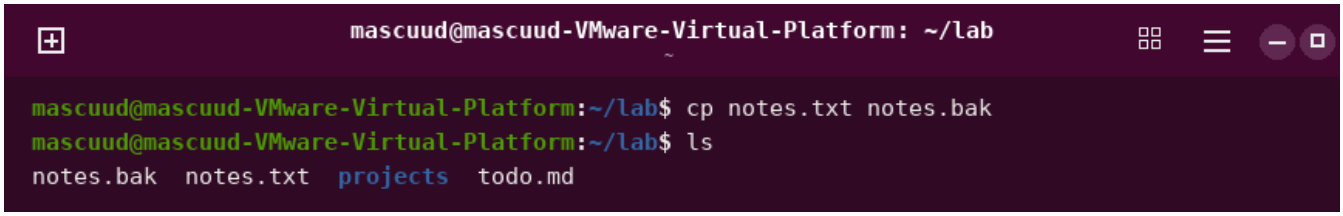
`rmdir projects`

Does not remove the folder, because it is not empty.



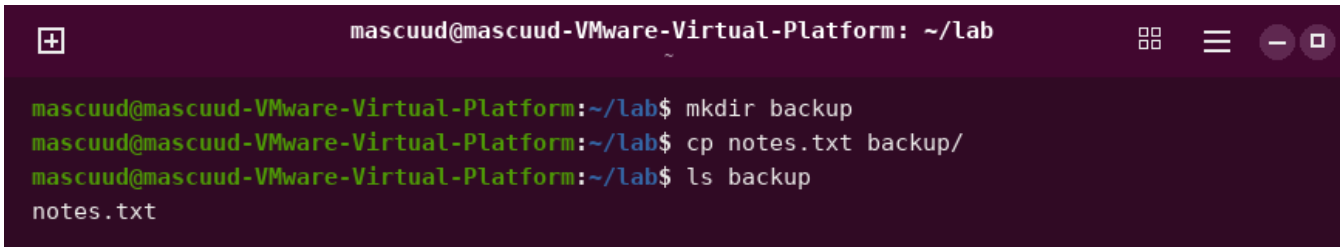
`cp notes.txt notes.bak`

Makes a copy of a file.



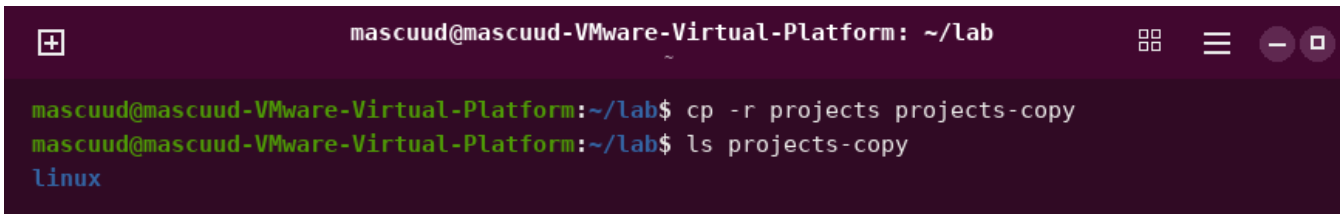
`cp notes.txt backup/`

Copies a file into a folder.



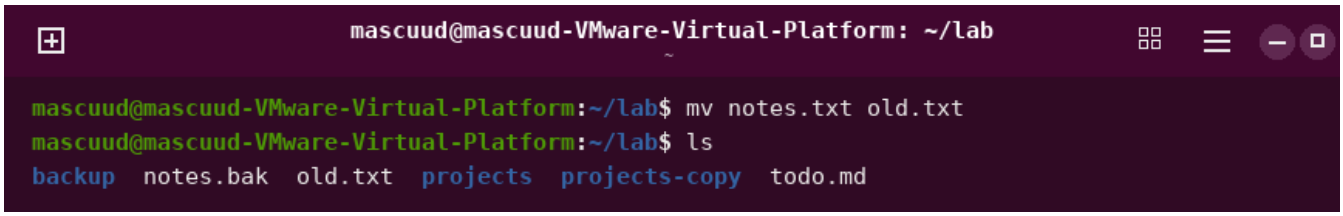
`cp -r projects projects-copy`

Copies a folder and everything inside it.



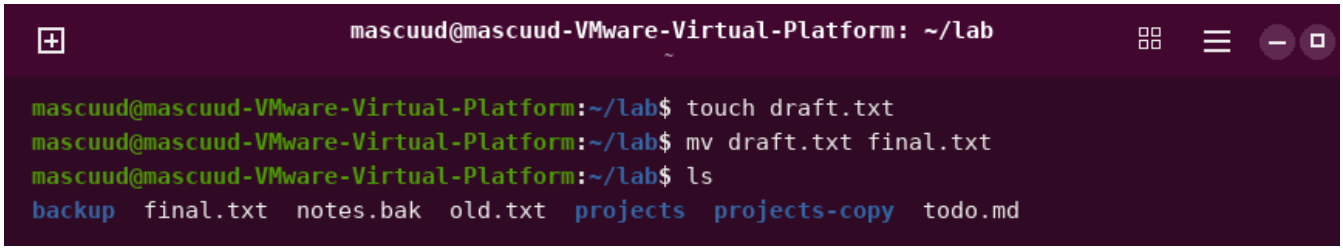
`mv notes.txt old.txt`

Changes the name of a file.



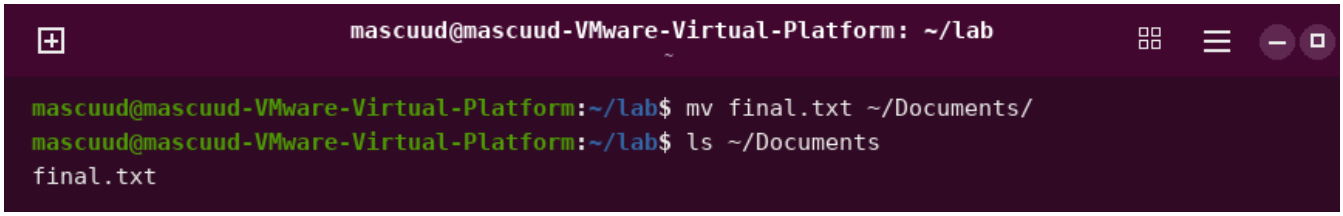
`mv draft.txt final.txt`

Changes the name of a file.



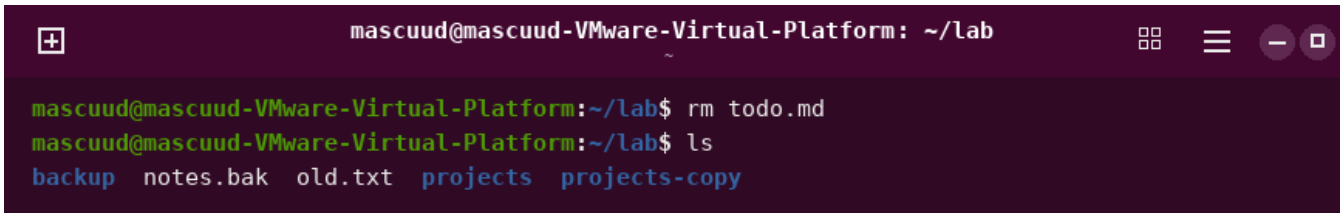
`mv final.txt ~/Documents/`

Moves a file into another folder.



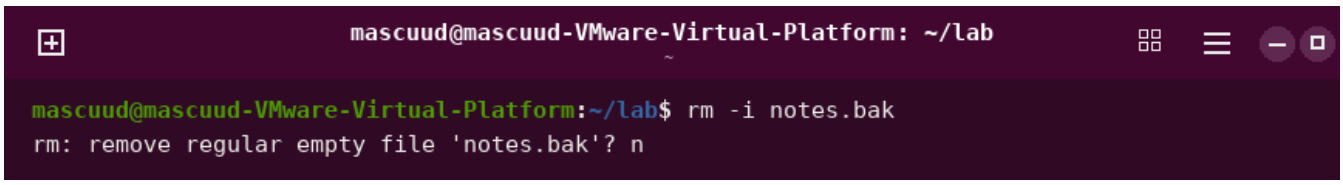
`rm todo.md`

Deletes a file. You cannot get it back.



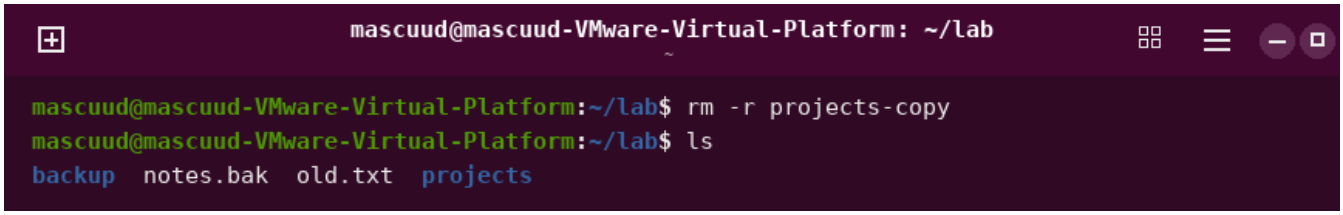
`rm -i notes.bak`

Asks you first, then deletes the file if you say yes.



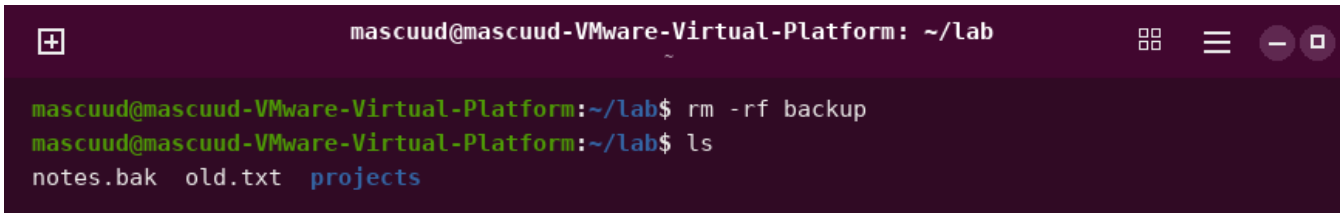
`rm -r projects-copy`

Deletes a folder and everything inside it.



`rm -rf folder`

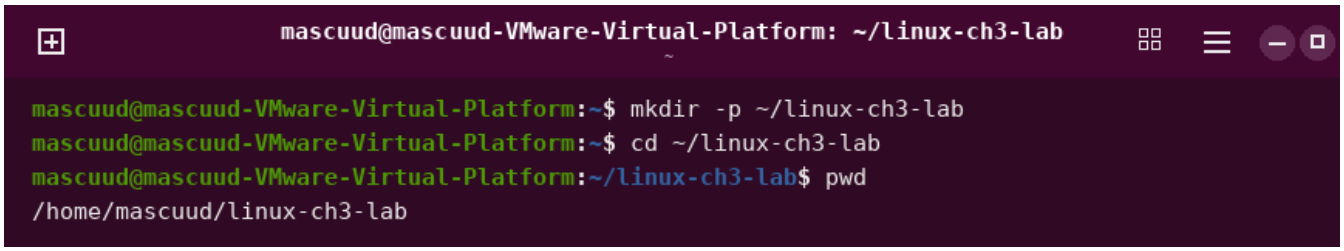
Deletes a folder at once, with no questions. Be careful.



Chapter Three

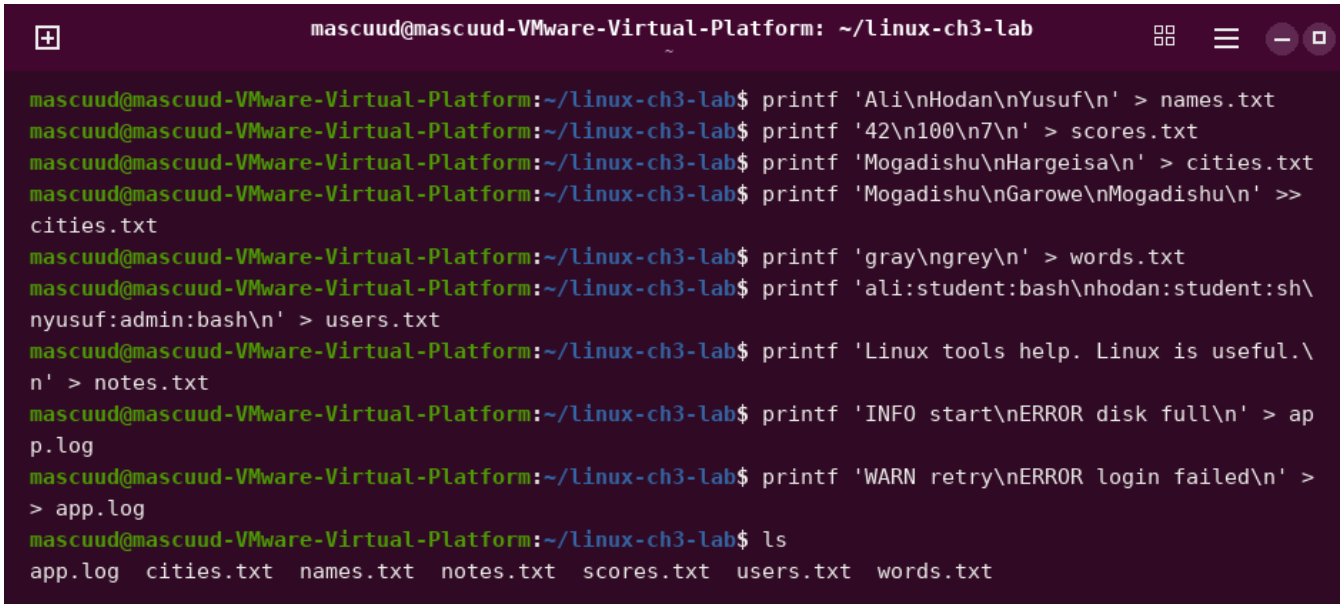
`mkdir -p ~/linux-ch3-lab`

Makes the lab folder and goes into it.



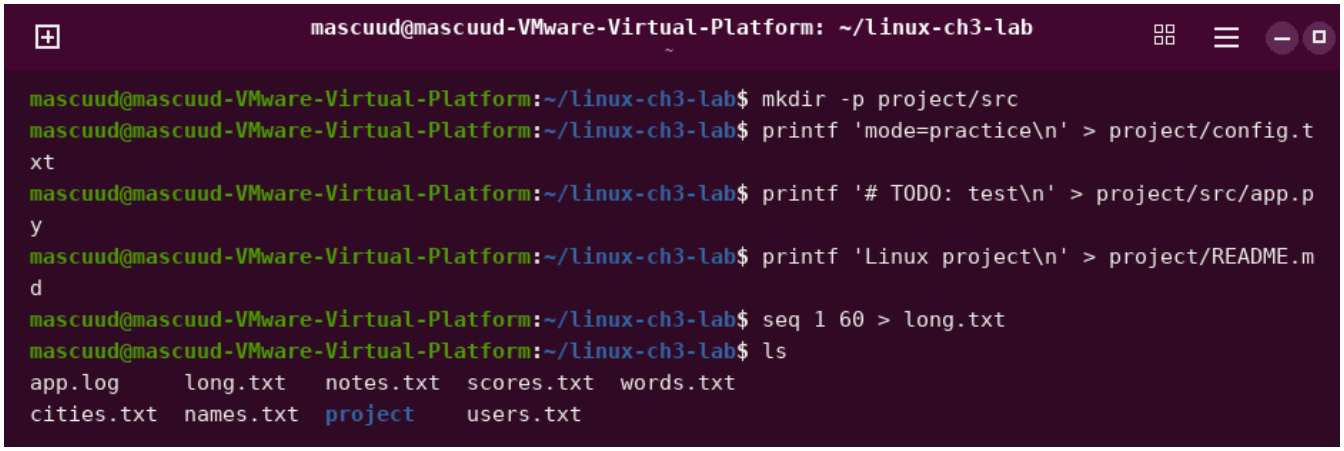
`printf 'text\n' > file`

Puts text into a file. This makes the sample files.



`mkdir -p project/src and seq 1 60 > long.txt`

Makes a folder inside a folder, and a file with the numbers 1 to 60.



`cat names.txt`

Shows everything inside a file.



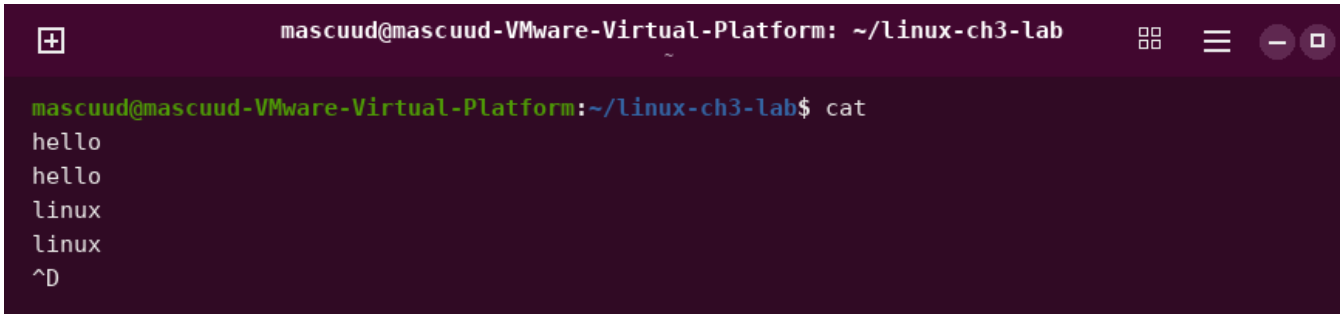
`cat -n project/README.md`

Shows the file with a number on each line.



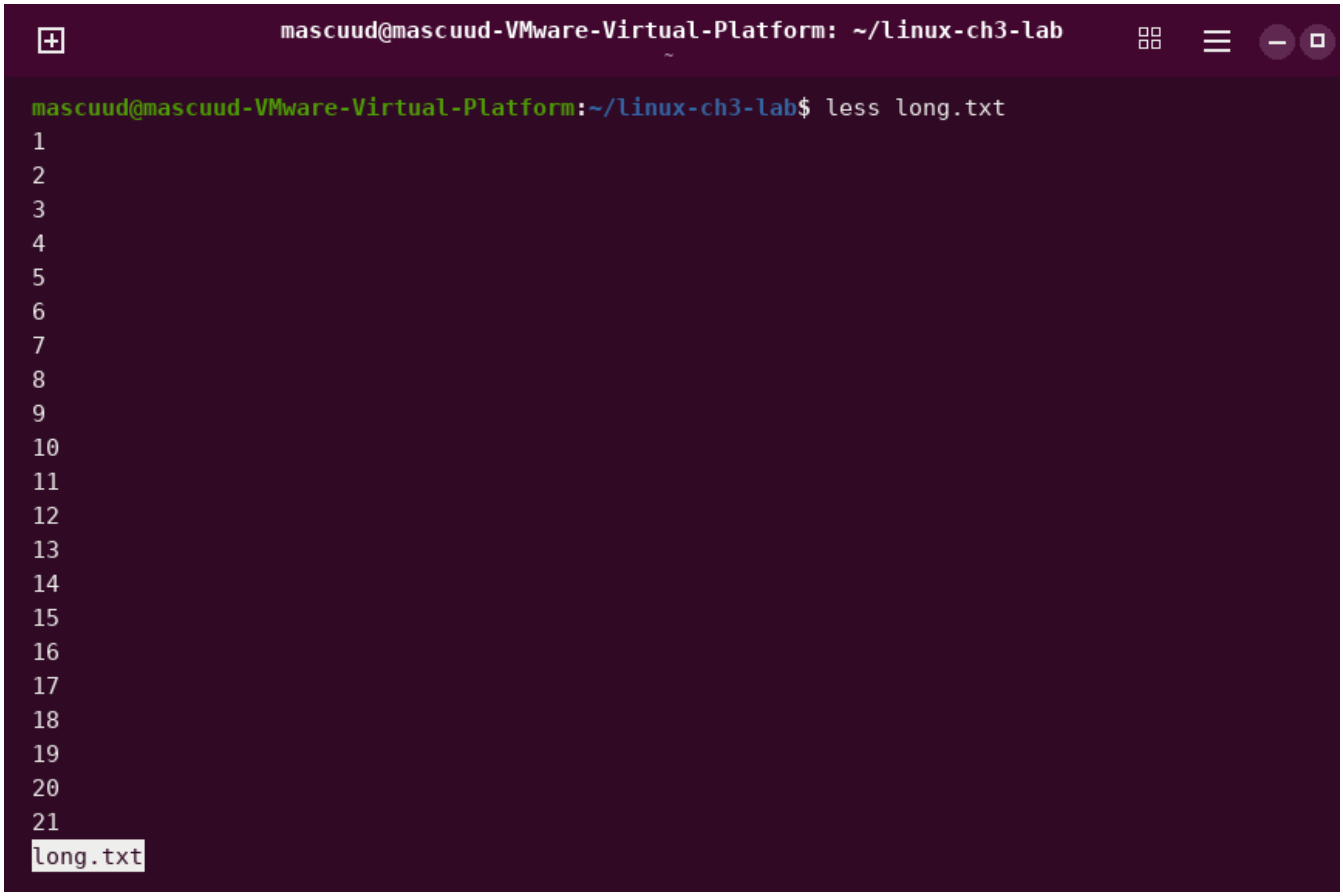
`cat`

Repeats every line you type. Press Ctrl+D to stop.



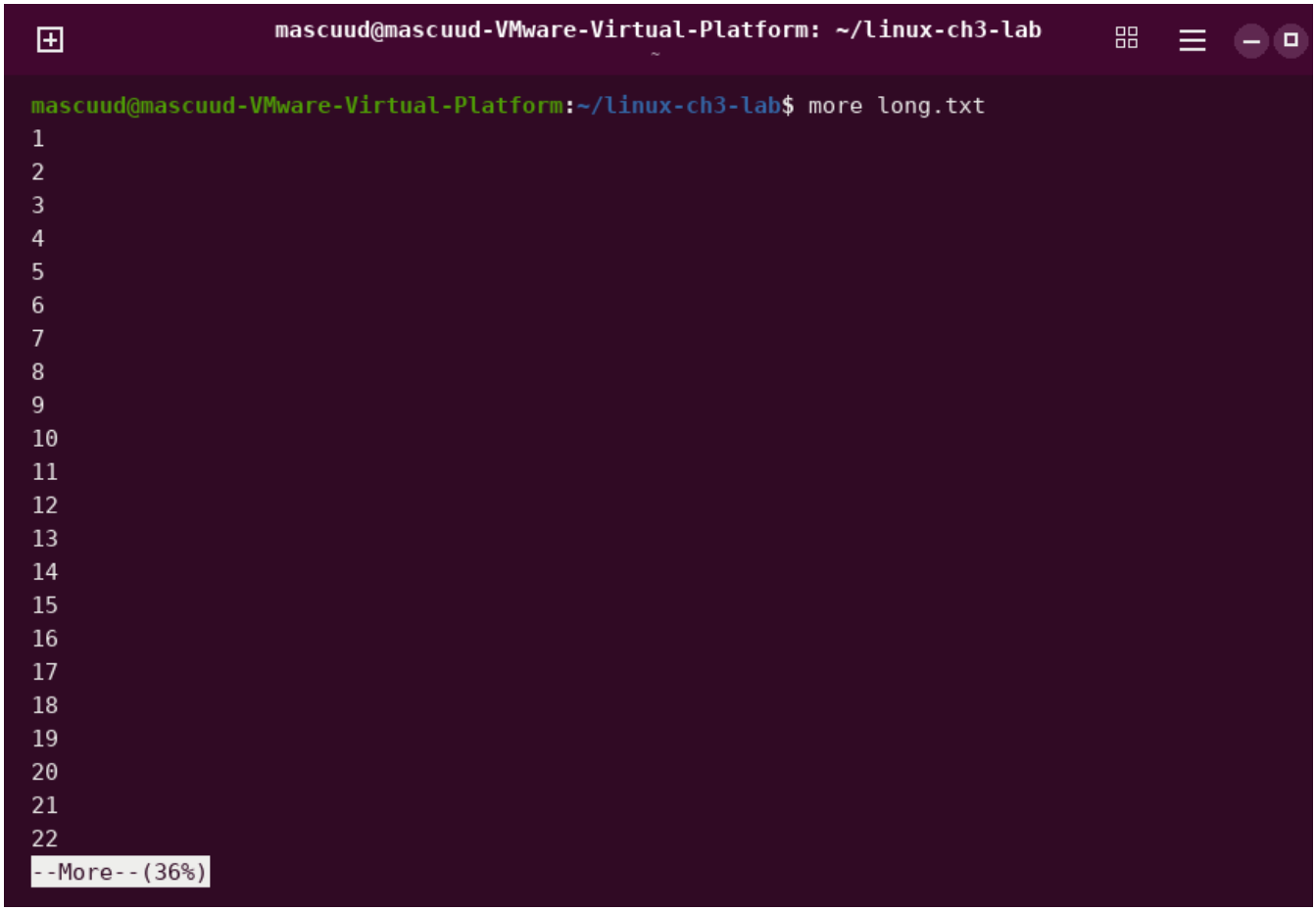
`less long.txt`

Shows a long file one page at a time. Press q to quit.



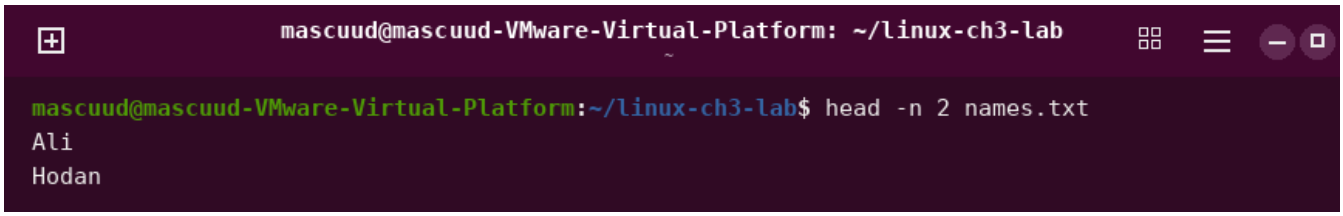
`more long.txt`

Shows a long file one page at a time. Press q to quit.



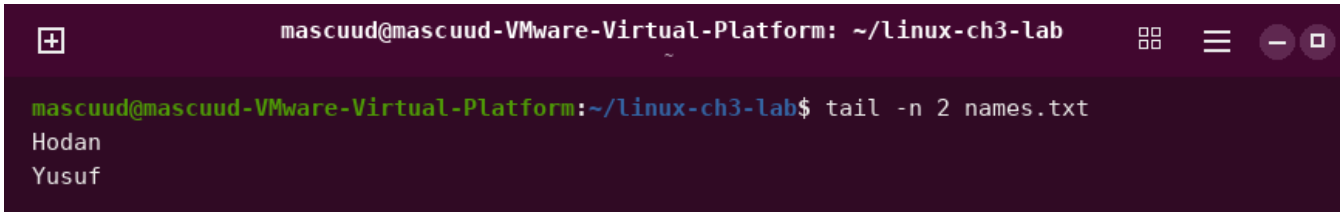
`head -n 2 names.txt`

Shows the first 2 lines of a file.



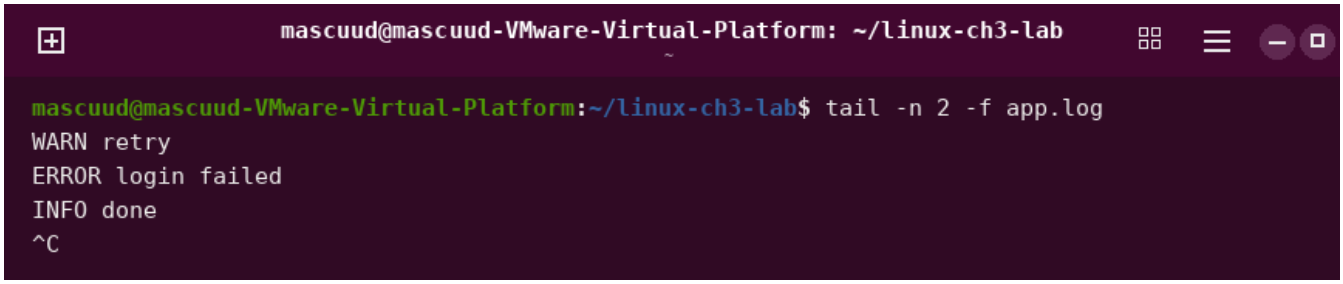
`tail -n 2 names.txt`

Shows the last 2 lines of a file.



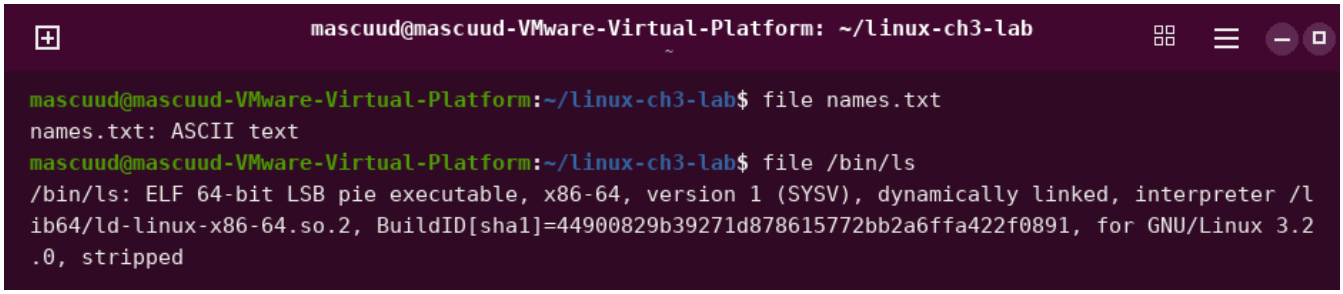
`tail -n 2 -f app.log`

Shows new lines at the end of a file as they come. Press Ctrl+C to stop.



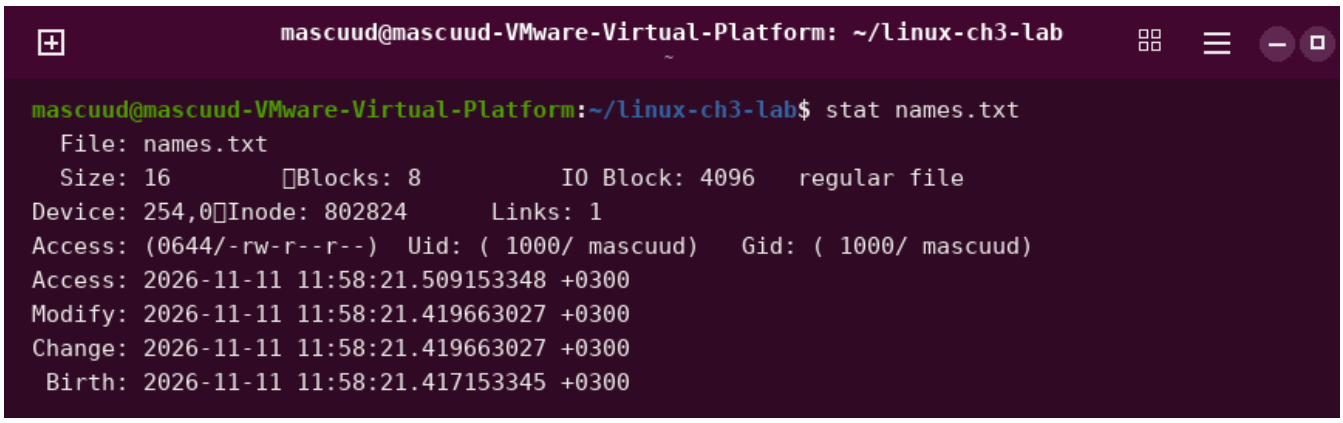
`file names.txt`

Tells you what type of file it is.



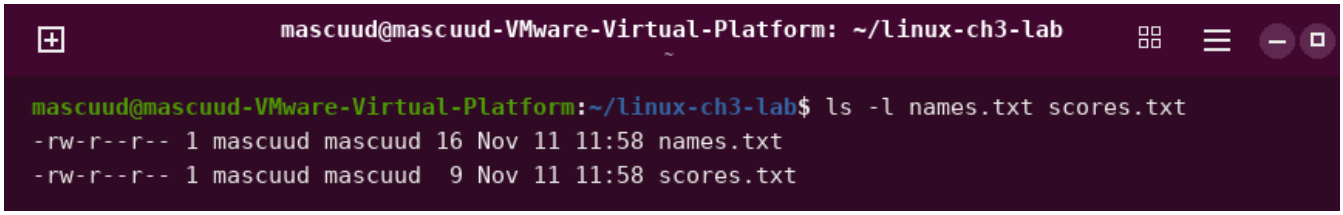
`stat names.txt`

Shows details about a file, like size and times.



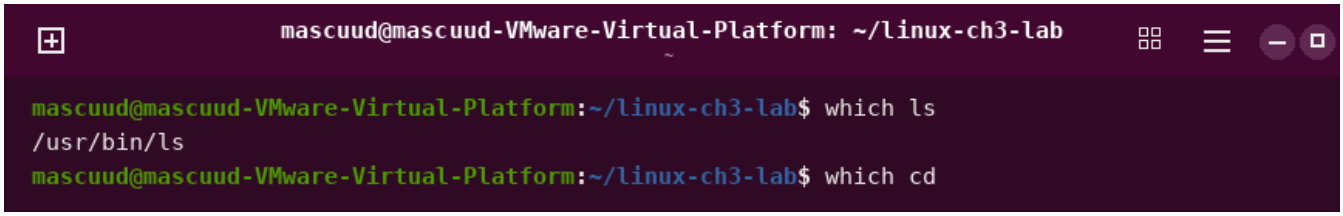
`ls -l`

Shows files with their rights, owner, size and date.



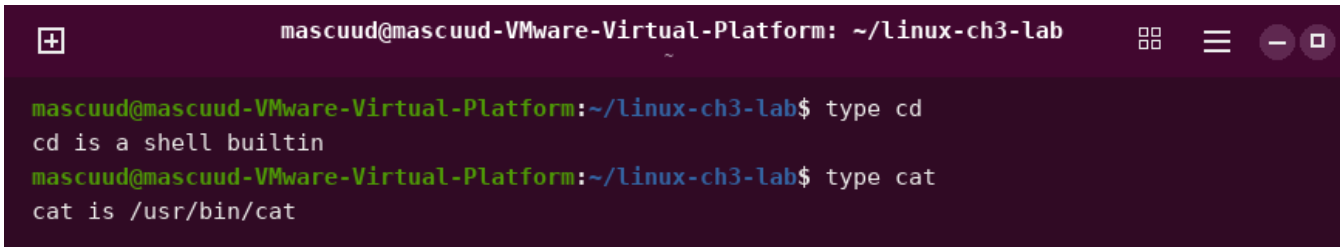
`which ls`

Shows where the program is.



`type cd`

Tells you what kind of command this is.



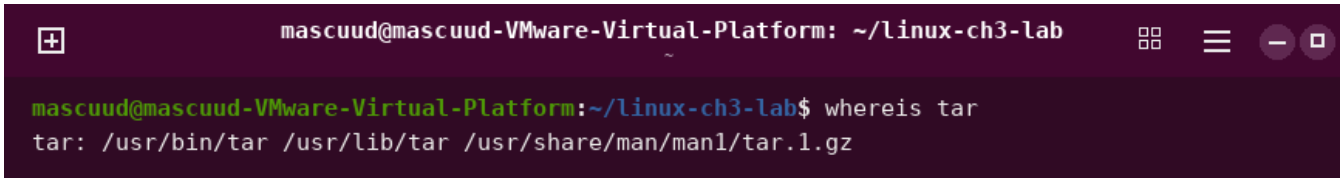
`command -v ls`

Shows where the command is.



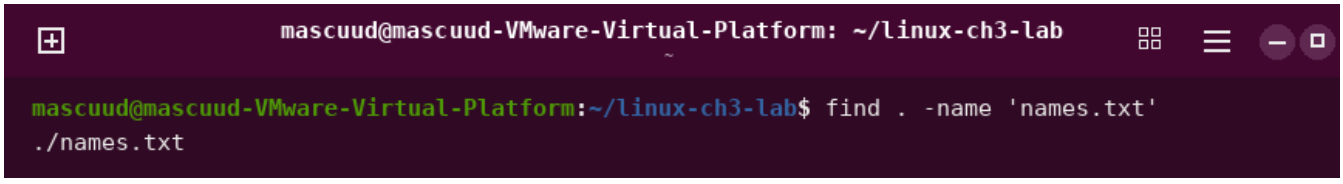
`whereis tar`

Shows where the program and its manual are.



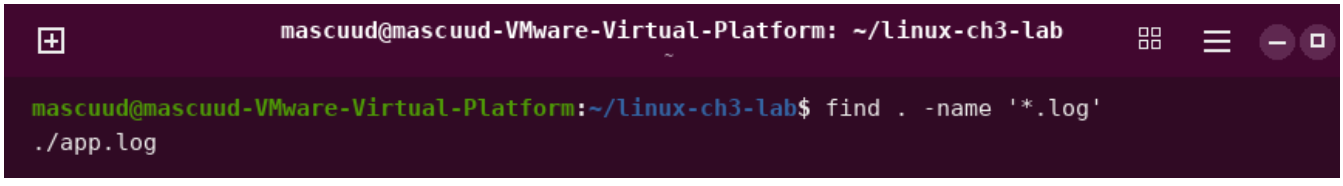
`find . -name 'names.txt'`

Looks for a file by its name.



`find . -name '*.log'`

Looks for files that end with .log.



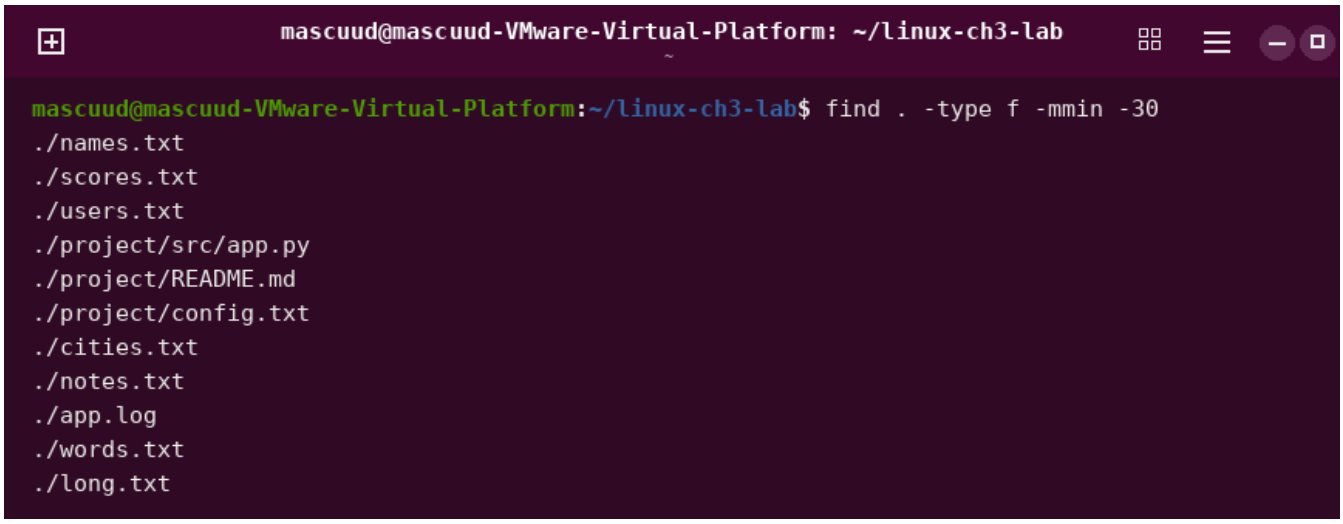
`find . -name 'names.txt' -size +10c`

Looks for a file by name that is bigger than 10 bytes.



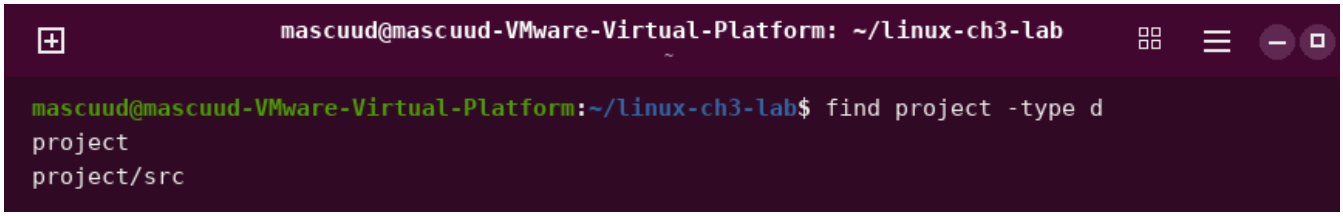
`find . -type f -mmin -30`

Looks for files that changed in the last 30 minutes.



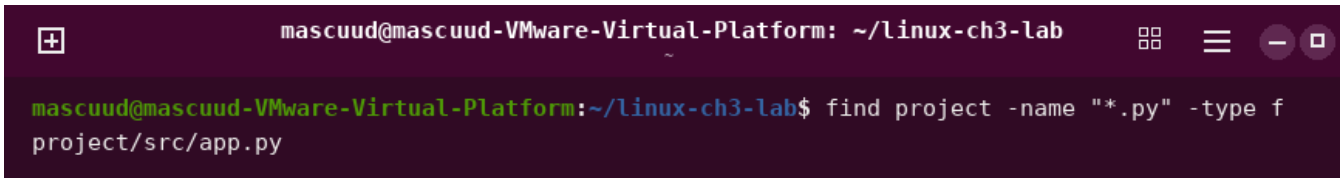
`find project -type d`

Looks for folders inside project.



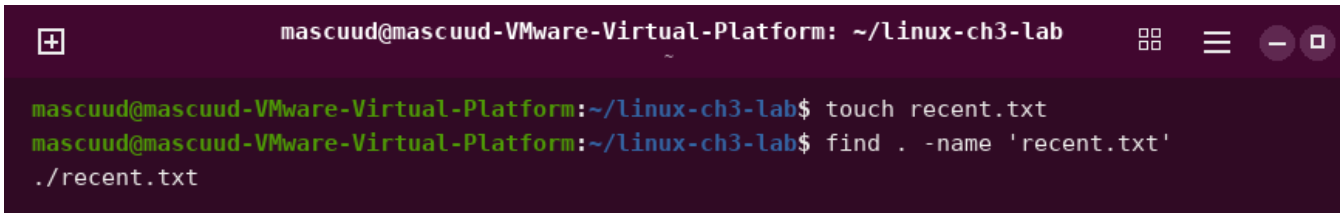
`find project -name "*.py" -type f`

Looks for .py files inside project.



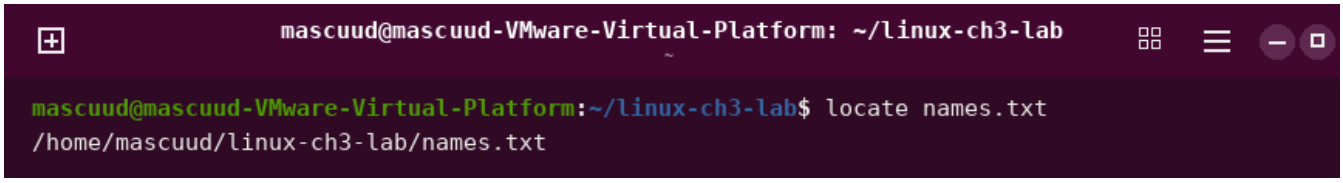
`touch recent.txt and find . -name 'recent.txt'`

Makes a new file, then finds it.



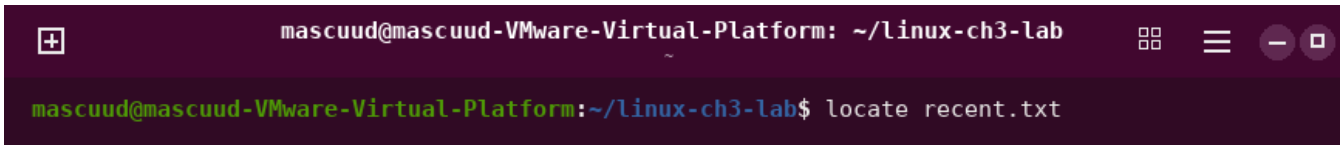
`locate names.txt`

Finds a file very fast by using a ready list.



`locate recent.txt`

May find nothing, because the ready list is old.



`grep 'Ali' names.txt`

Shows the lines that have the word Ali.



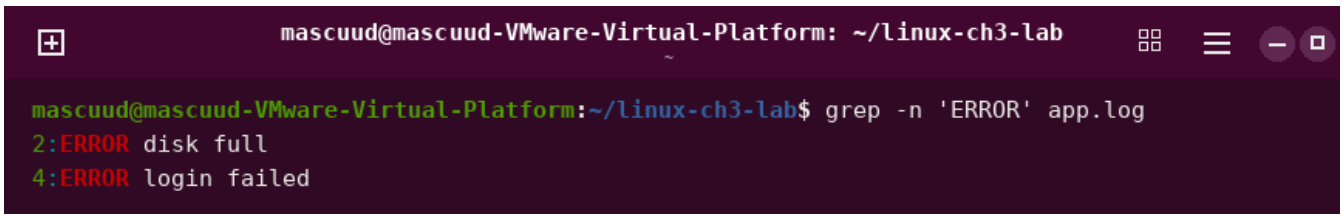
`grep -i 'error' app.log`

Shows lines with the word error. Big and small letters are the same.



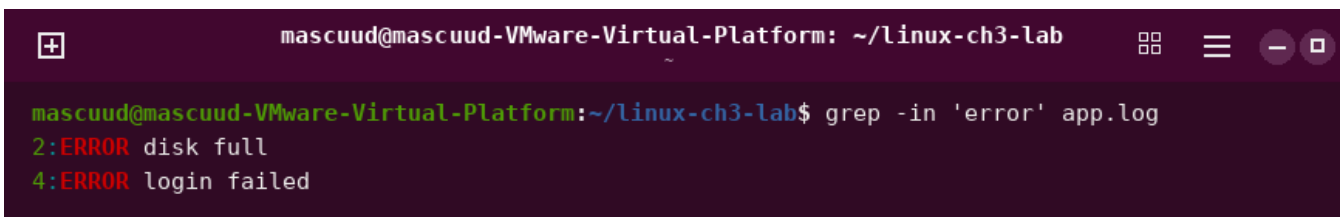
`grep -n 'ERROR' app.log`

Shows matching lines with their line numbers.



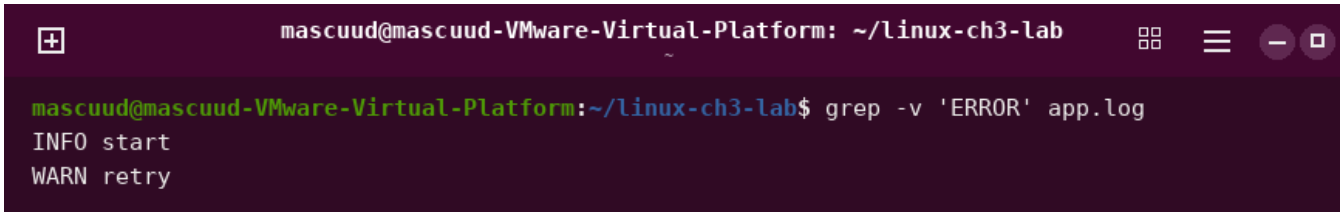
`grep -in 'error' app.log`

Shows matching lines with numbers. Big and small letters are the same.



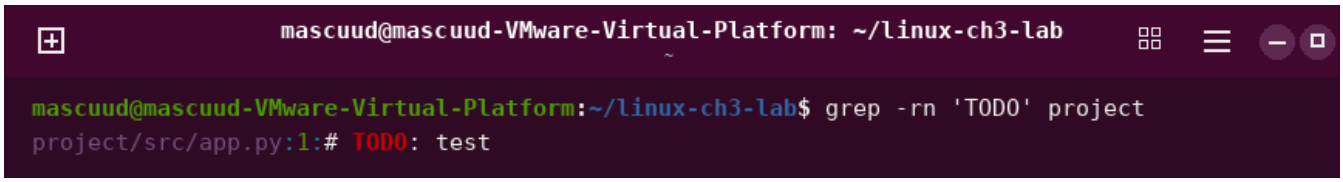
`grep -v 'ERROR' app.log`

Shows the lines that do not have the word.



`grep -rn 'TODO' project`

Looks for the word in all files inside the folder.



`grep '^Ali' names.txt`

Shows lines that start with Ali.



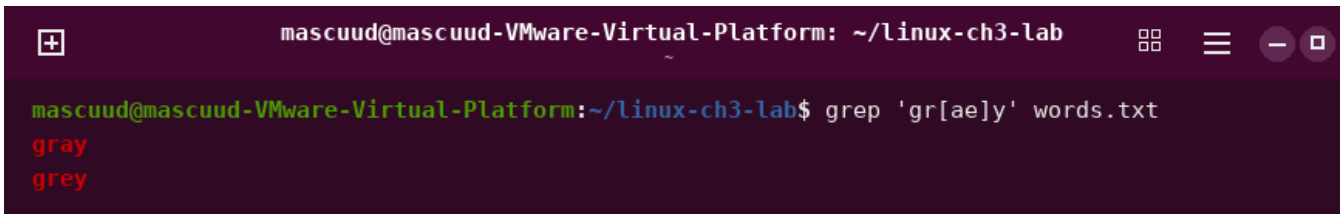
`grep 'failed$' app.log`

Shows lines that end with failed.



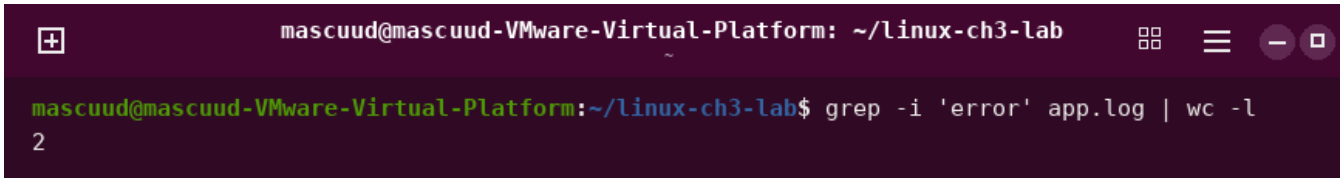
`grep 'gr[ae]y' words.txt`

Finds both gray and grey.



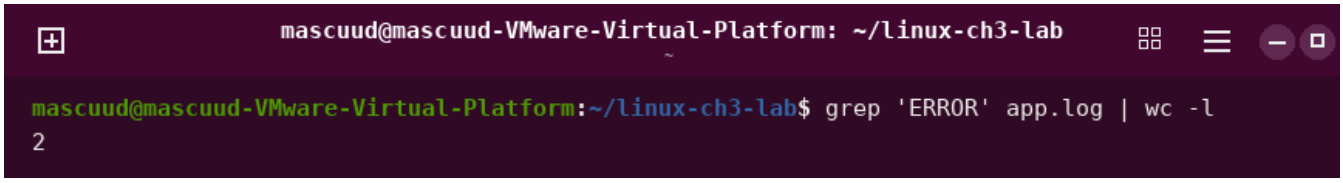
`grep -i 'error' app.log | wc -l`

Counts the lines that have the word error.



`grep 'ERROR' app.log | wc -l`

Counts the lines that have ERROR.



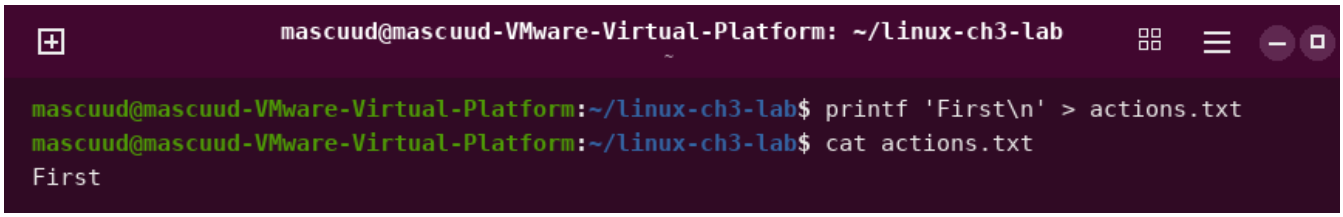
`grep 'Mogadishu' cities.txt | wc -l`

Counts the lines that have the word Mogadishu.



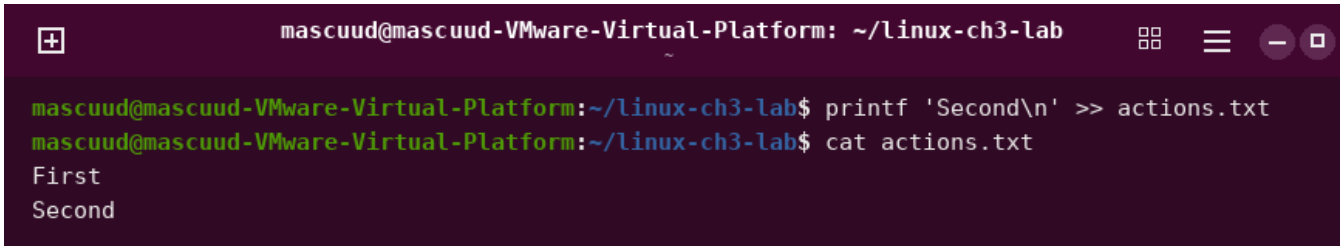
`printf 'First\n' > actions.txt`

Saves the word First in a file.



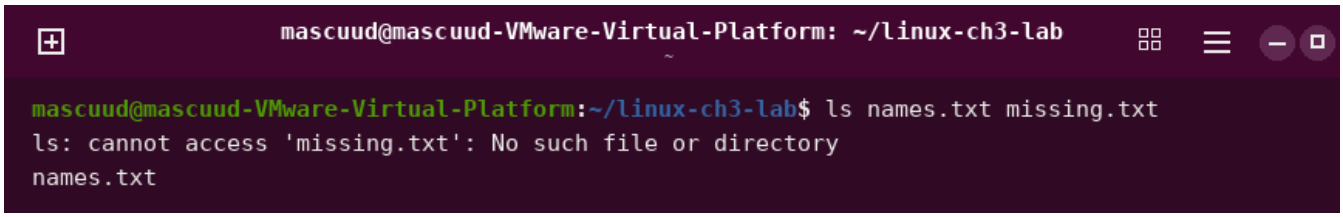
`printf 'Second\n' >> actions.txt`

Adds the word Second at the end of the file.



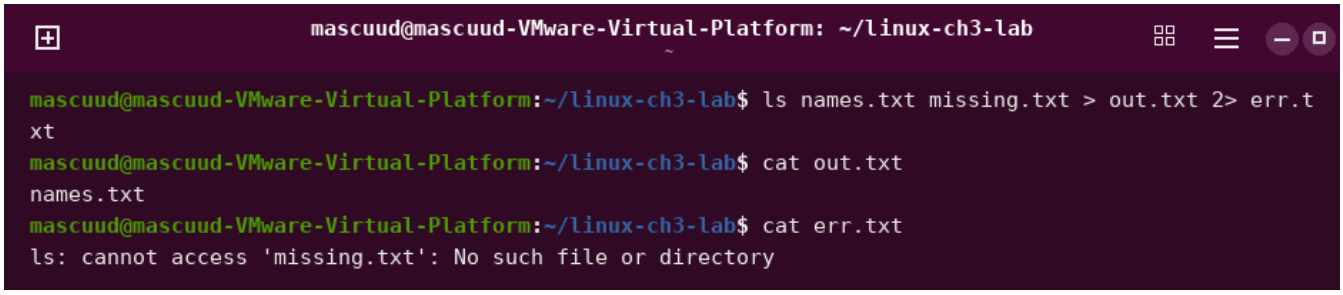
`ls names.txt missing.txt`

Shows the file name and an error for the missing file.



`ls names.txt missing.txt > out.txt 2> err.txt`

Saves the normal output in one file and the error in another file.



`wc -l < names.txt`

Counts the lines of a file by reading it as input.



`cat names.txt | wc -l`

Counts the lines by sending the file through a pipe.



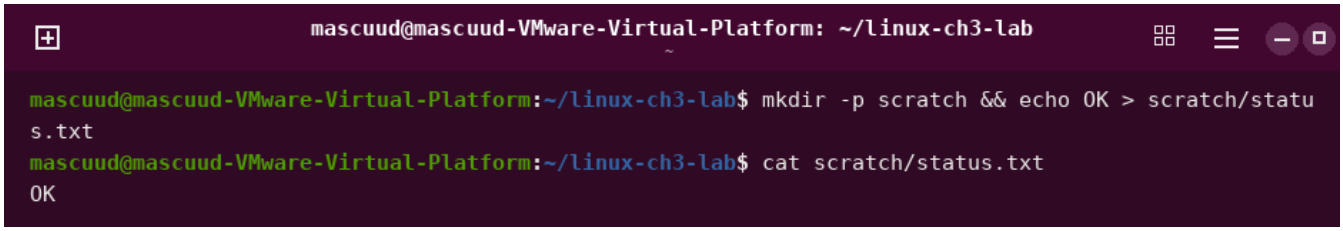
`printf 'one\n'; printf 'two\n'`

Runs two commands one after the other.



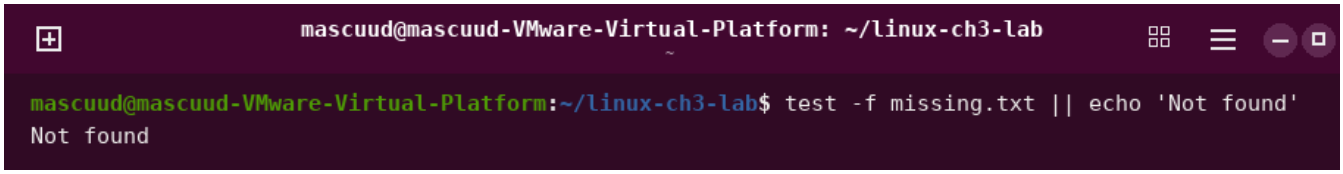
`mkdir -p scratch && echo OK > scratch/status.txt`

Makes a folder. Then, if it worked, it makes a file.



`test -f missing.txt || echo 'Not found'`

Prints Not found if the file does not exist.



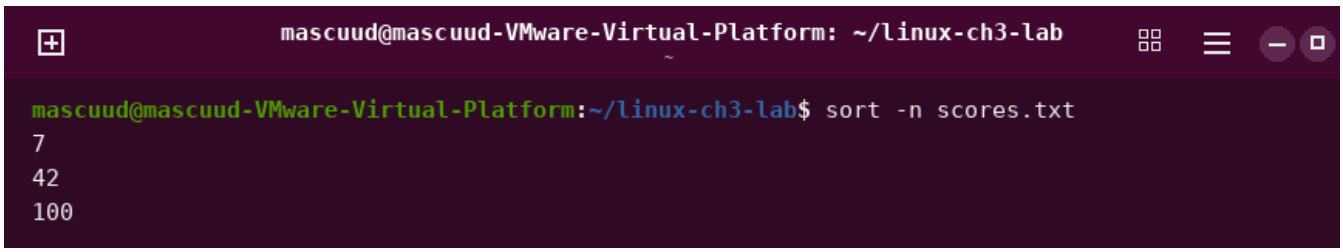
`sort scores.txt`

Puts the lines in order, like a dictionary.



`sort -n scores.txt`

Puts numbers in order from small to big.



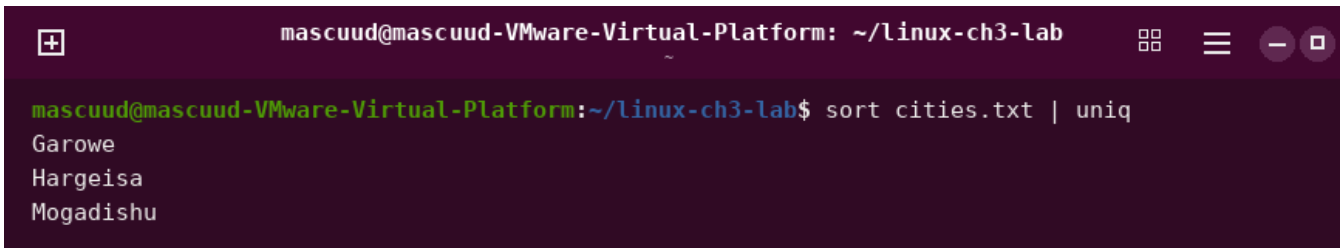
`sort -rn scores.txt`

Puts numbers in order from big to small.



`sort cities.txt | uniq`

Removes lines that repeat.



`sort cities.txt | uniq -c`

Counts how many times each line shows.

```
mascuud@mascuud-VMware-Virtual-Platform: ~/linux-ch3-lab
~
mascuud@mascuud-VMware-Virtual-Platform:~/linux-ch3-lab$ sort cities.txt | uniq -c
      1 Garowe
      1 Hargeisa
      3 Mogadishu
```

`cut -d: -f1 users.txt`

Shows the first part of each line. The parts are split by a colon.

```
mascuud@mascuud-VMware-Virtual-Platform: ~/linux-ch3-lab
~
mascuud@mascuud-VMware-Virtual-Platform:~/linux-ch3-lab$ cut -d: -f1 users.txt
ali
hodan
yusuf
```

`cut -d: -f 1,3 users.txt`

Shows part 1 and part 3 of each line.

```
mascuud@mascuud-VMware-Virtual-Platform: ~/linux-ch3-lab
~
mascuud@mascuud-VMware-Virtual-Platform:~/linux-ch3-lab$ cut -d: -f 1,3 users.txt
ali:bash
hodan:sh
yusuf:bash
```

`cut -d: -f3 users.txt | sort`

Shows part 3 of each line, in order.

```
mascuud@mascuud-VMware-Virtual-Platform: ~/linux-ch3-lab
~
mascuud@mascuud-VMware-Virtual-Platform:~/linux-ch3-lab$ cut -d: -f3 users.txt | sort
bash
bash
sh
```

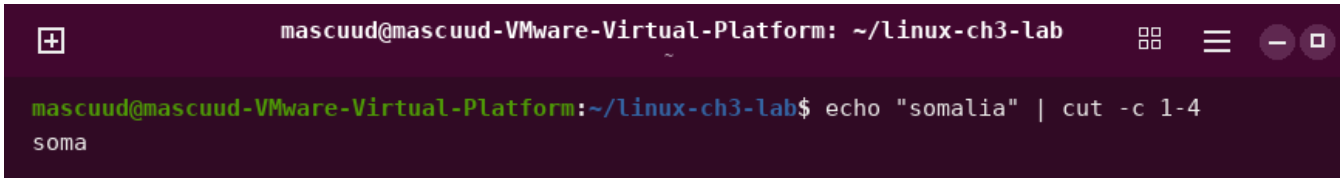
`cut -d: -f3 users.txt | sort | uniq -c`

Counts how many times each part 3 value shows.

```
mascuud@mascuud-VMware-Virtual-Platform: ~/linux-ch3-lab
~
mascuud@mascuud-VMware-Virtual-Platform:~/linux-ch3-lab$ cut -d: -f3 users.txt | sort | uniq -c
      2 bash
      1 sh
```

`echo "somalia" | cut -c 1-4`

Shows letters 1 to 4 of the word.



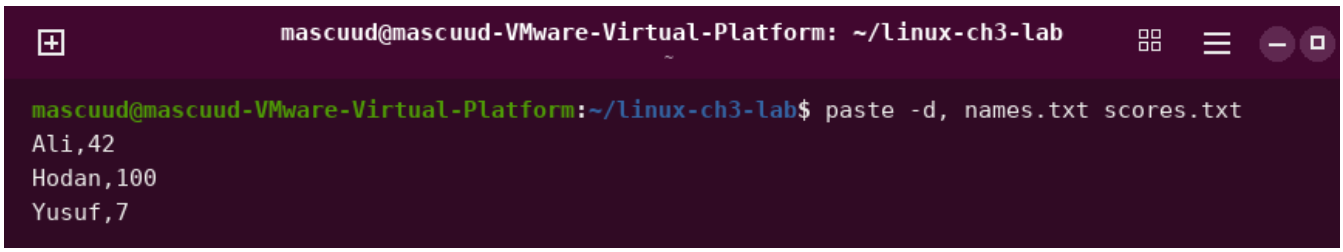
`paste names.txt scores.txt`

Puts two files side by side, line by line.



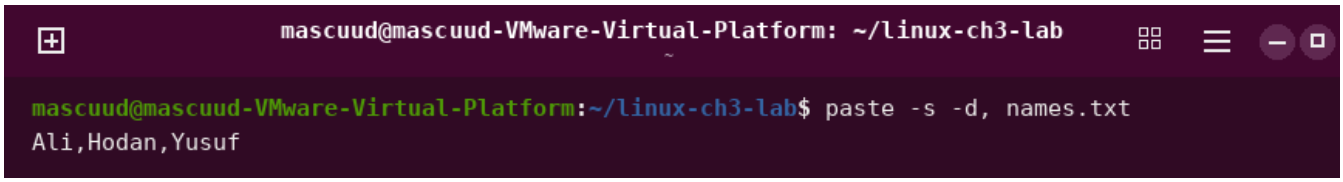
`paste -d, names.txt scores.txt`

Puts two files side by side with a comma between them.



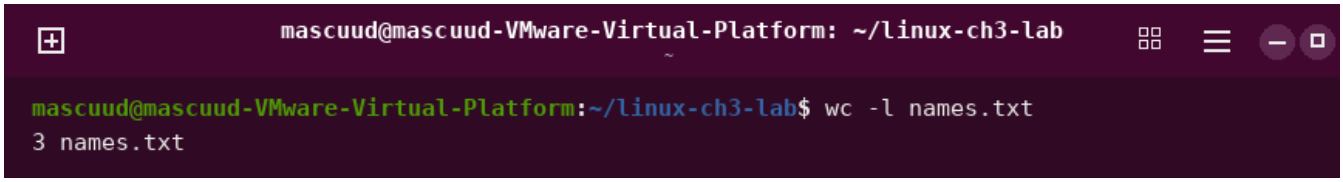
`paste -s -d, names.txt`

Puts all lines of a file on one line.



`wc -l names.txt`

Counts the lines in a file.



`wc -w notes.txt`

Counts the words in a file.



`tr 'a-z' 'A-Z' < names.txt`

Changes small letters to big letters.



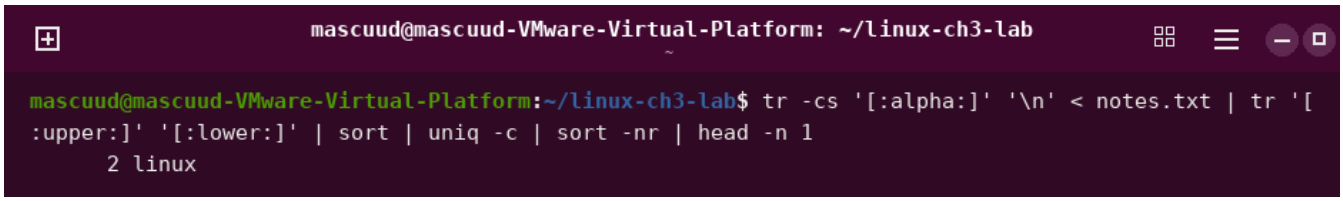
`echo 'too many spaces' | tr -s ' '`

Changes many spaces into one space.



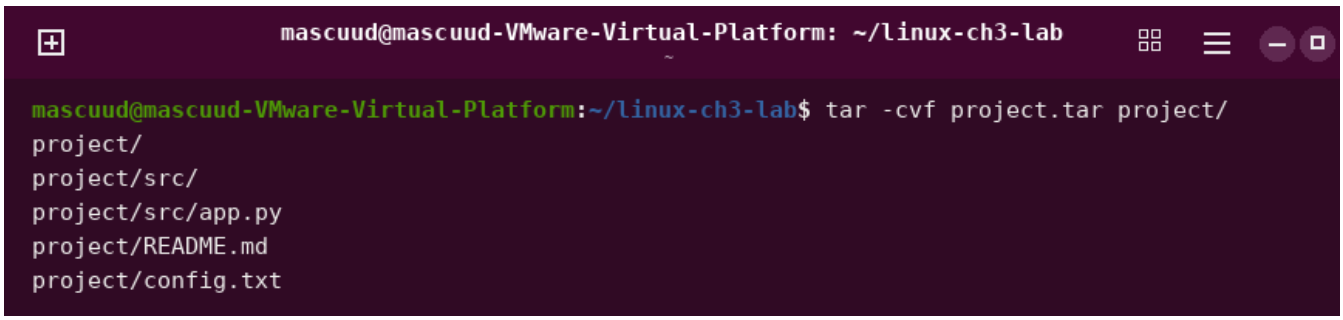
`tr -cs '[:alpha:]' '\n' < notes.txt | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -n 1`

Finds the word that shows most in a file.



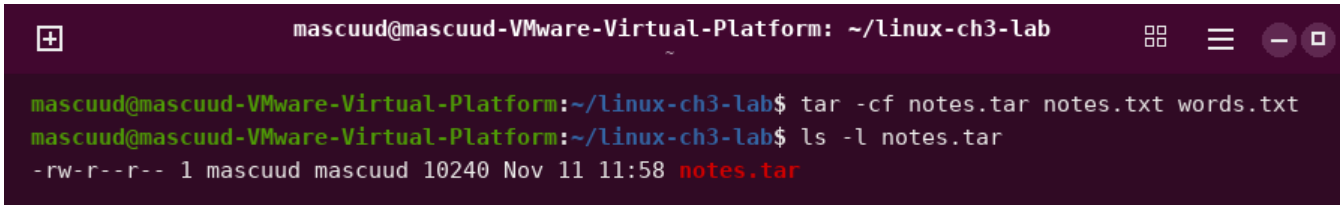
`tar -cvf project.tar project/`

Puts a folder into one file called an archive.



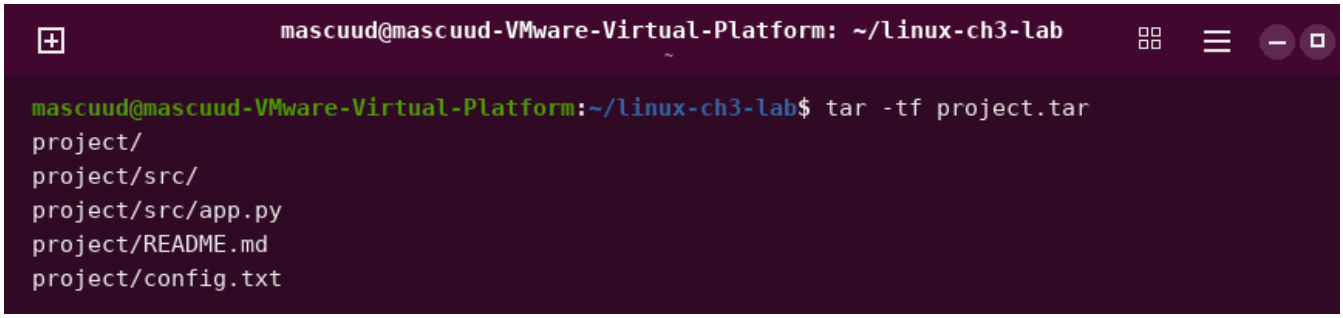
`tar -cf notes.tar notes.txt words.txt`

Puts two files into one archive.



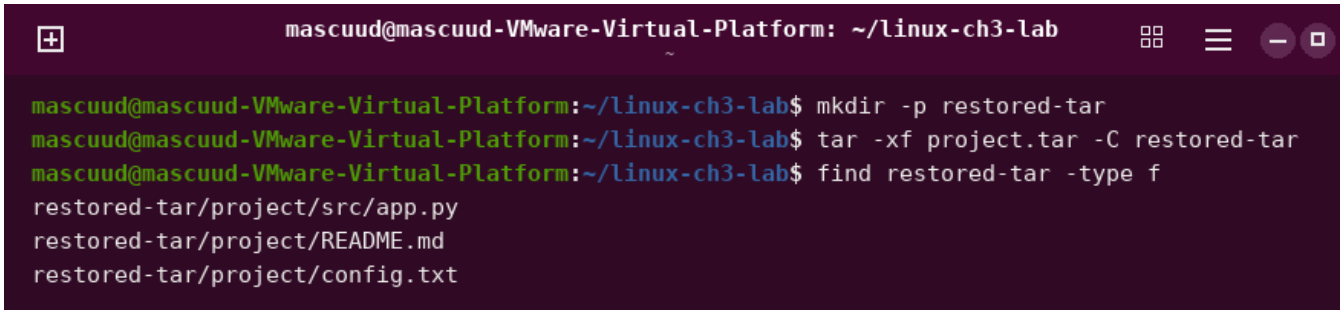
`tar -tf project.tar`

Shows what is inside an archive.



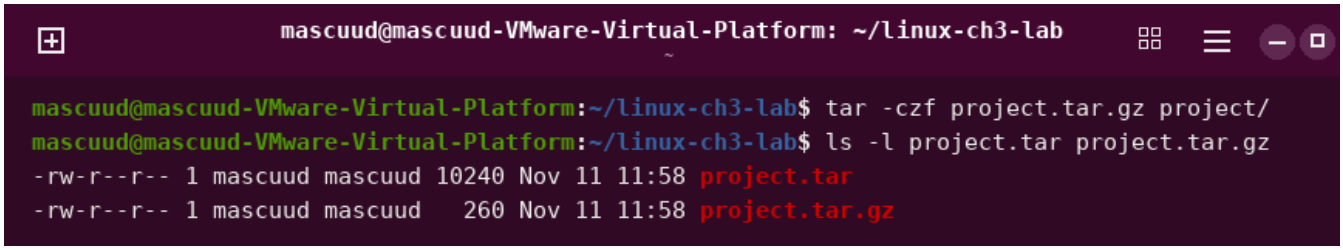
`tar -xf project.tar -C restored-tar`

Opens an archive into a folder.



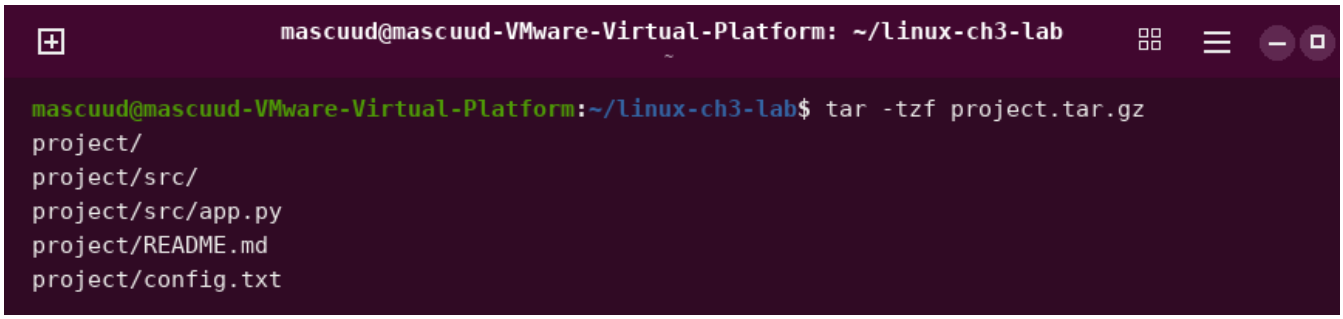
`tar -czf project.tar.gz project/`

Makes an archive and makes it smaller with gzip.



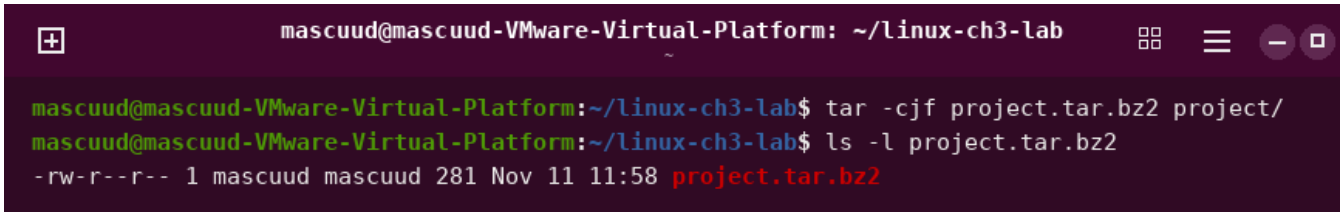
`tar -tzf project.tar.gz`

Shows what is inside a gzip archive.



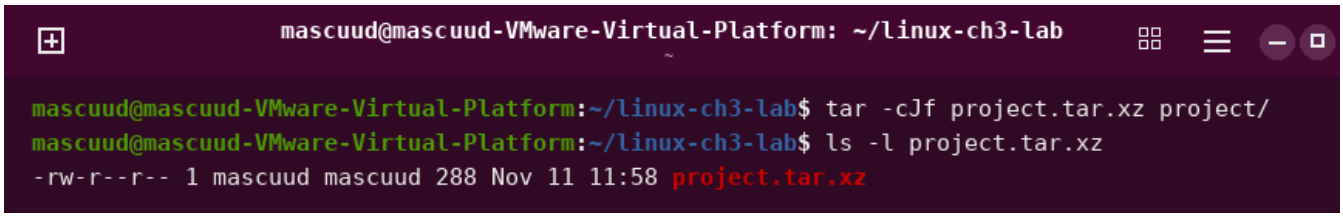
`tar -cjf project.tar.bz2 project/`

Makes an archive and makes it smaller with bzip2.



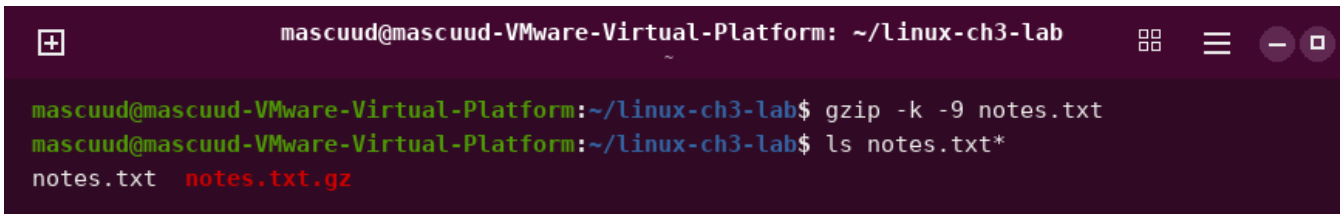
`tar -cJf project.tar.xz project/`

Makes an archive and makes it smaller with xz.



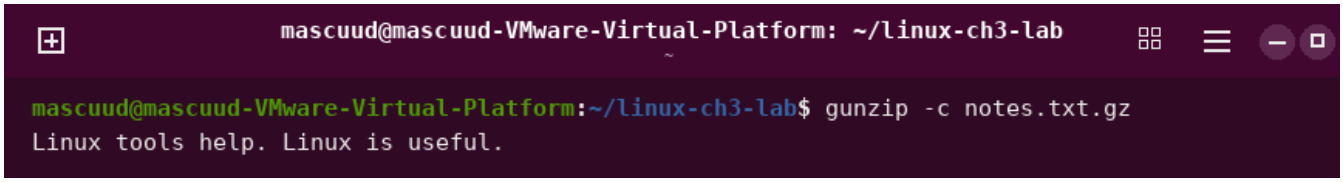
`gzip -k -9 notes.txt`

Makes a file smaller with gzip. It keeps the old file.



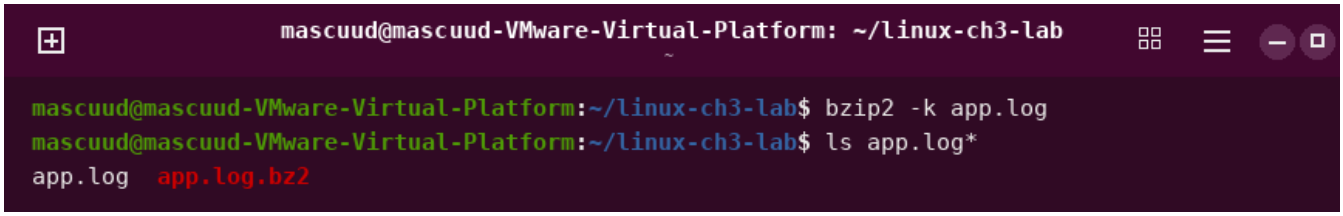
`gunzip -c notes.txt.gz`

Shows the text inside a .gz file.



`bzip2 -k app.log`

Makes a file smaller with bzip2. It keeps the old file.



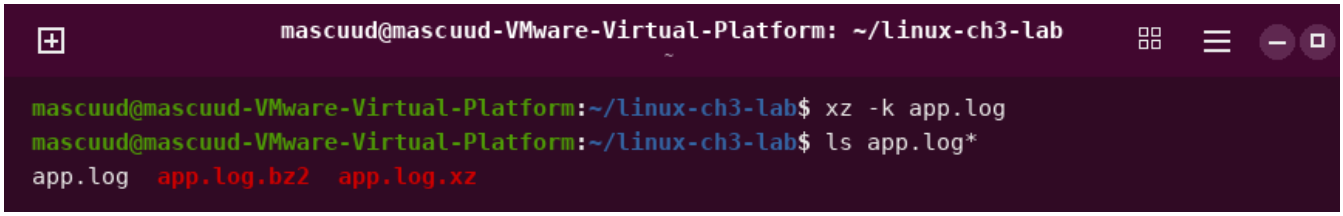
`bzcat app.log.bz2 | head -n 1`

Shows the text inside a .bz2 file. We show only the first line.



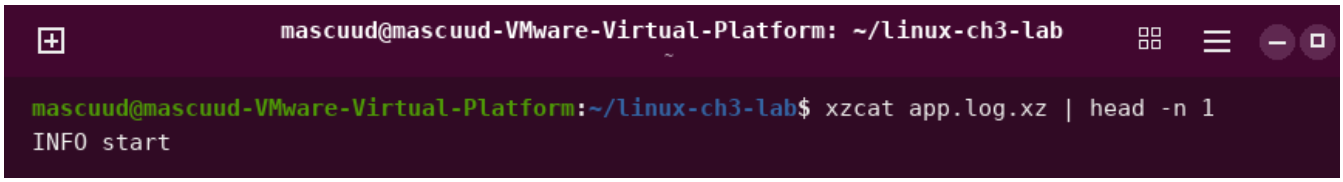
`xz -k app.log`

Makes a file smaller with xz. It keeps the old file.



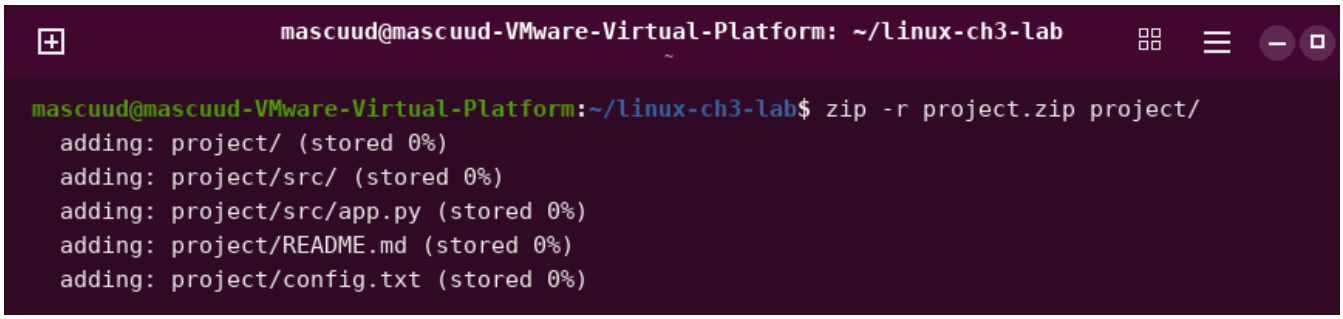
`xzcat app.log.xz | head -n 1`

Shows the text inside an .xz file. We show only the first line.



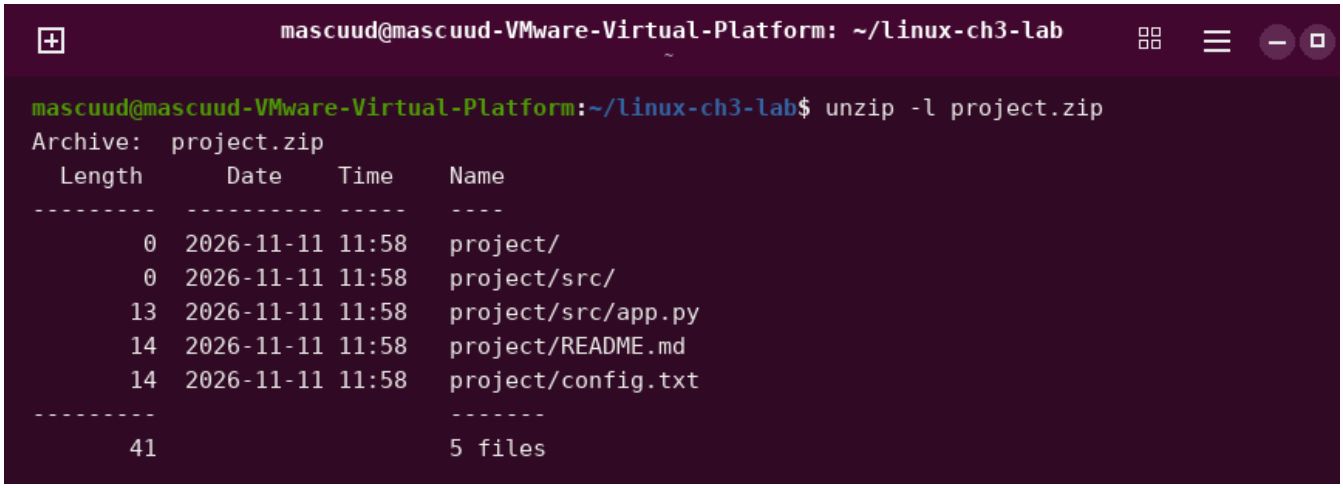
`zip -r project.zip project/`

Puts a folder into a zip file.



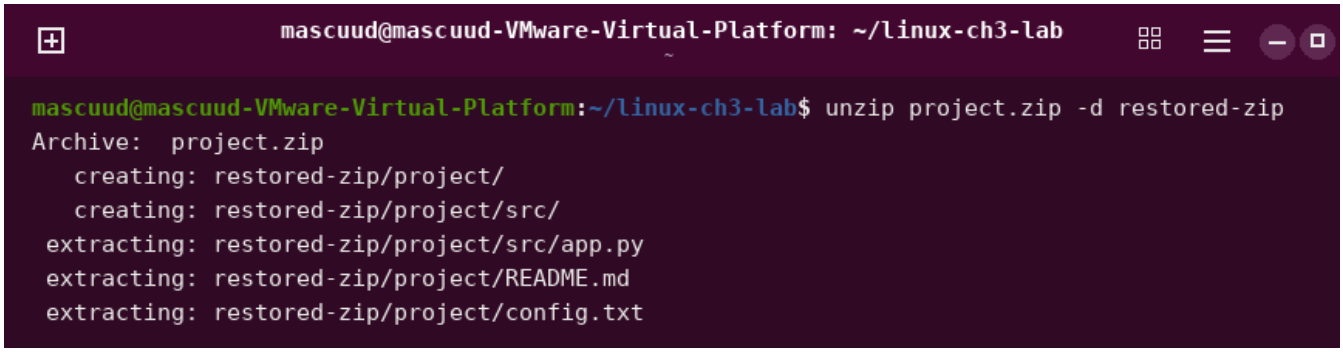
`unzip -l project.zip`

Shows what is inside a zip file.



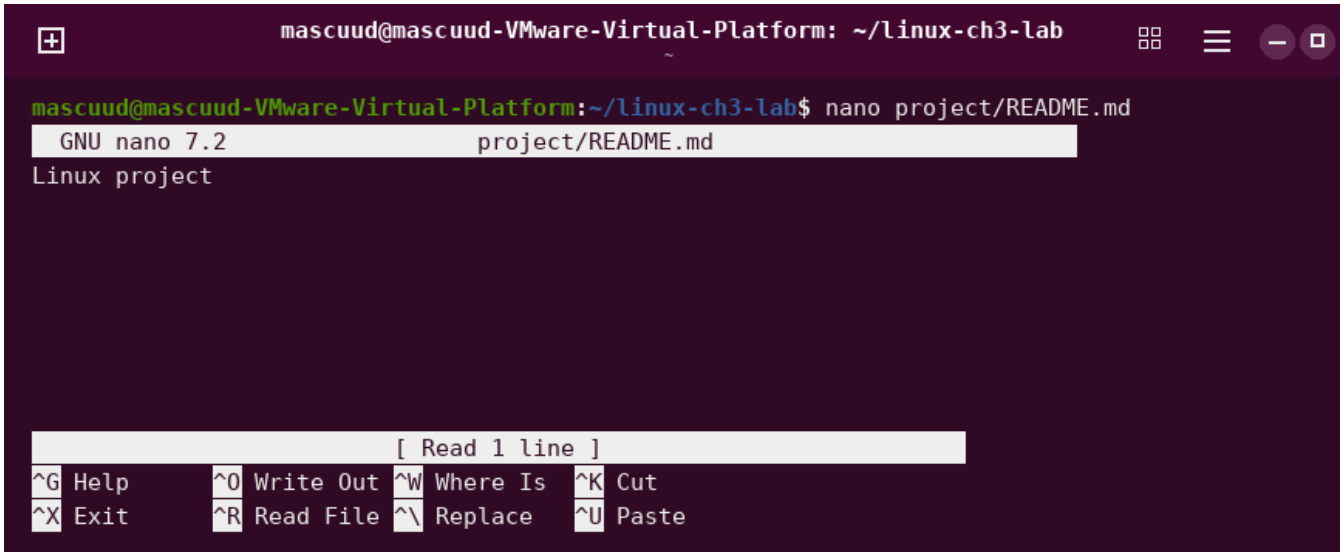
`unzip project.zip -d restored-zip`

Opens a zip file into a folder.



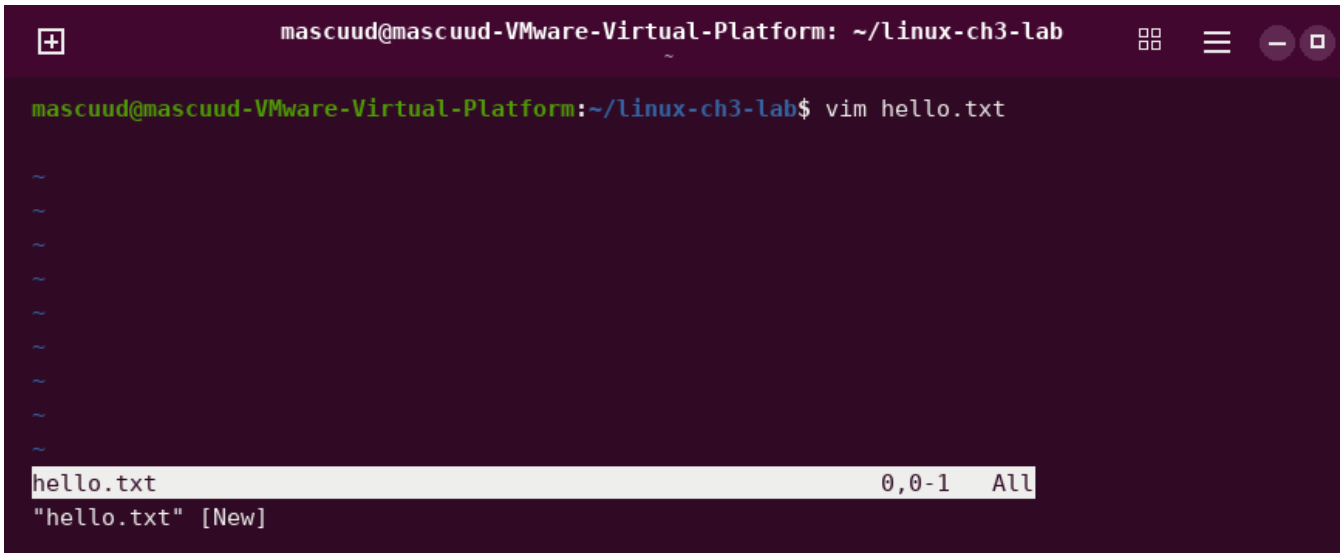
`nano project/README.md`

Opens a simple editor. Press Ctrl+O to save and Ctrl+X to exit.



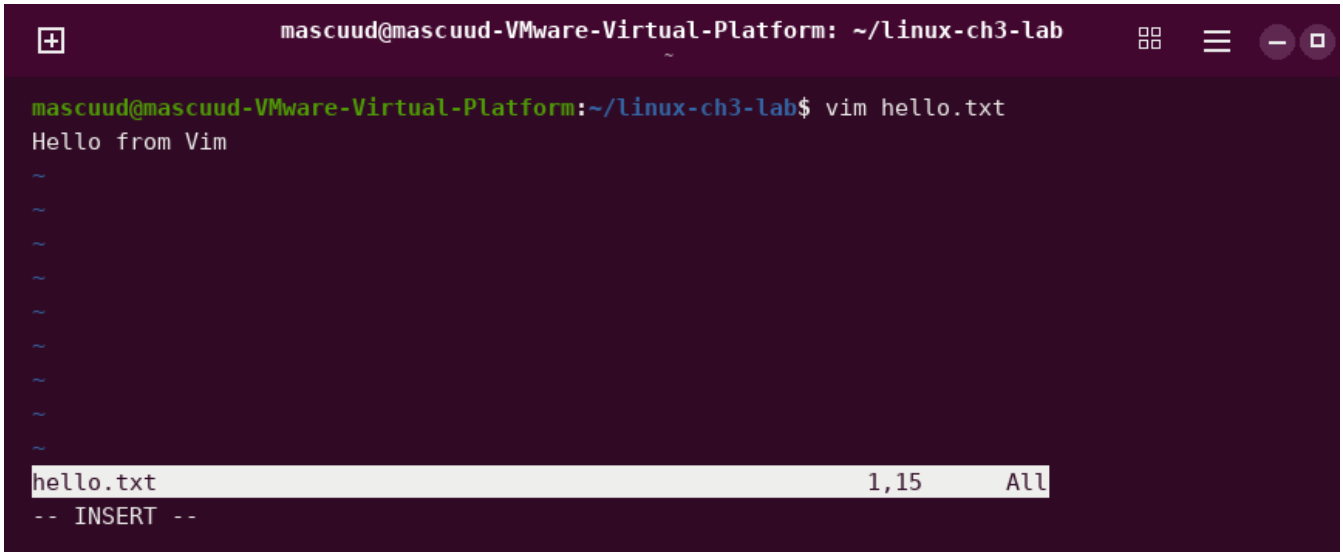
`vim hello.txt`

Opens a file in the Vim editor.



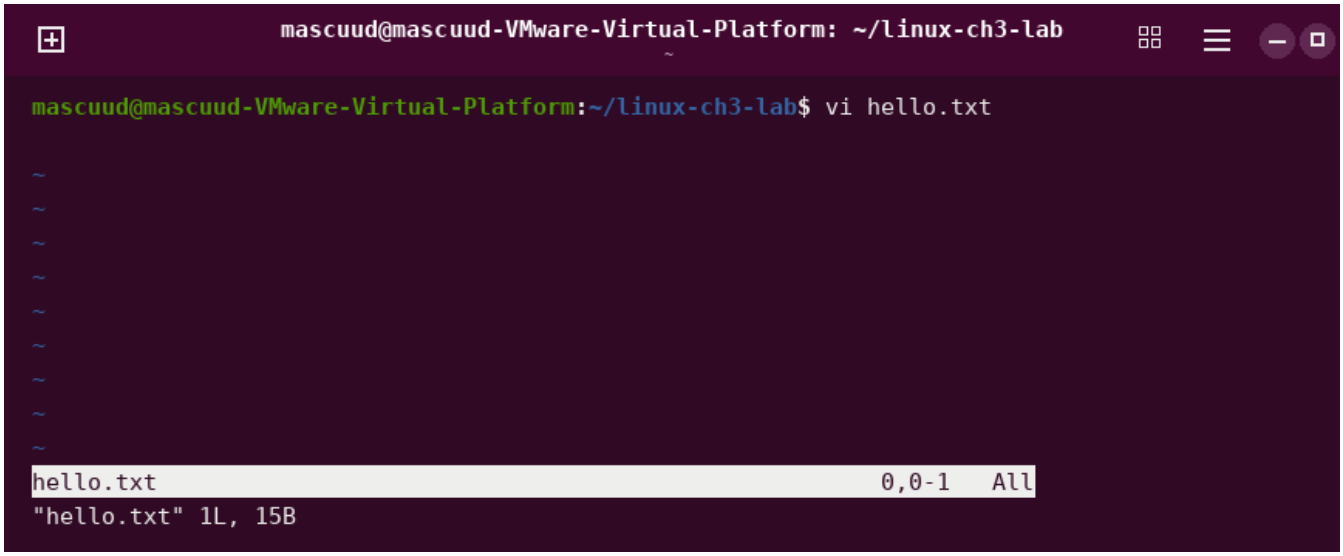
`vim hello.txt`

Press i to type text in Vim.



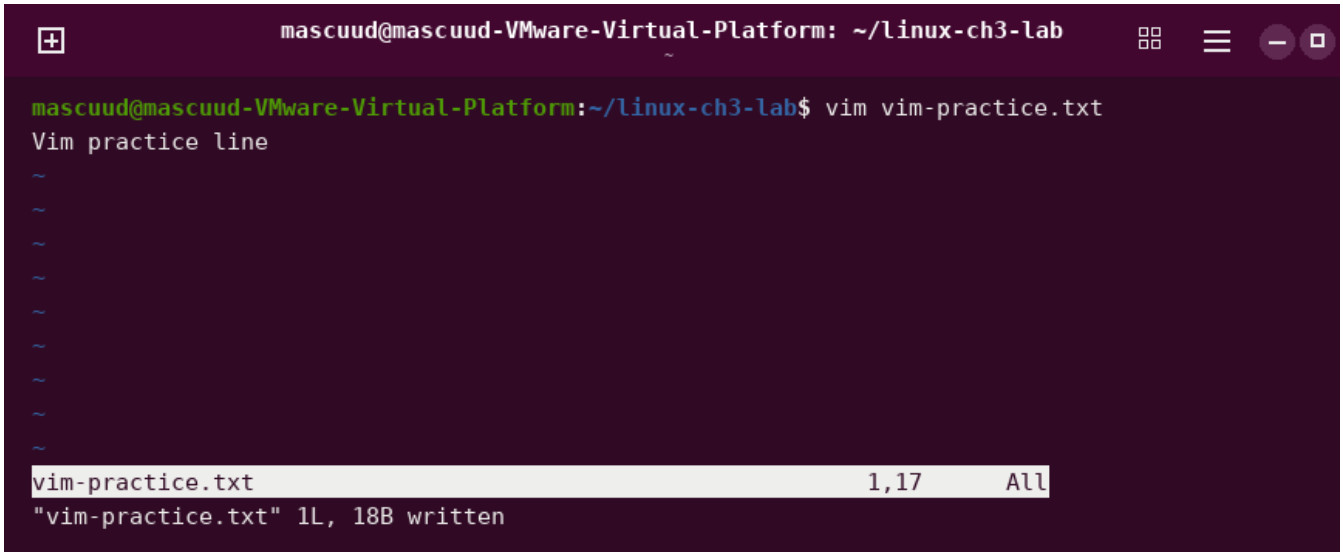
`vi hello.txt`

Opens a file in the vi editor. It works like Vim.



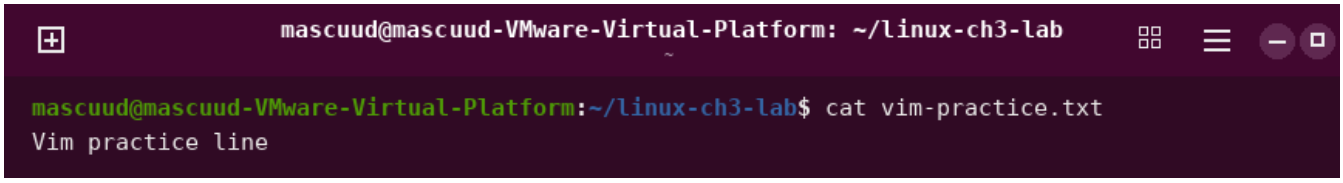
`vim vim-practice.txt`

Opens a practice file in Vim.



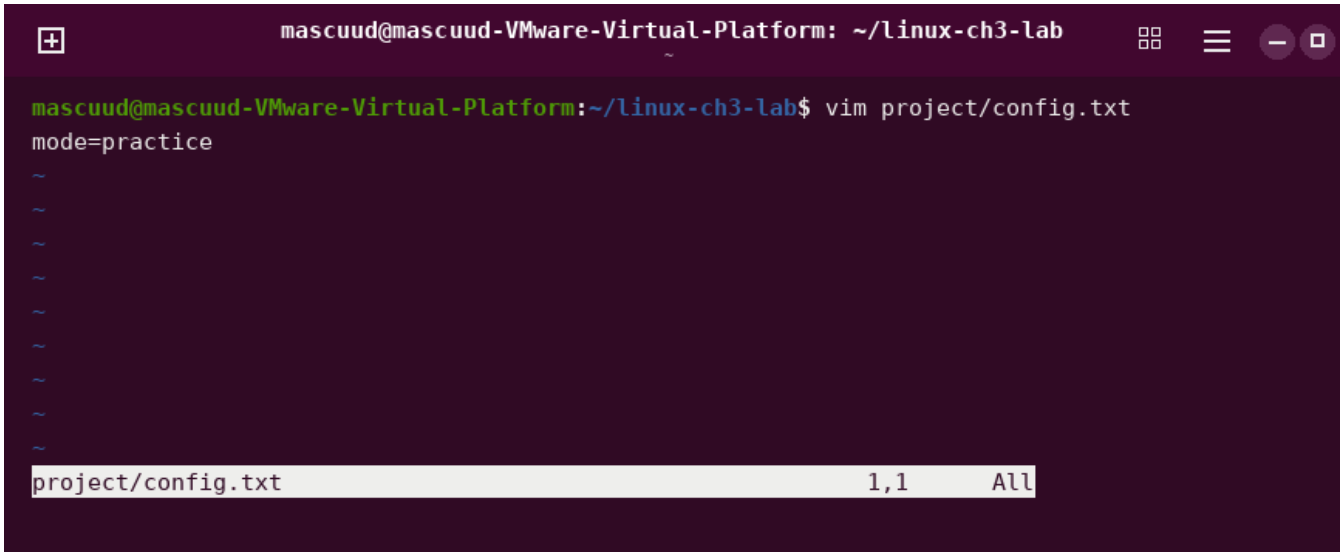
`cat vim-practice.txt`

Shows the text you saved.



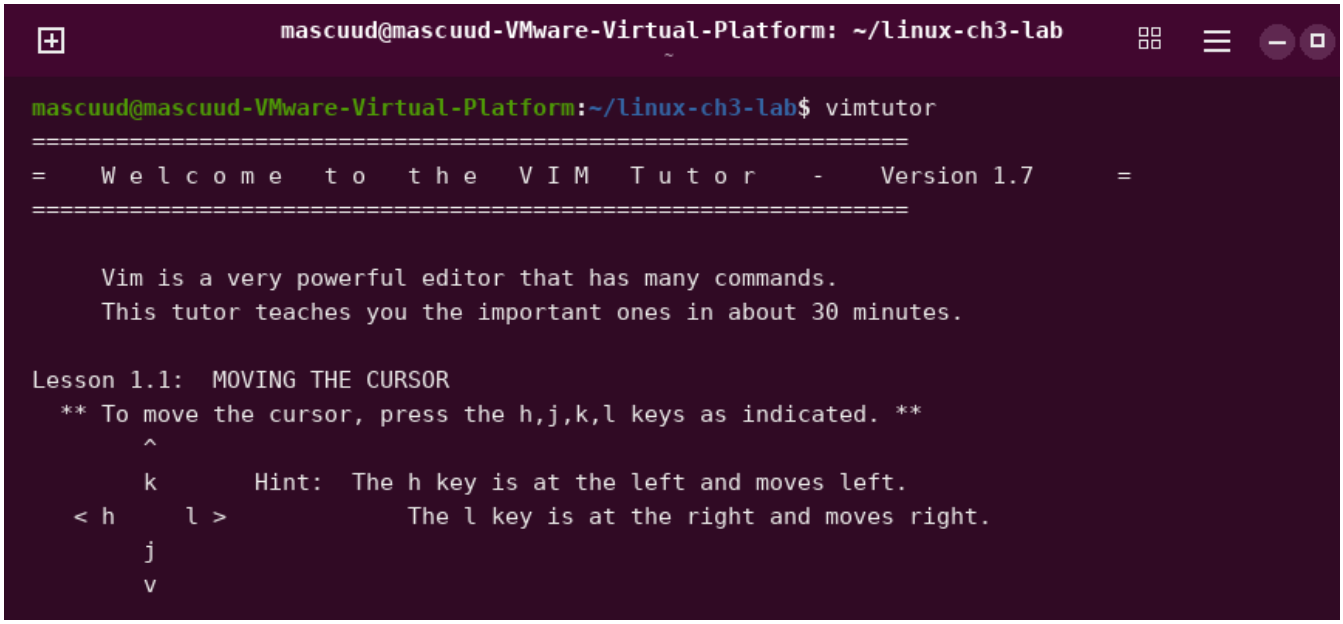
`vim project/config.txt`

Opens a file in Vim. Type :q! to leave without saving.



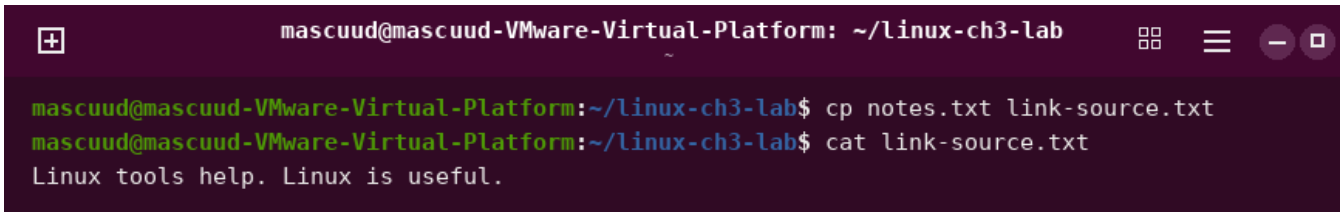
`vimtutor`

Opens a lesson to learn Vim.



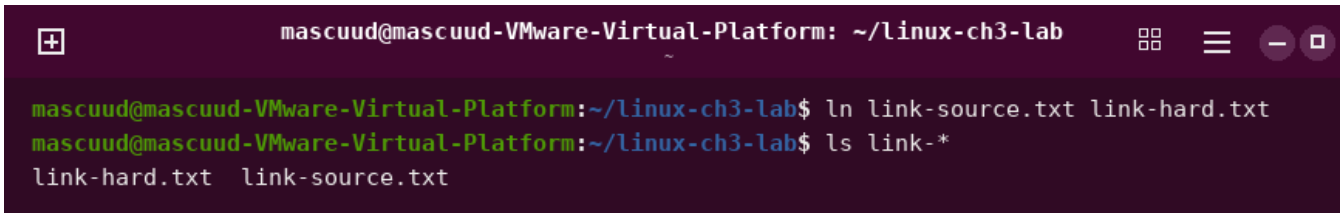
`cp notes.txt link-source.txt`

Makes a copy of a file to use for links.



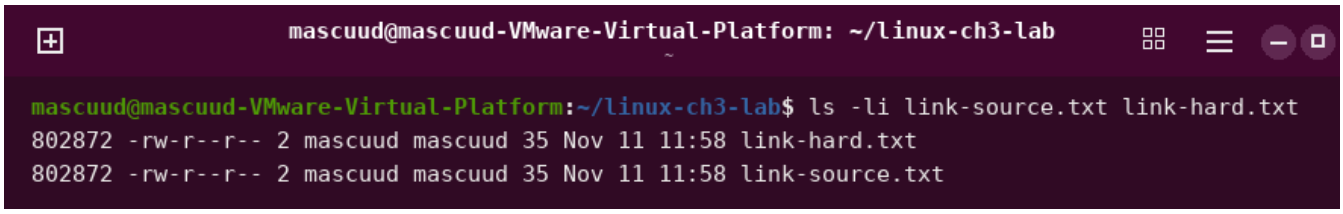
`ln link-source.txt link-hard.txt`

Makes a hard link. It is a second name for the same file.



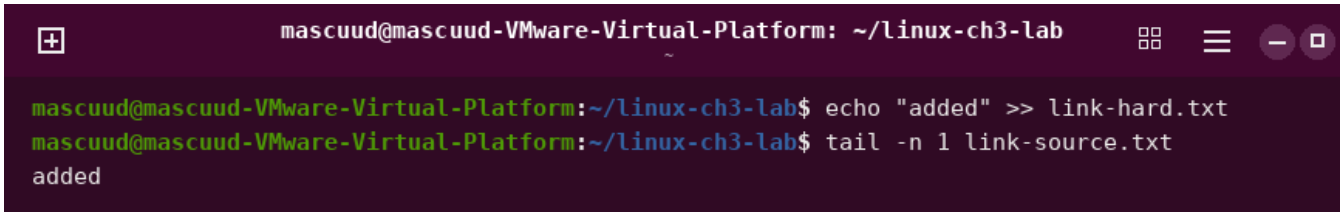
`ls -li link-source.txt link-hard.txt`

Shows the inode numbers. A hard link has the same number.



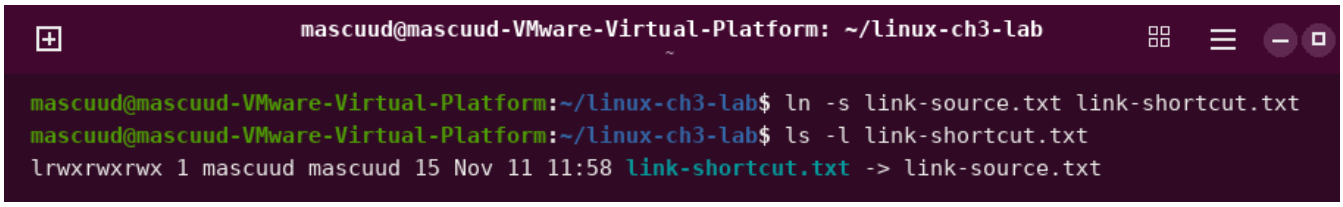
`echo "added" >> link-hard.txt`

Adds a line to the file. Both names show it.



`ln -s link-source.txt link-shortcut.txt`

Makes a symbolic link. It is like a shortcut.



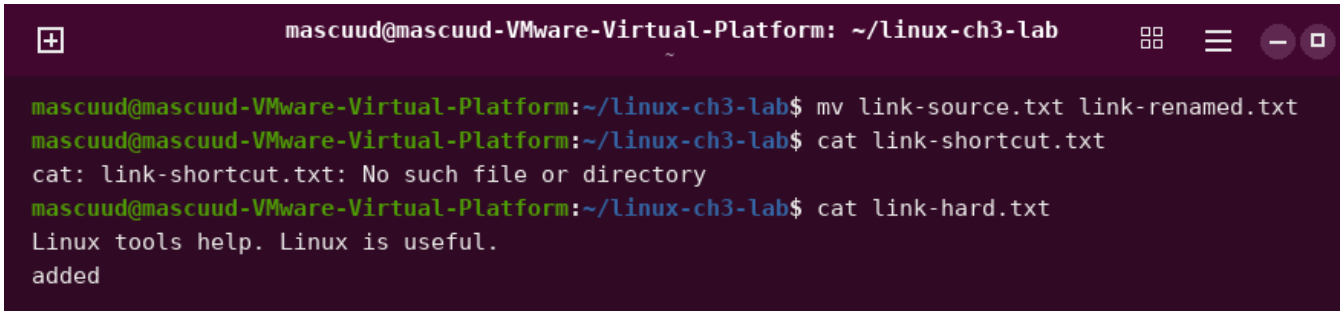
`cat link-shortcut.txt`

Shows the file that the shortcut points to.



`mv link-source.txt link-renamed.txt`

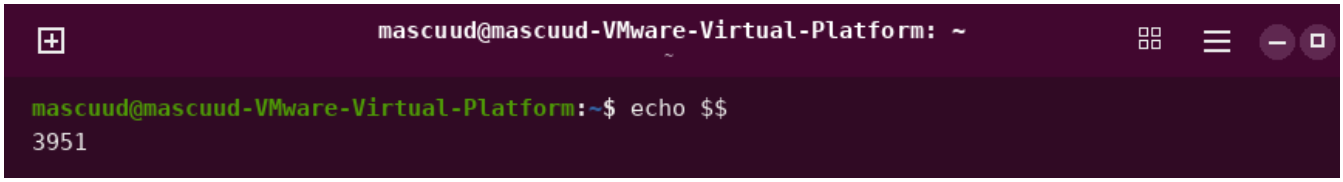
Renames the file. The shortcut stops working.



Chapter Four

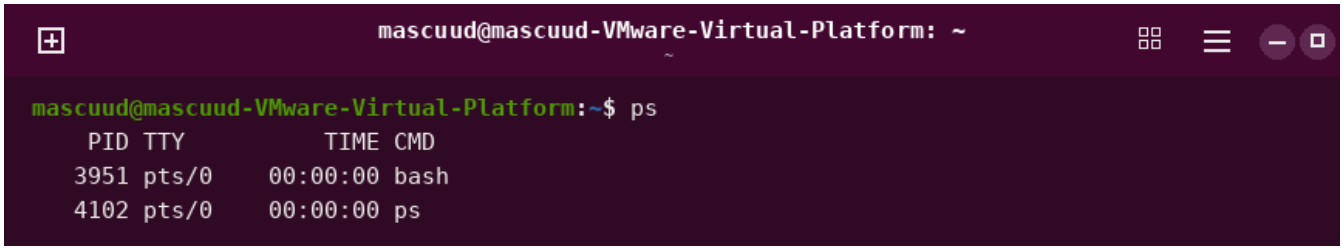
`echo $$`

Shows the process number of your shell.



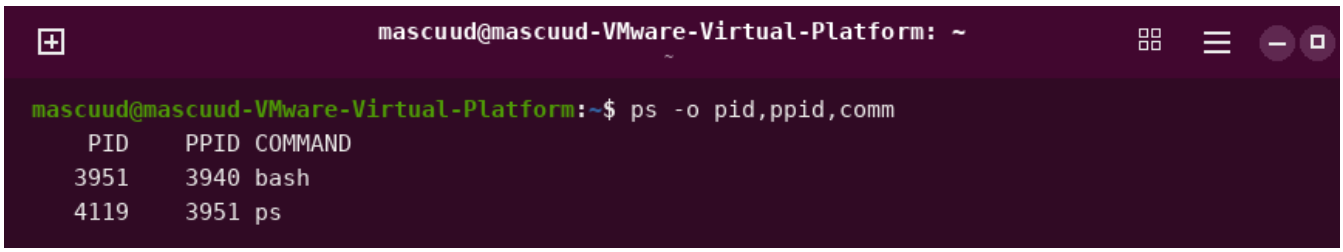
ps

Shows the processes in your terminal.



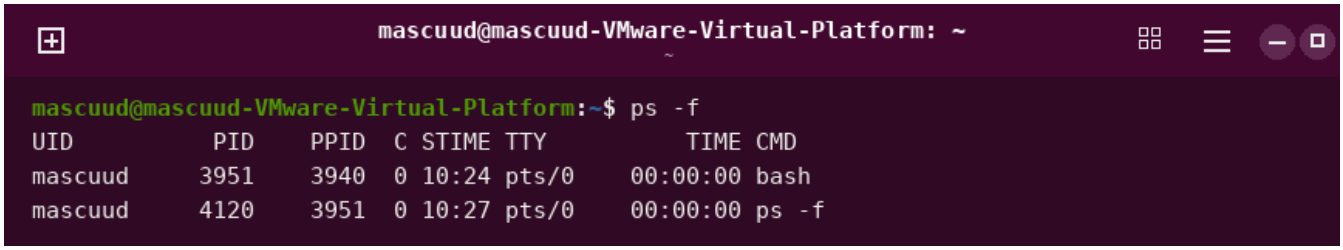
ps -o pid,ppid,comm

Shows the process number, the parent number and the name.



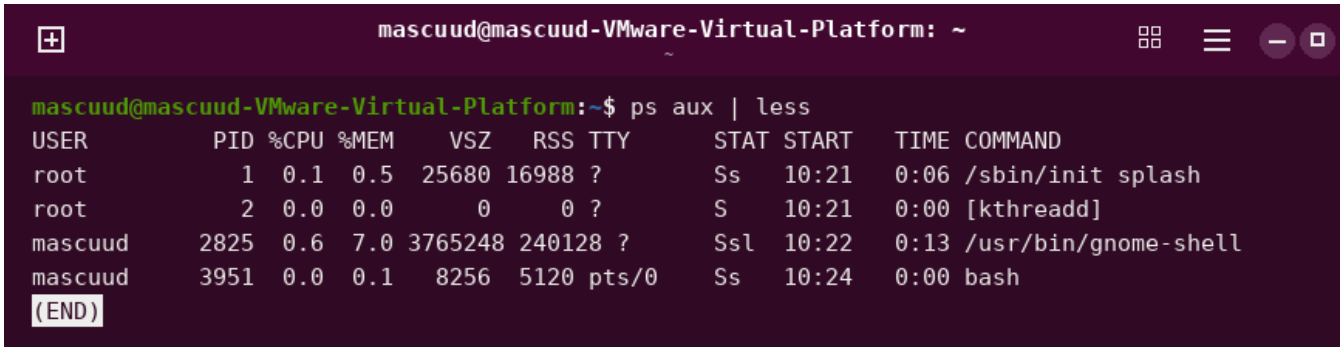
ps -f

Shows a longer list of processes.



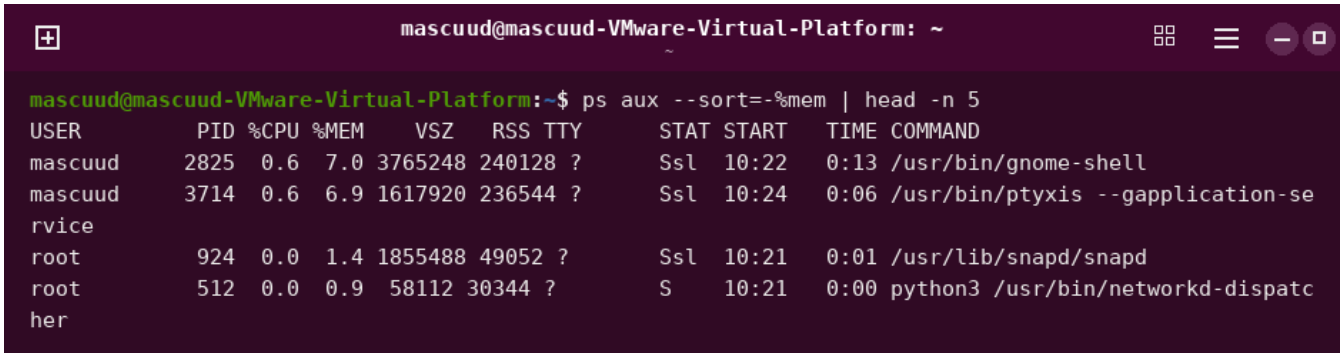
ps aux | less

Shows all processes, one page at a time.



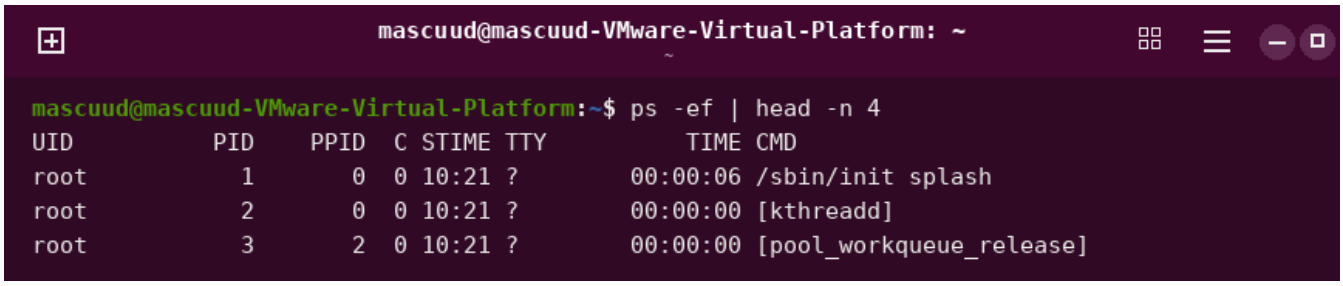
`ps aux --sort=-%mem | head -n 5`

Shows the 5 processes that use the most memory.



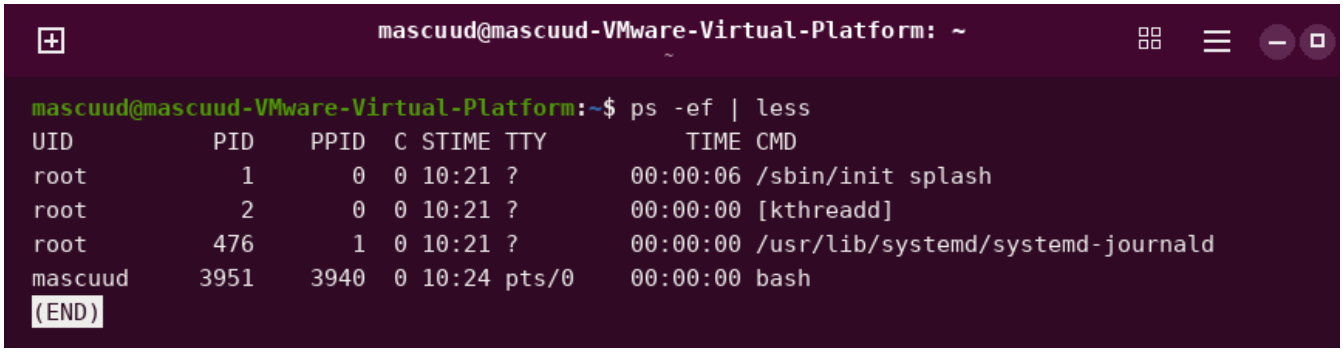
`ps -ef | head -n 4`

Shows the first 4 lines of all processes.



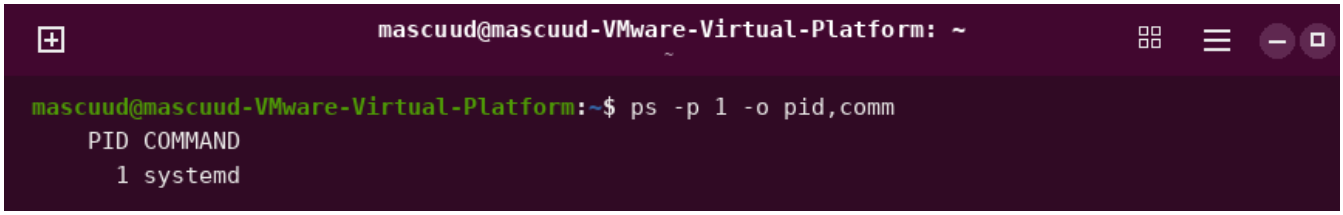
`ps -ef | less`

Shows all processes, one page at a time.



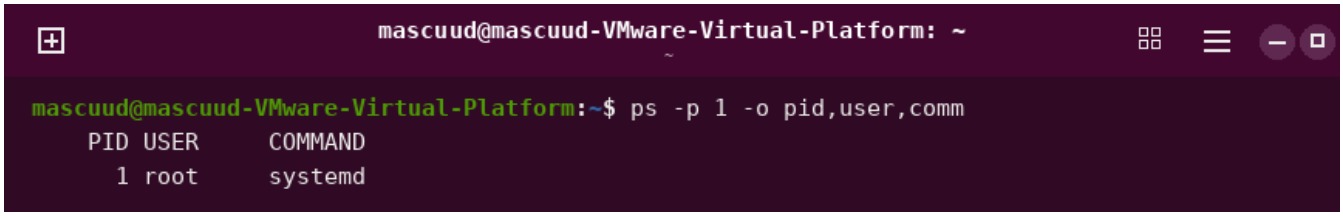
`ps -p 1 -o pid,comm`

Shows the process with number 1.



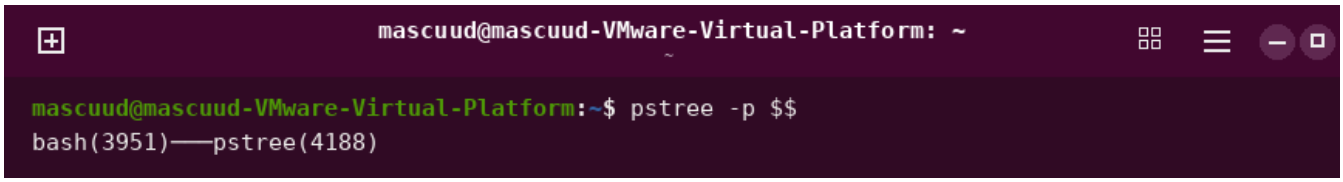
`ps -p 1 -o pid,user,comm`

Shows process 1 with its user.



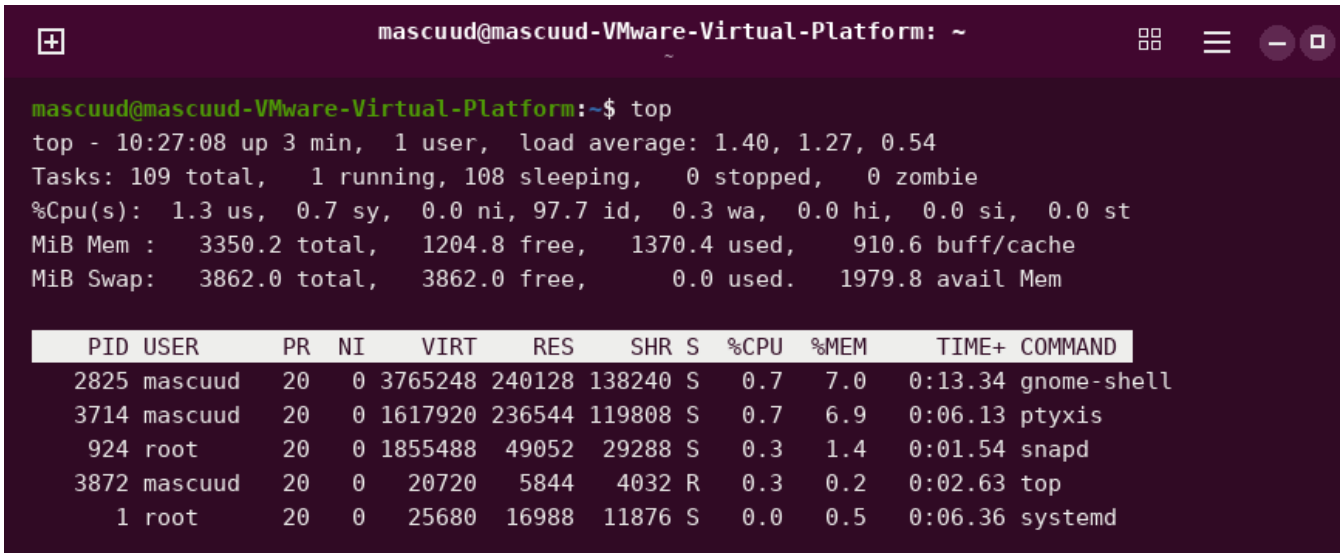
`pstree -p $$`

Shows the processes as a family tree.



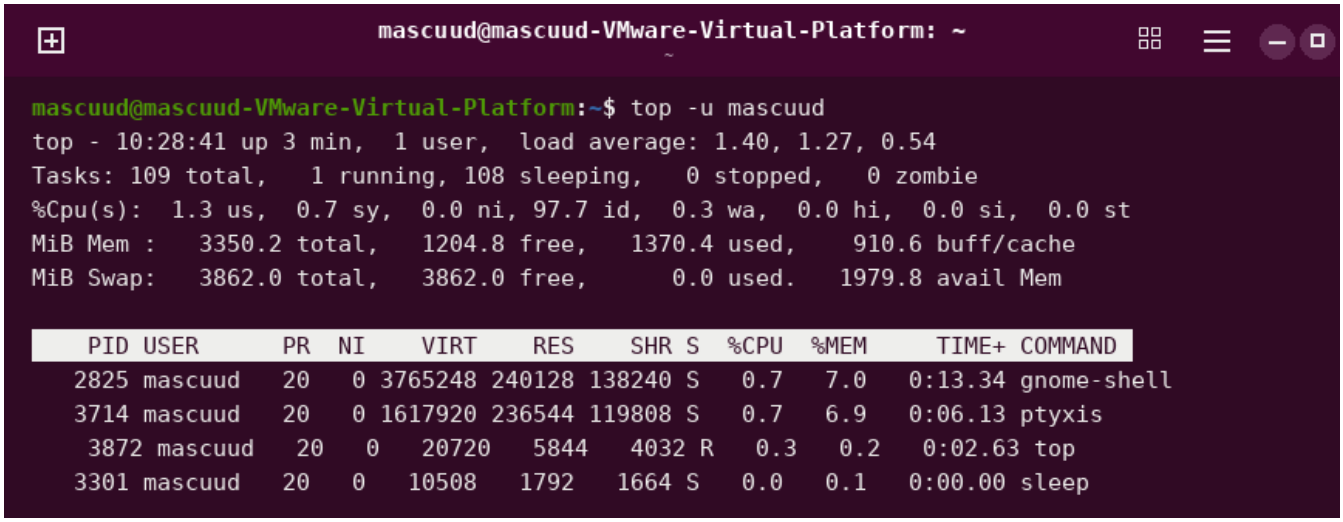
`top`

Shows live information about the processes. Press q to quit.



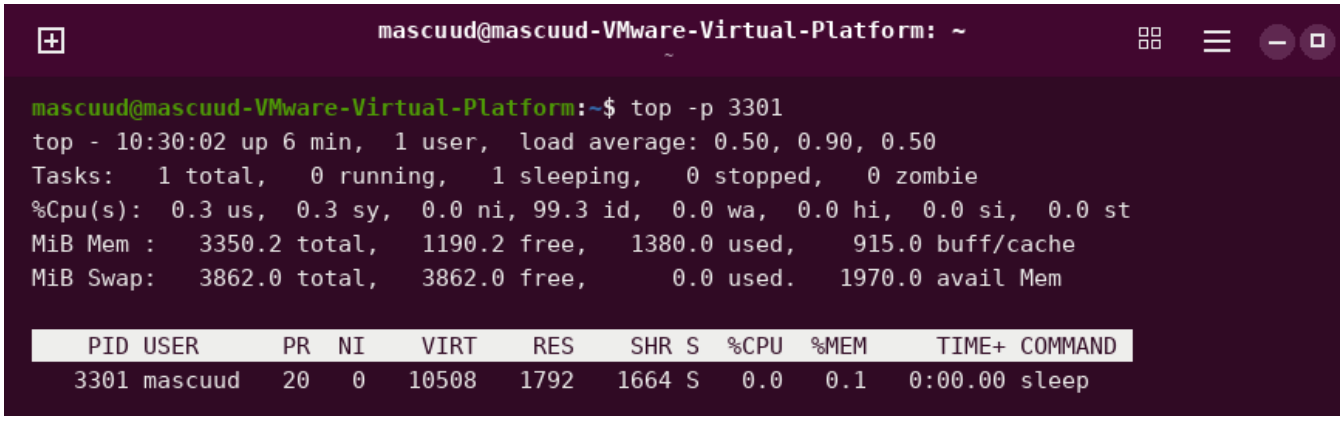
`top -u mascuud`

Shows live processes of one user only.



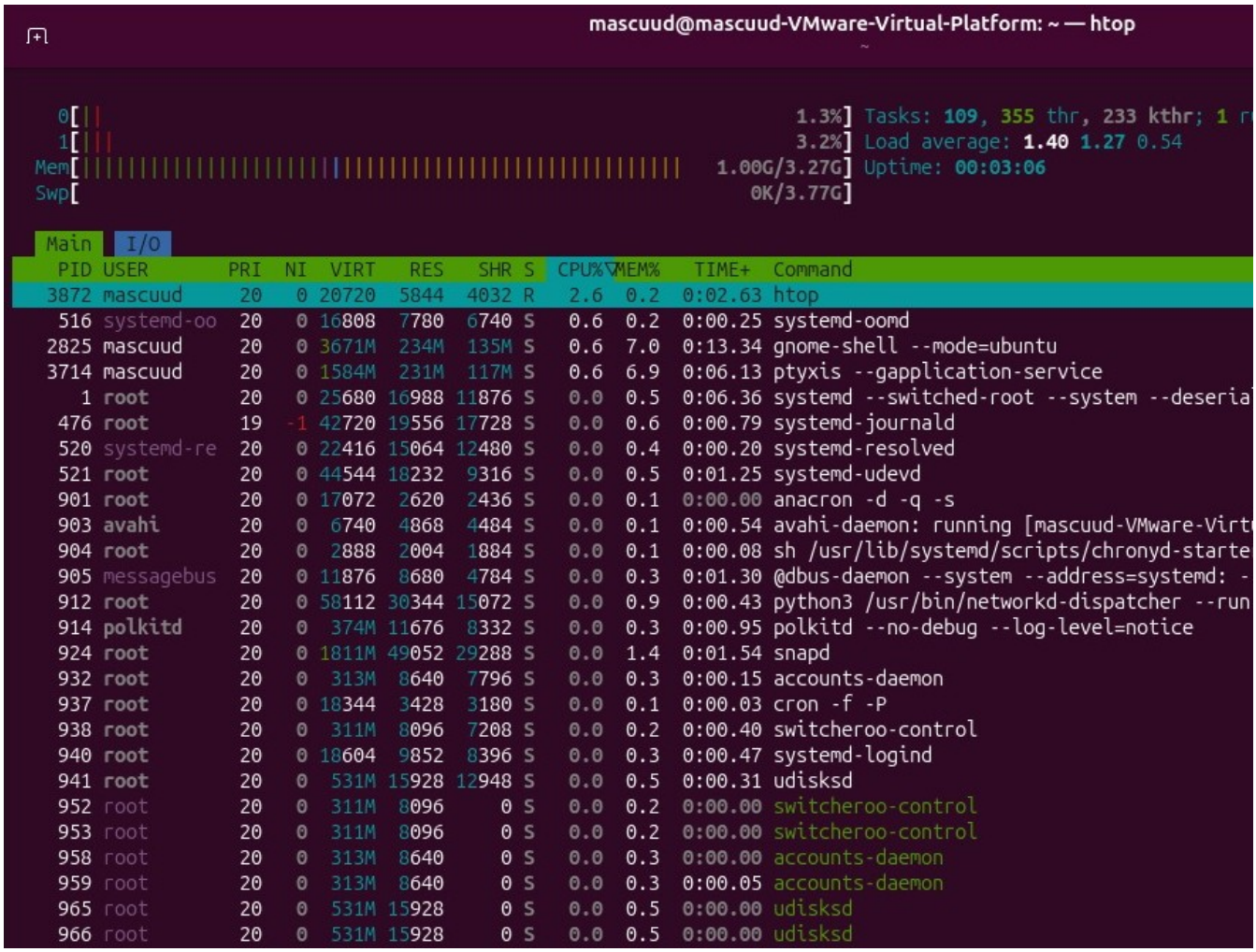
`top -p 3301`

Shows live information of one process only.



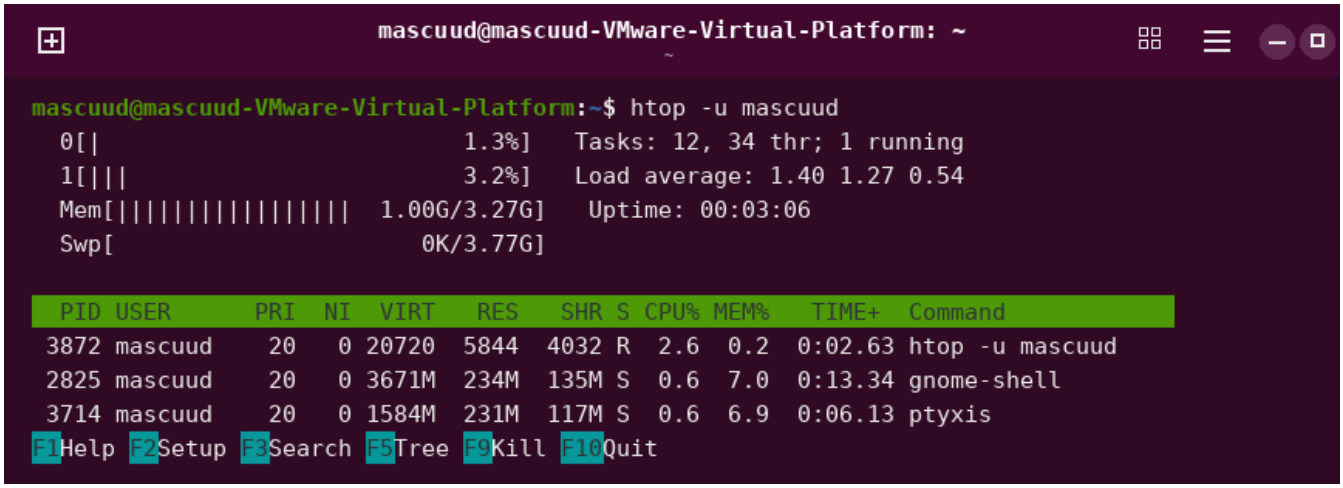
`htop`

Shows live processes with colors. It is easier than top.



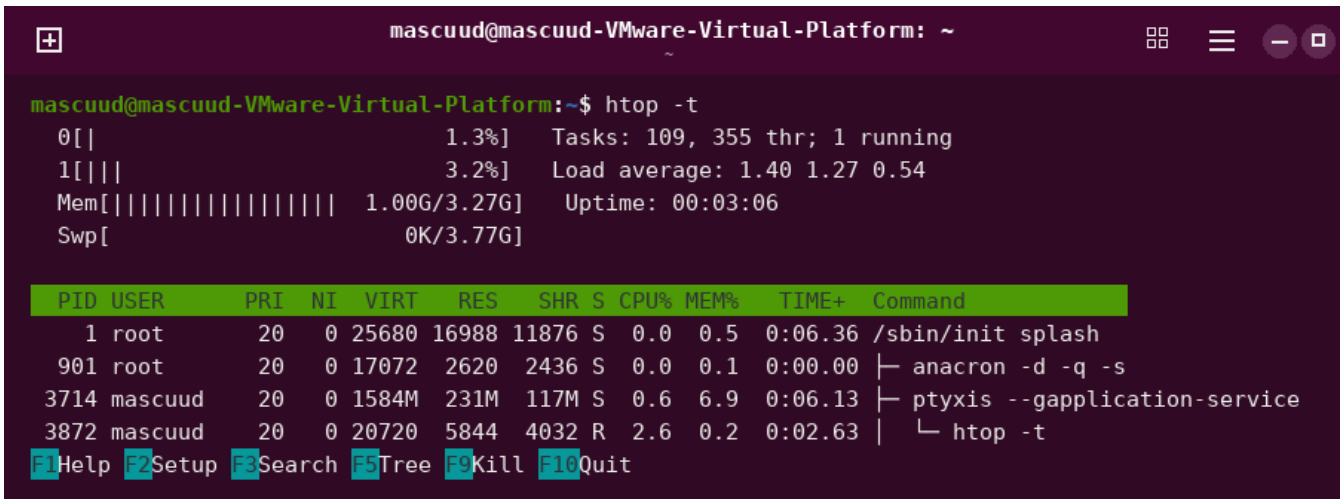
`htop -u mascuud`

Shows the live processes of one user.



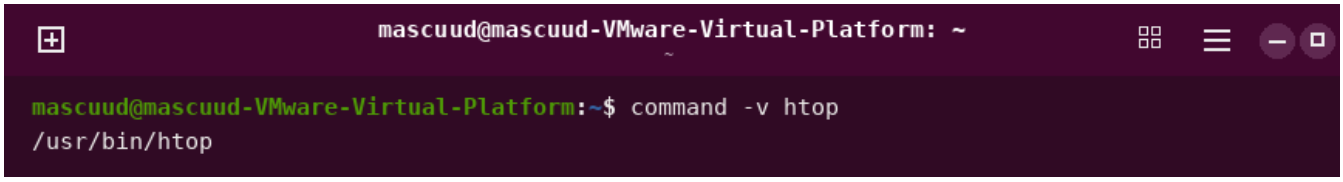
`htop -t`

Shows the processes as a tree.



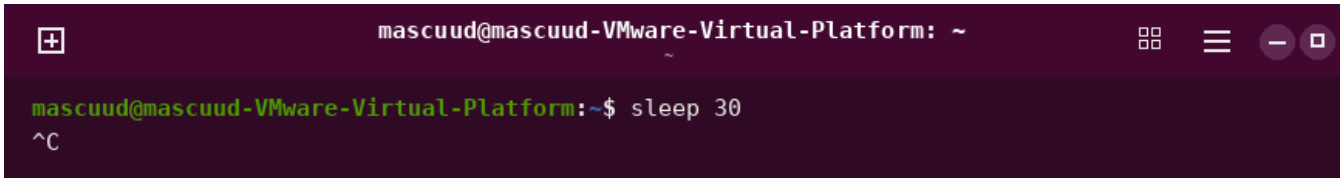
`command -v htop`

Shows where htop is. This tells you it is installed.



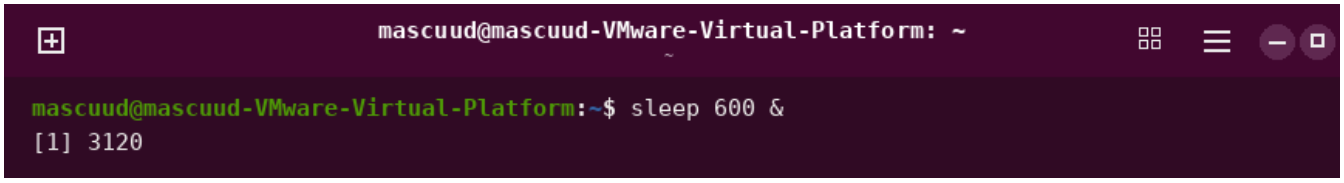
`sleep 30`

Waits for 30 seconds and does nothing.



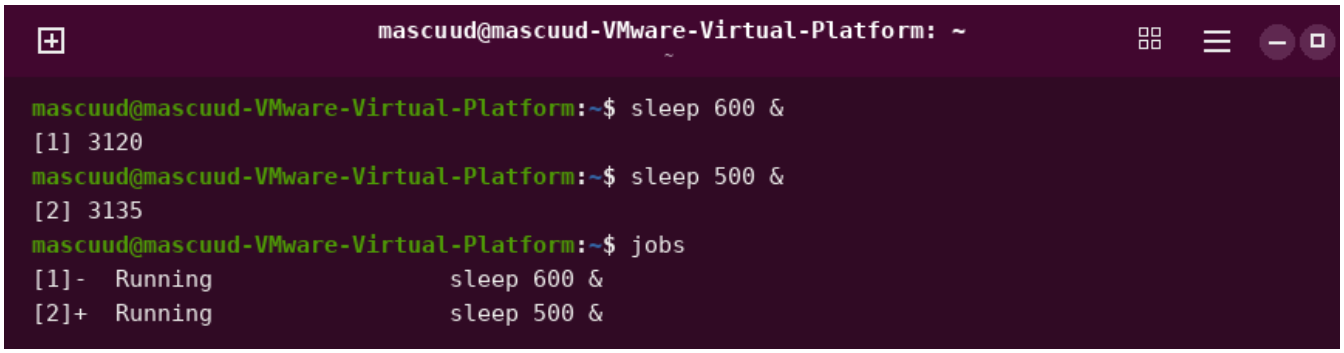
`sleep 600 &`

Runs sleep in the background. The terminal is free.



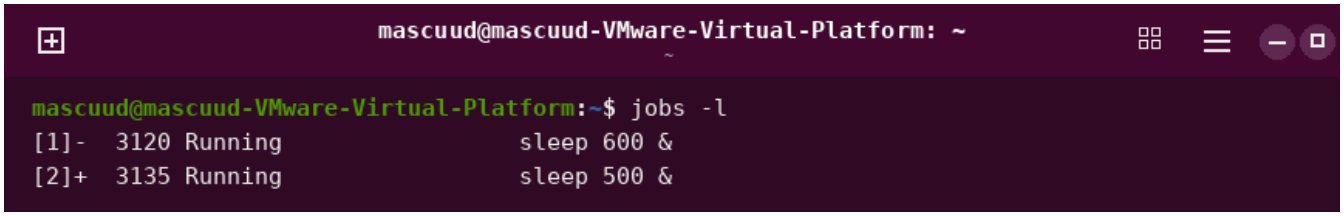
`jobs`

Shows the jobs in your shell.



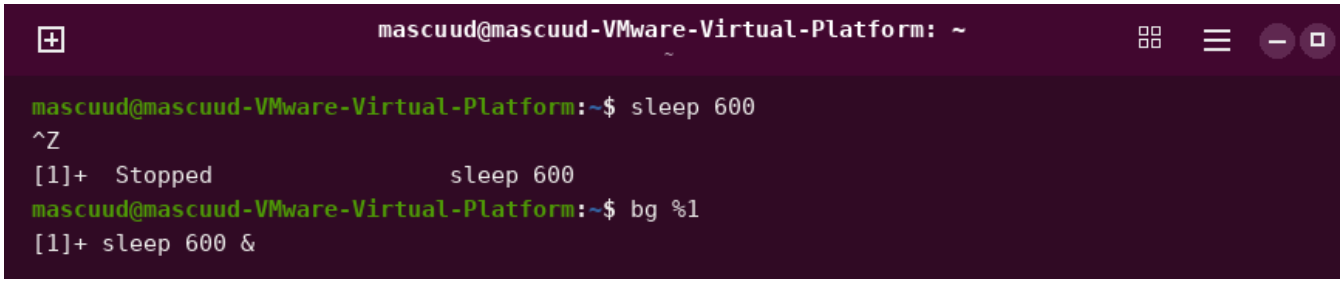
`jobs -l`

Shows the jobs with their process numbers.



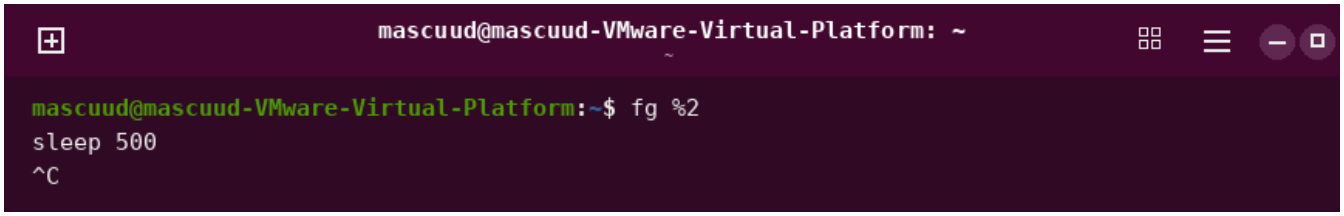
`bg %1`

Runs a stopped job in the background.



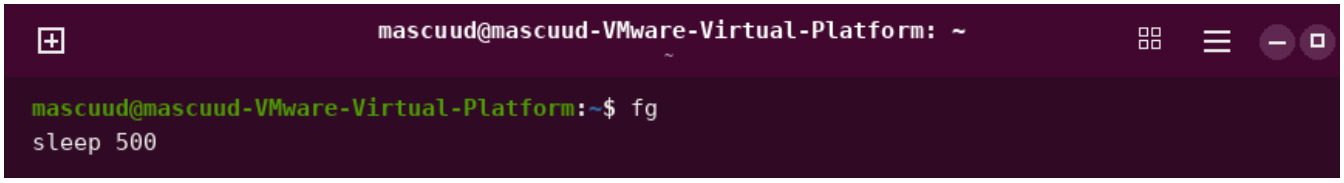
`fg %2`

Brings a background job to the front.



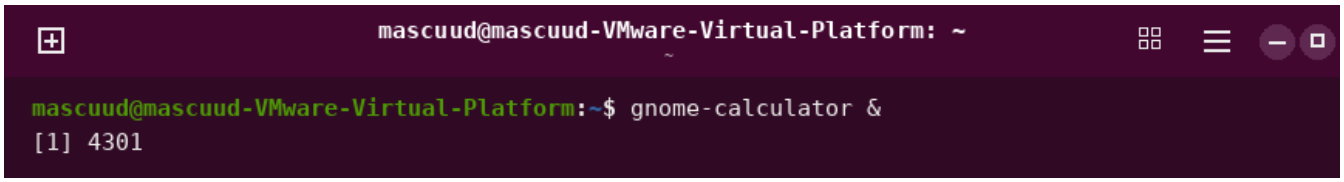
`fg`

Brings the last job to the front.



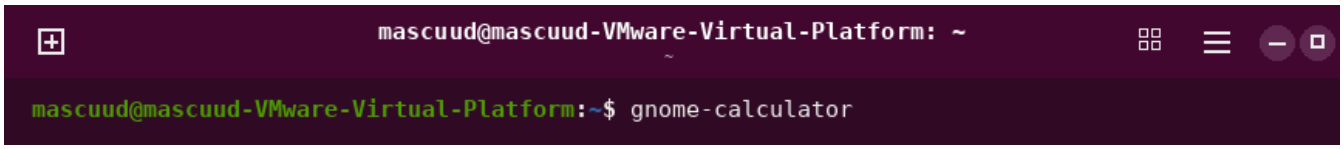
`gnome-calculator &`

Opens the calculator in the background.



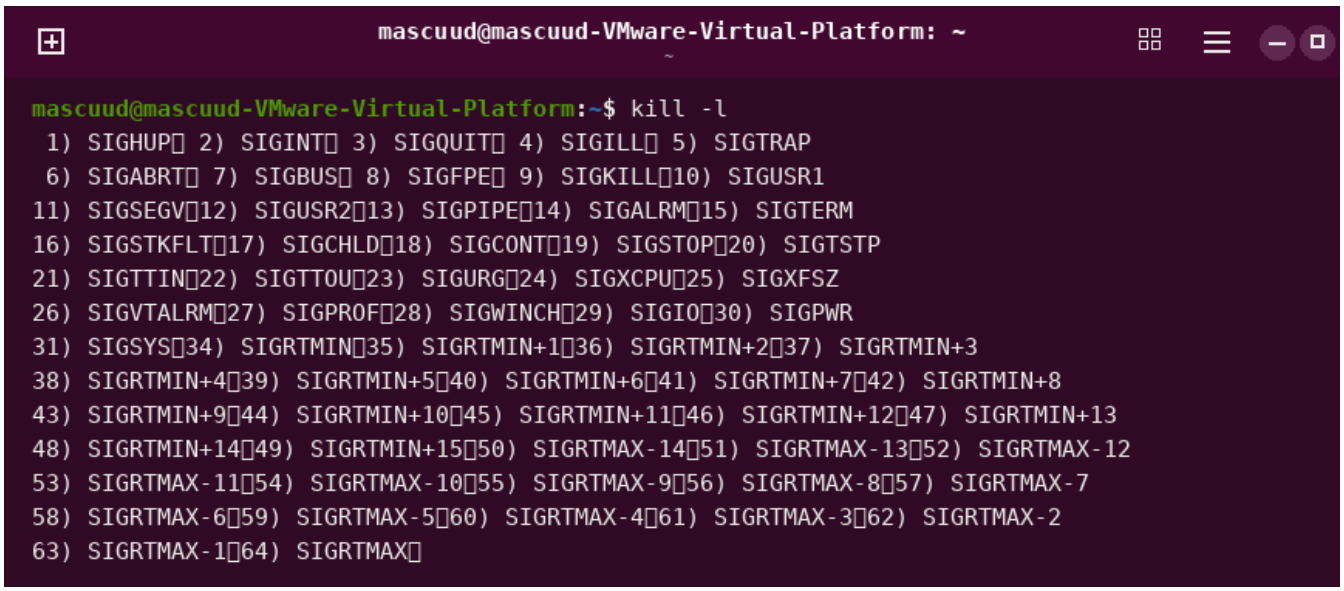
`gnome-calculator`

Opens the calculator. The terminal waits until you close it.



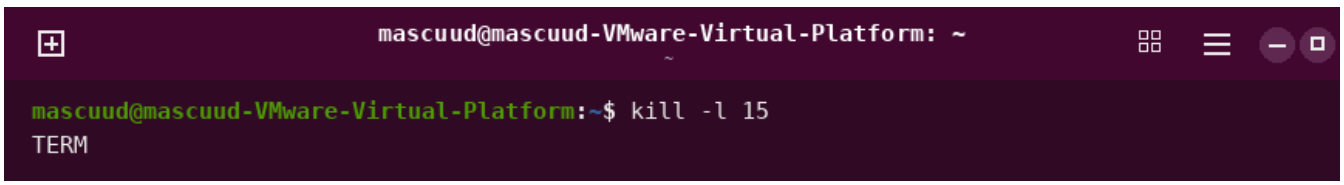
`kill -l`

Shows the list of signal names.



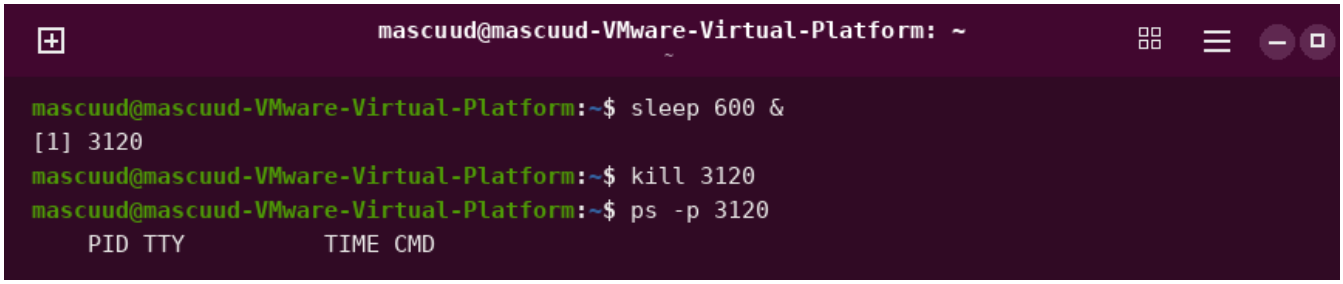
`kill -l 15`

Shows the name of signal 15.



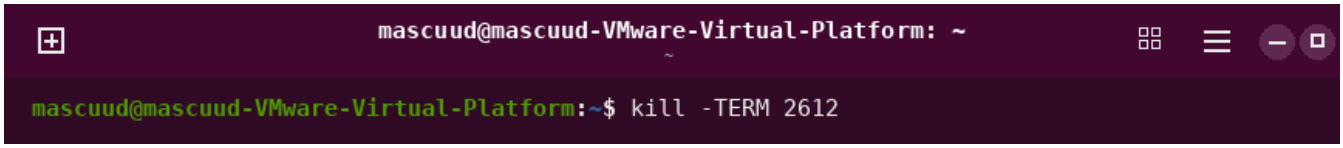
kill 3120

Asks a process to stop.



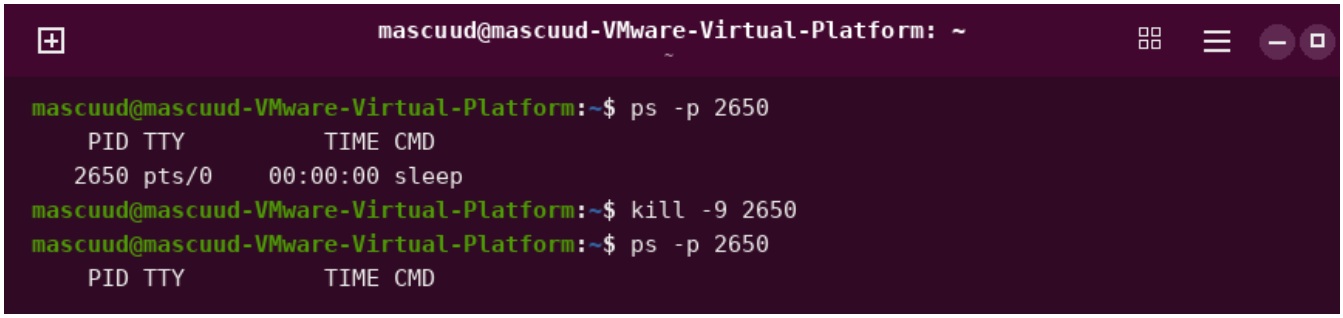
kill -TERM 2612

Asks a process to stop, using the signal name.



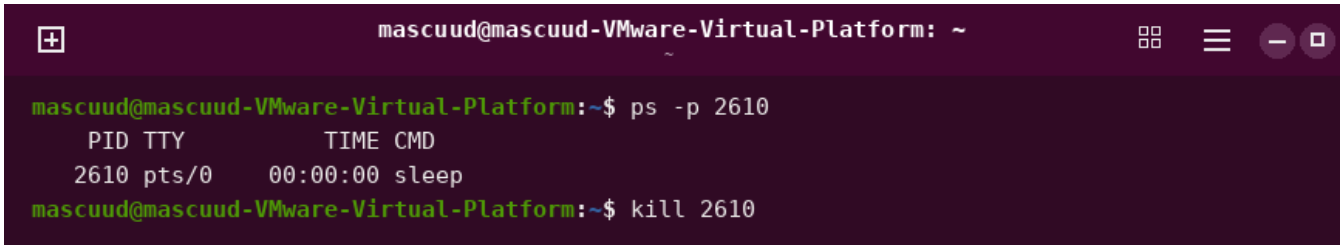
kill -9 2650

Forces a process to stop.



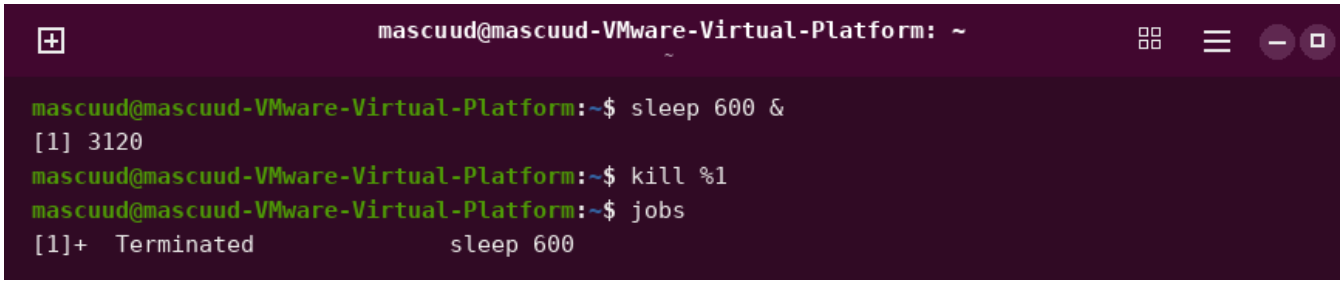
kill 2650 and kill 2610

Stops two processes by their numbers.



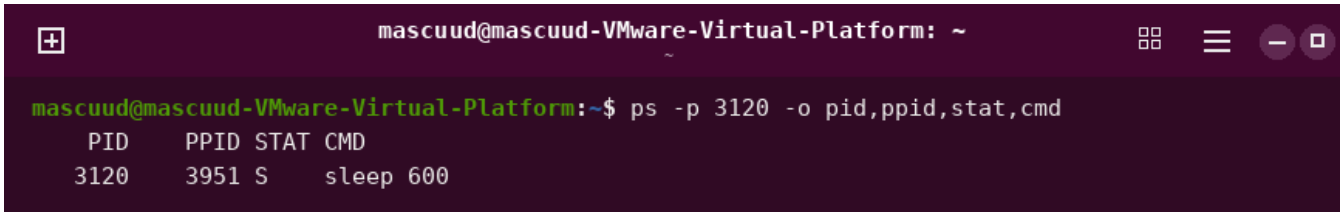
kill %1

Stops a job by its job number.



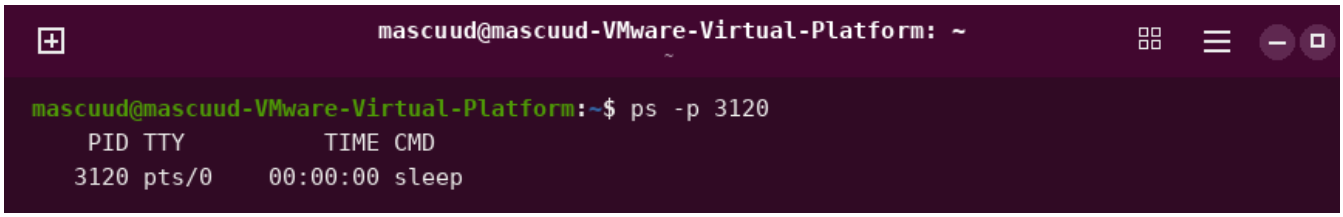
`ps -p 3120 -o pid,ppid,stat,cmd`

Shows details of one process.



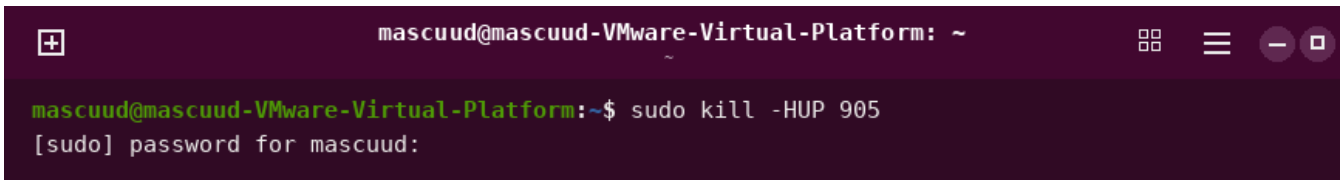
`ps -p 3120`

Shows if a process is still running.



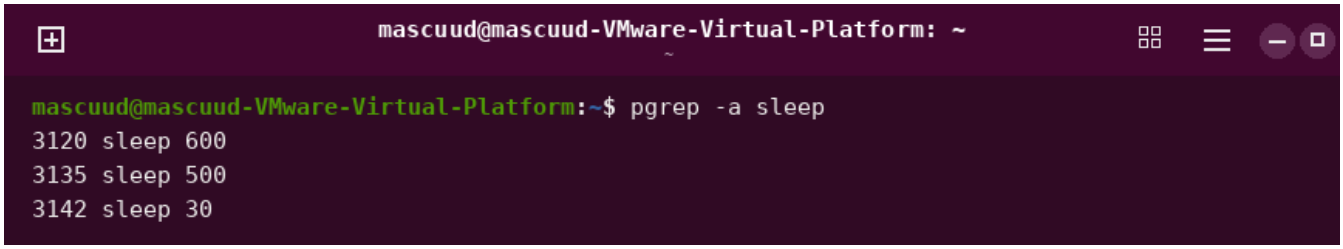
`sudo kill -HUP 905`

Asks a service to read its settings again.



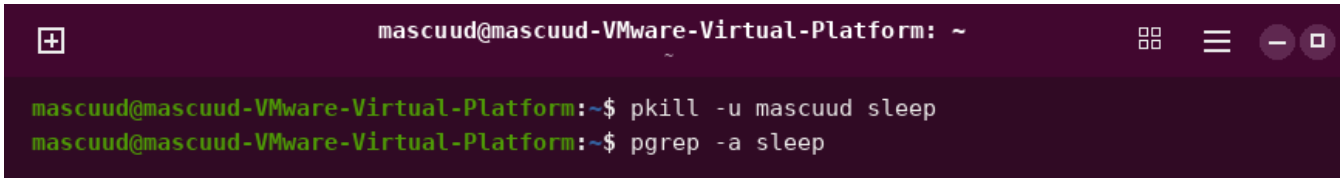
`pgrep -a sleep`

Finds processes by name.



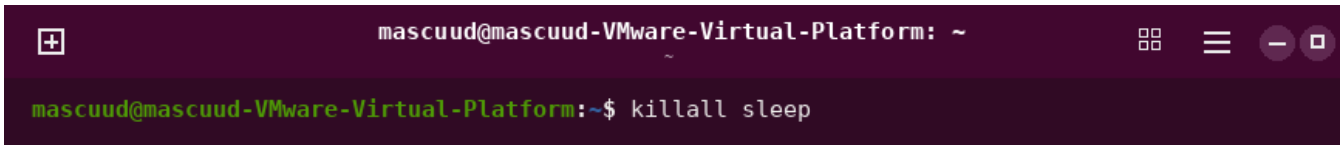
`pkill -u mascuud sleep`

Stops processes by name.



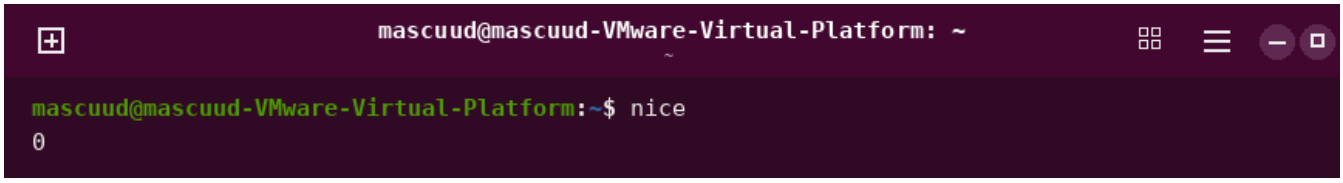
`killall sleep`

Stops all processes with this name.



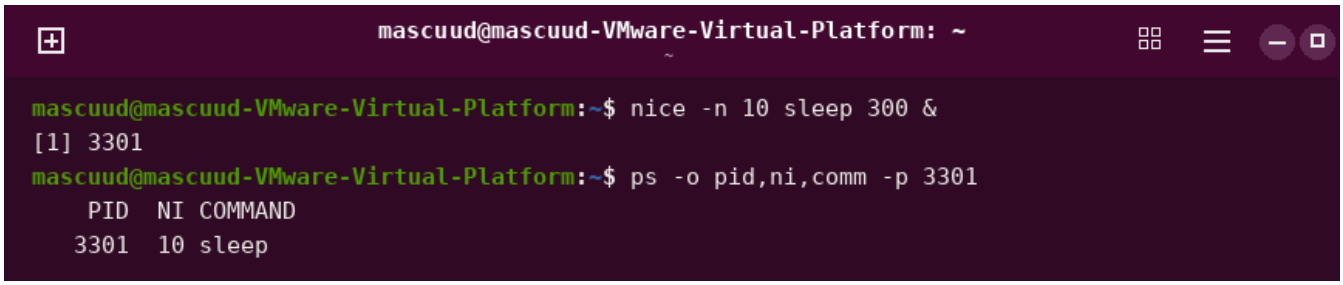
nice

Shows the nice number of your shell.



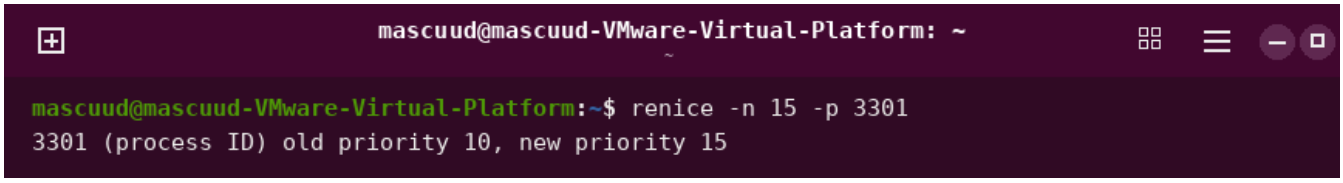
nice -n 10 sleep 300 &

Starts a program with a lower priority.



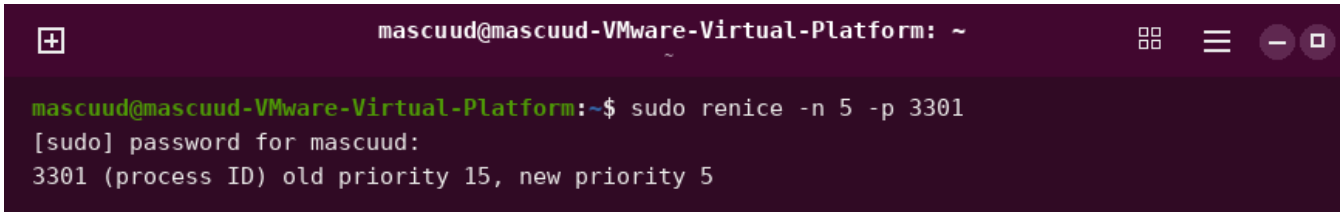
renice -n 5 -p 3301

Changes the priority of a running process.



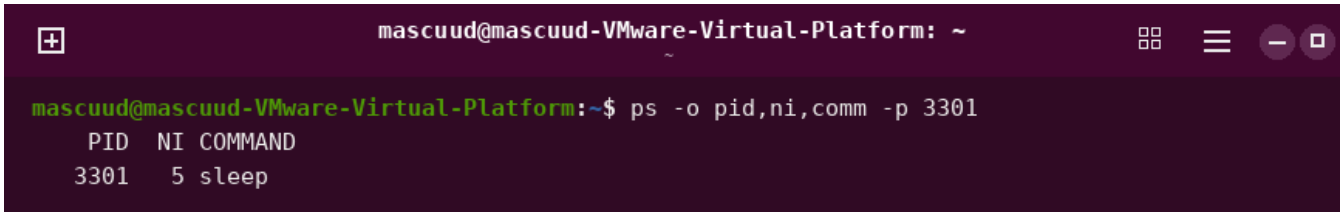
sudo renice -n 5 -p 3301

Changes the priority with admin rights.



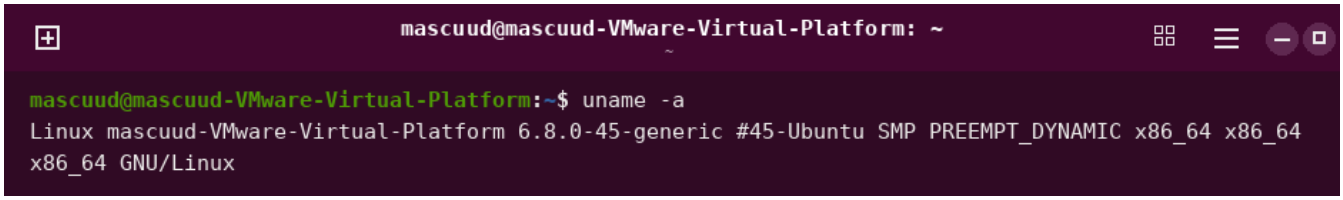
ps -o pid,ni,comm -p 3301

Shows the nice number of a process.



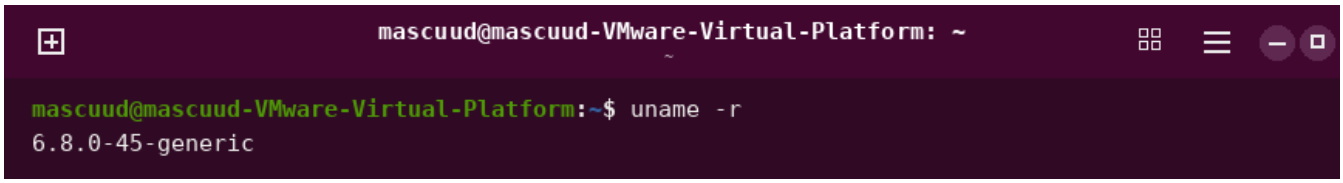
`uname -a`

Shows all system information.



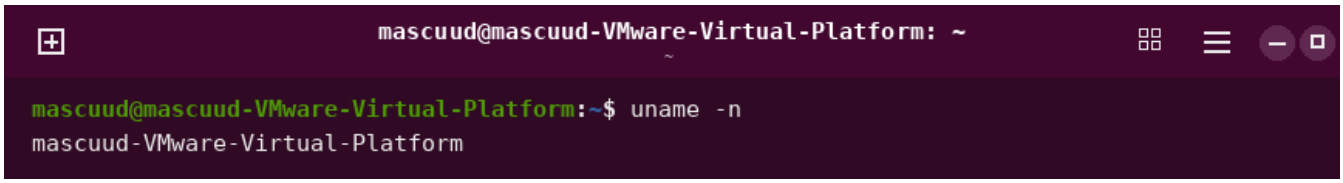
`uname -r`

Shows the kernel version.



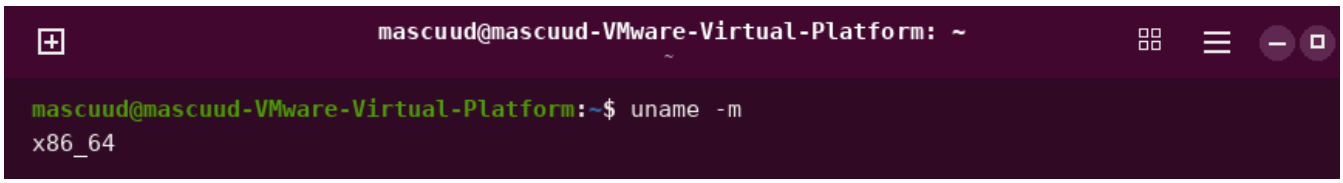
`uname -n`

Shows the name of the computer.



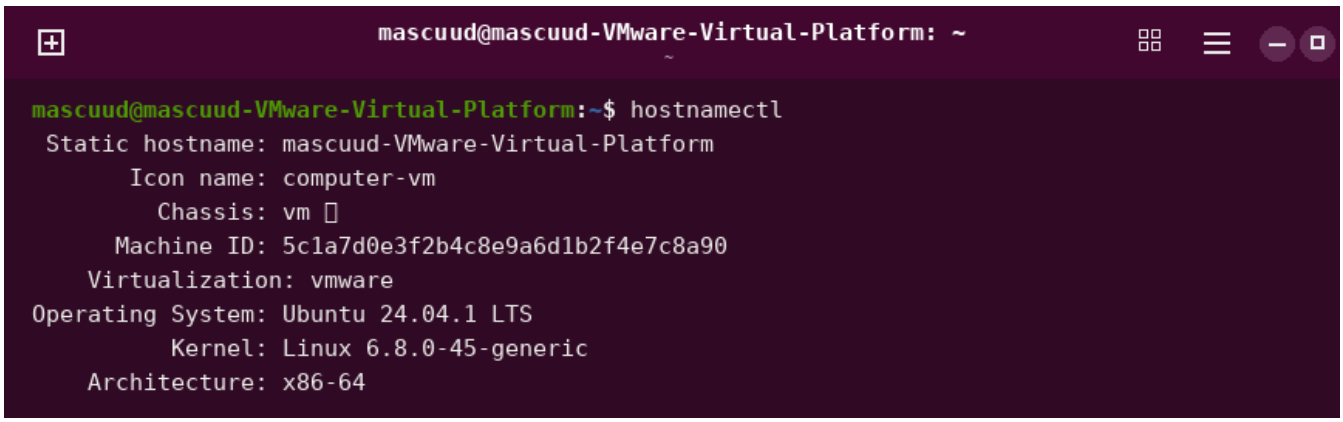
`uname -m`

Shows the type of computer hardware.



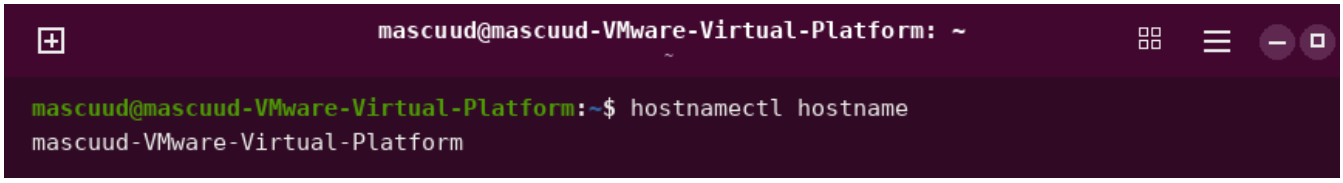
`hostnamectl`

Shows the computer name and system details.



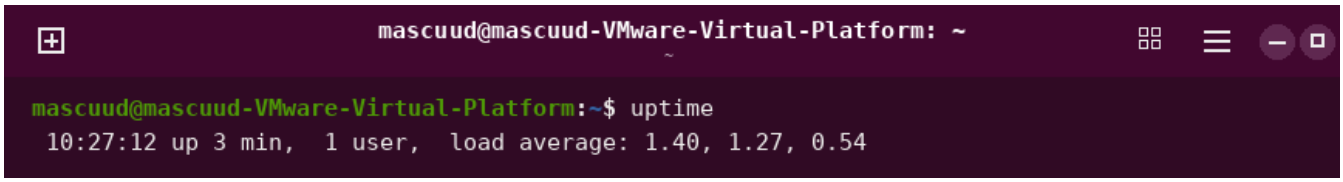
`hostnamectl hostname`

Shows only the computer name.



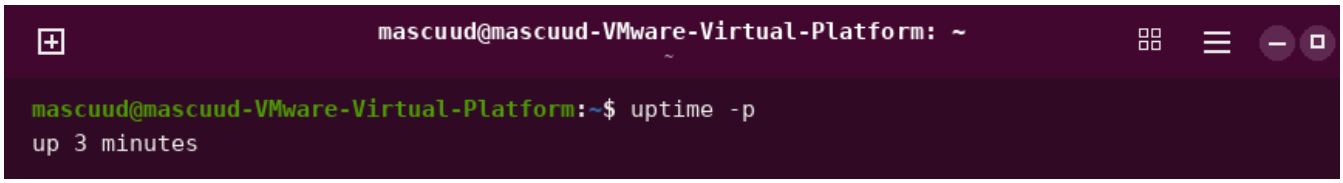
`uptime`

Shows how long the computer has been on.



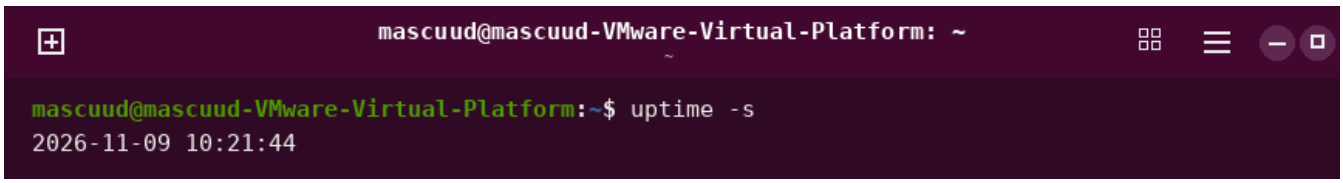
`uptime -p`

Shows the uptime in easy words.



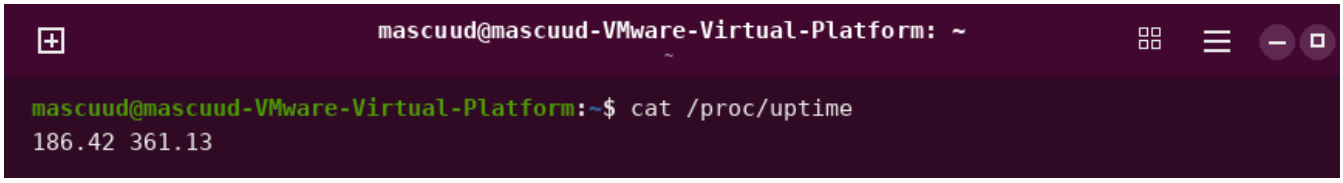
`uptime -s`

Shows the time when the computer started.



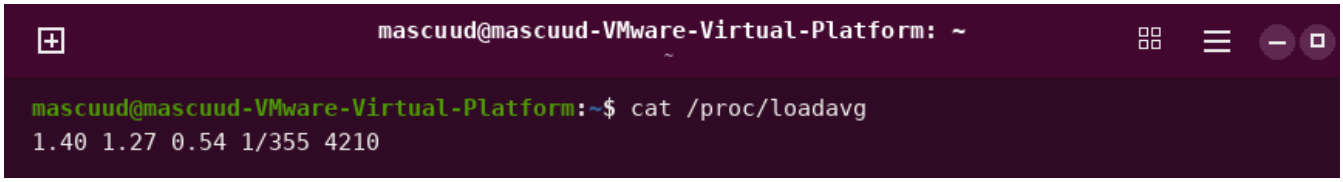
`cat /proc/uptime`

Shows the uptime in seconds.



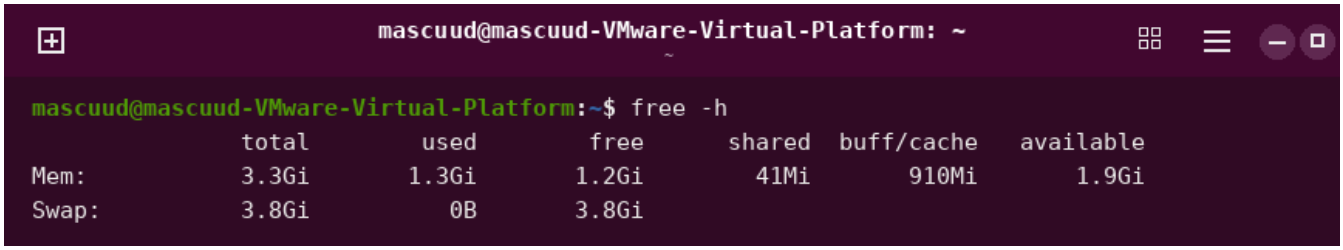
`cat /proc/loadavg`

Shows how busy the computer is.



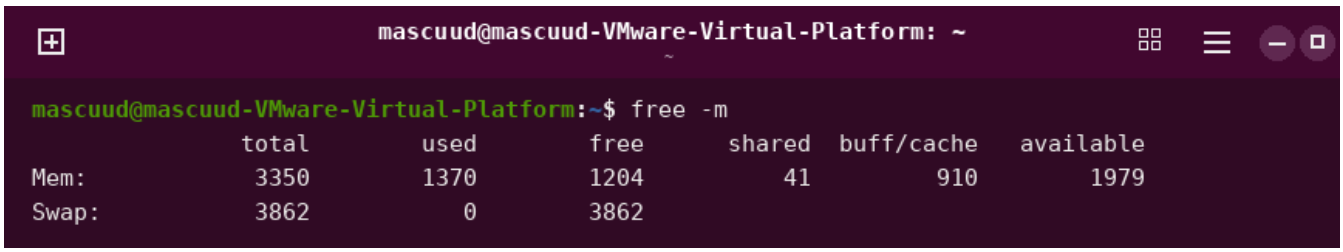
`free -h`

Shows memory use in easy units.



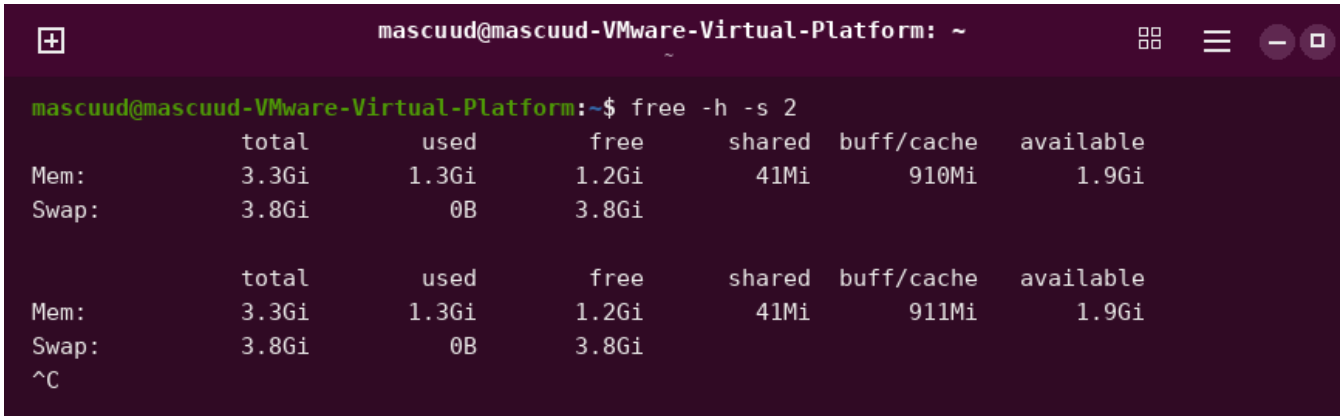
`free -m`

Shows memory use in megabytes.



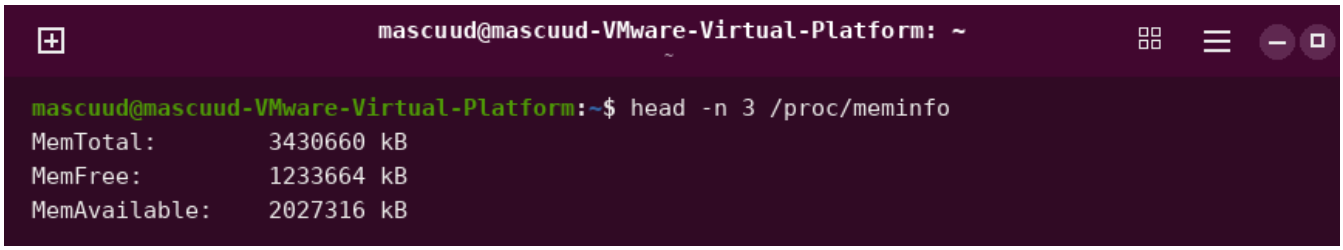
`free -h -s 2`

Shows memory use again every 2 seconds.



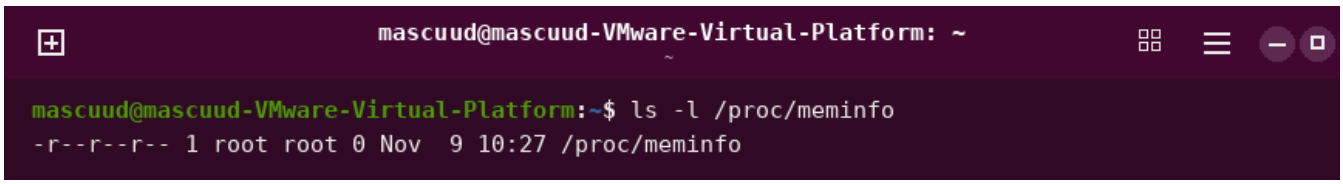
`head -n 3 /proc/meminfo`

Shows the first lines of the memory file.



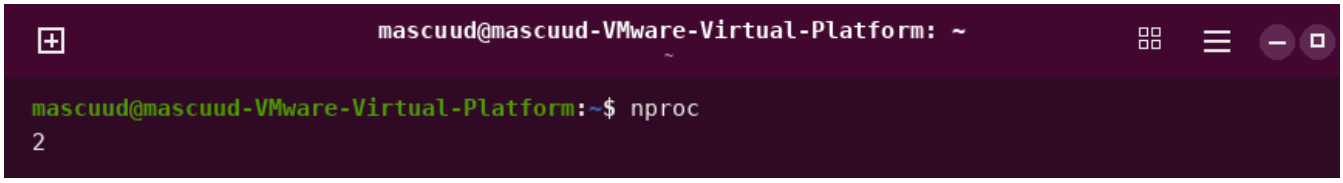
`ls -l /proc/meminfo`

Shows that the memory file has size 0.



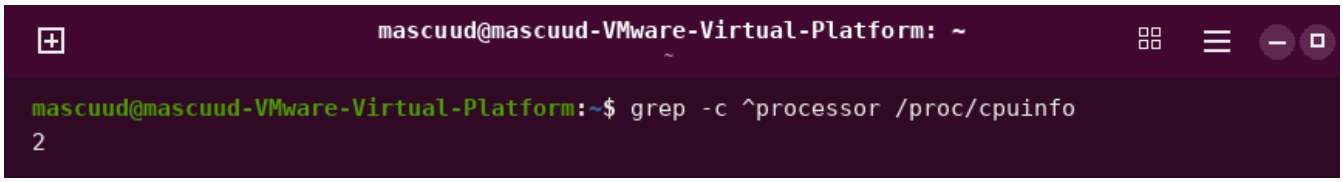
`nproc`

Shows how many CPUs the computer has.



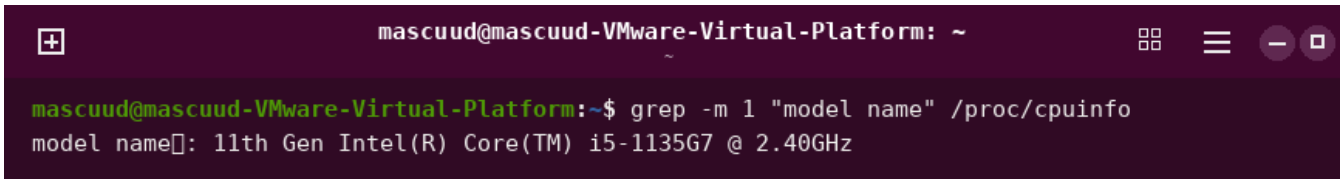
`grep -c ^processor /proc/cpuinfo`

Counts the CPUs.



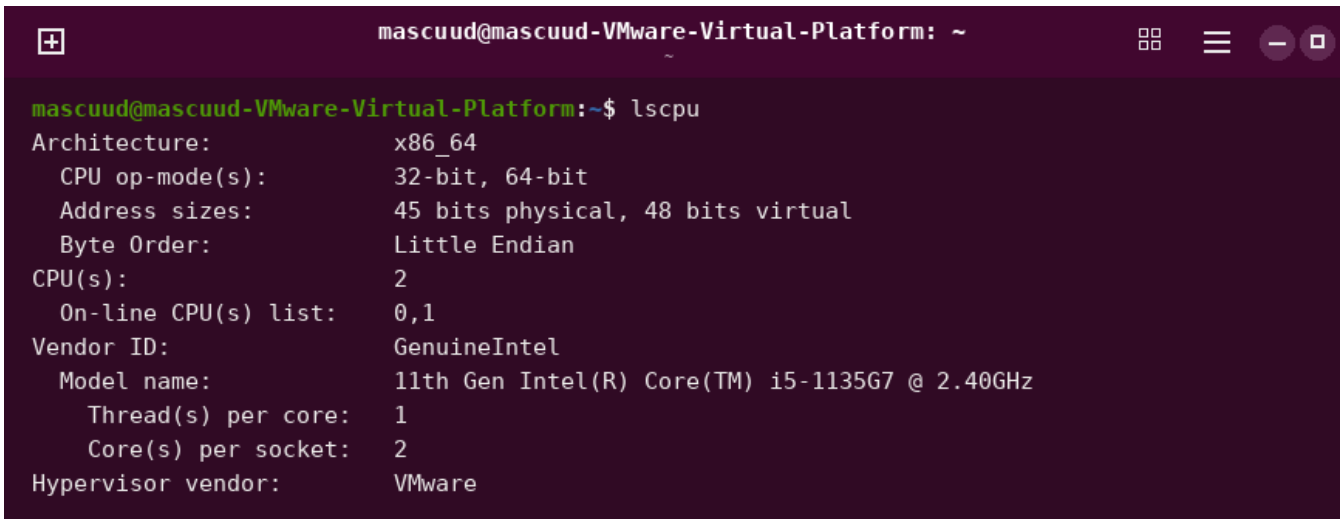
`grep -m 1 "model name" /proc/cpuinfo`

Shows the name of the CPU.



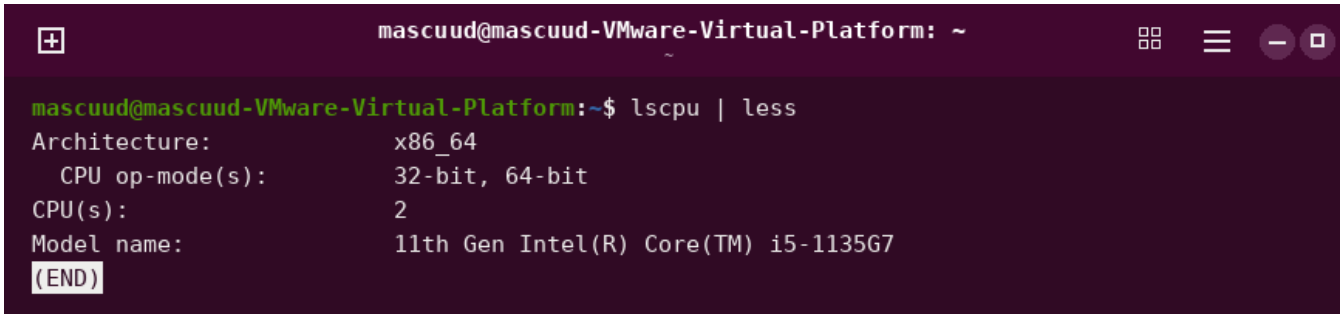
`lscpu`

Shows details about the CPU.



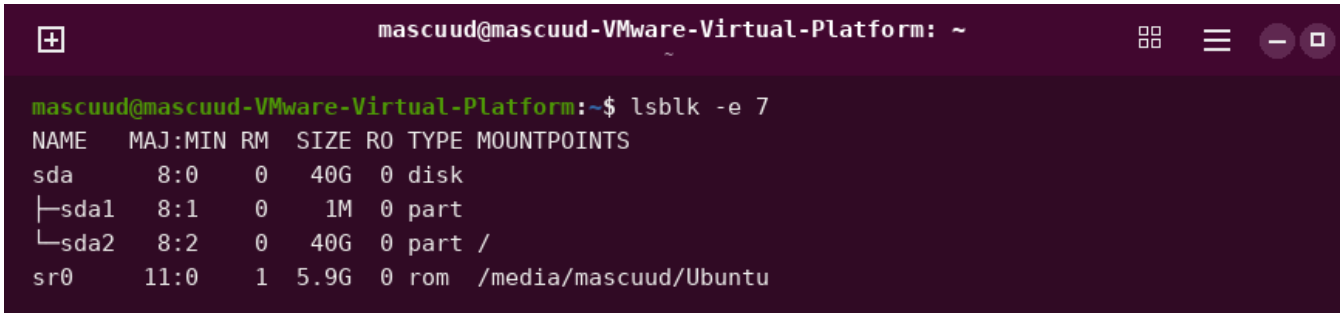
`lscpu | less`

Shows CPU details, one page at a time.



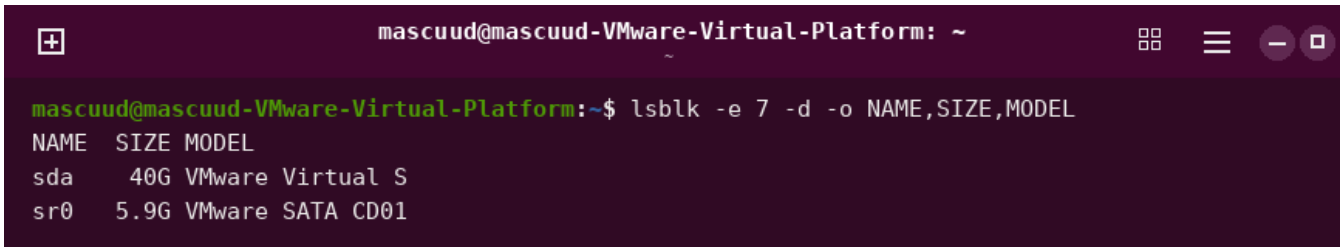
`lsblk -e 7`

Shows the disks and partitions.



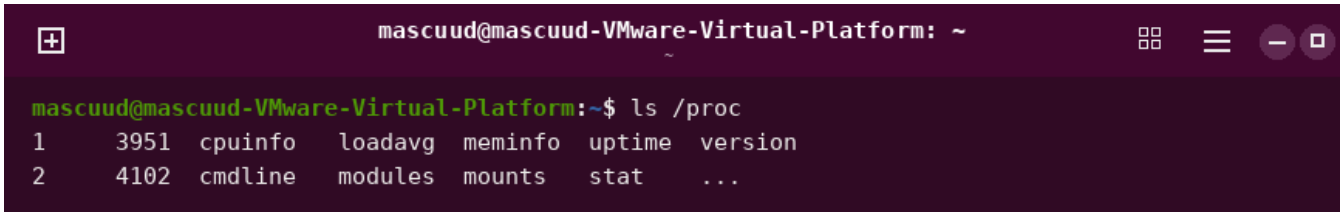
`lsblk -e 7 -d -o NAME,SIZE,MODEL`

Shows only the disks.



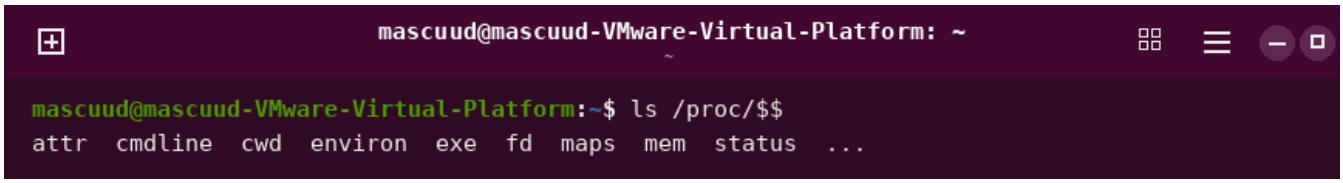
`ls /proc`

Shows the files in /proc.



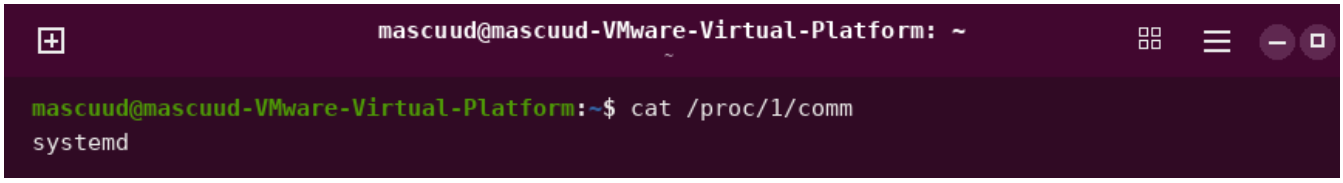
`ls /proc/$$`

Shows the files of your shell process.



`cat /proc/1/comm`

Shows the name of process 1.



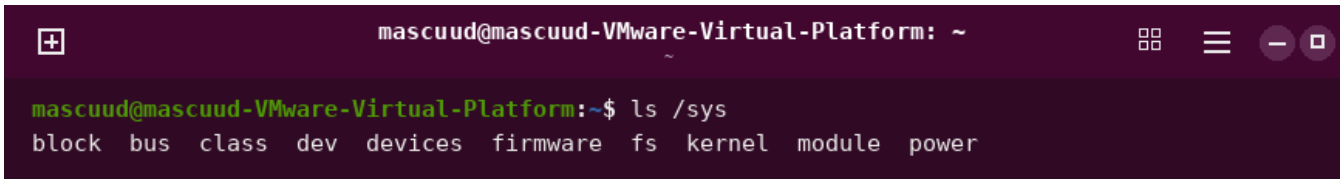
`head -n 7 /proc/$$/status`

Shows the status of your shell.



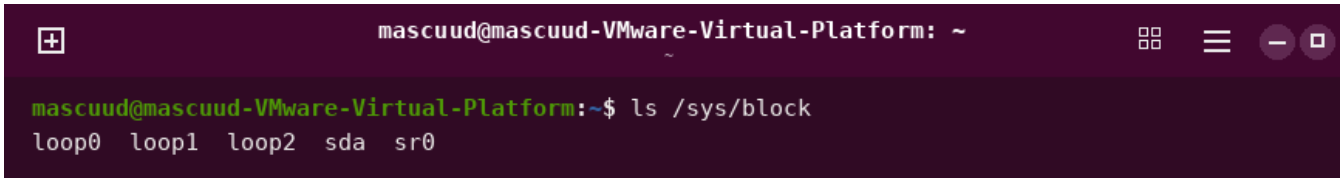
`ls /sys`

Shows the folders in /sys.



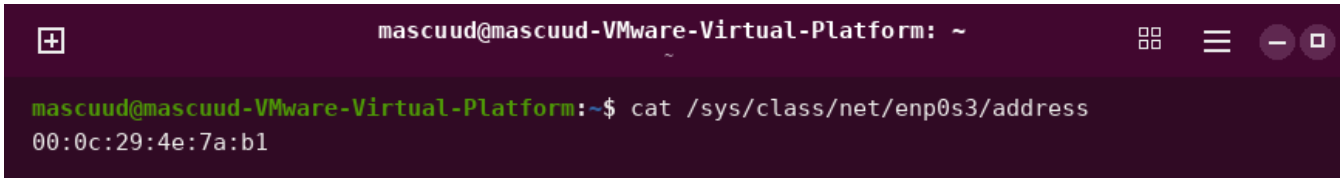
`ls /sys/block`

Shows the disks the kernel knows.



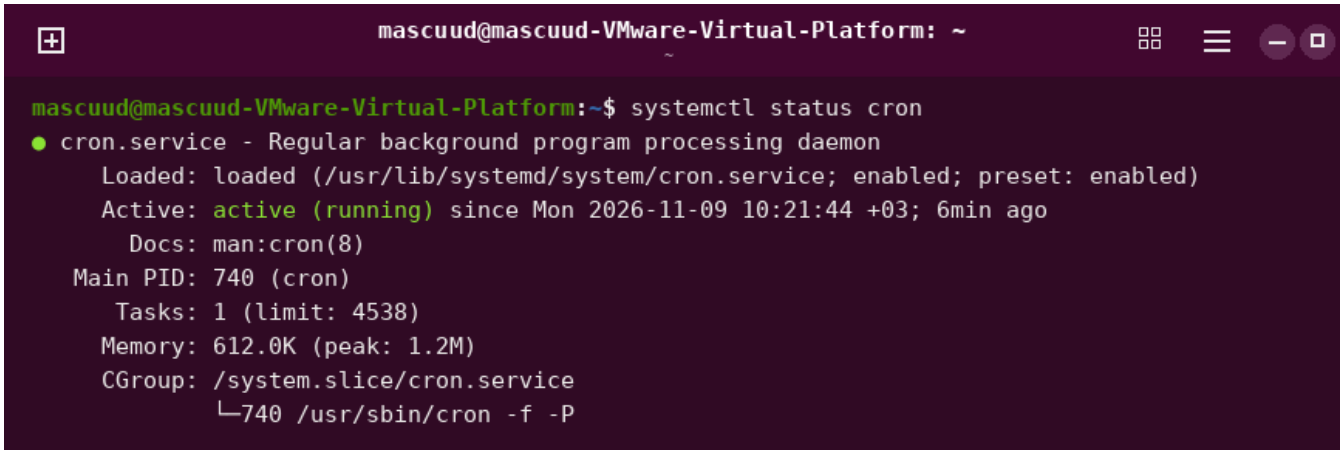
`cat /sys/class/net/enp0s3/address`

Shows the address of the network card.



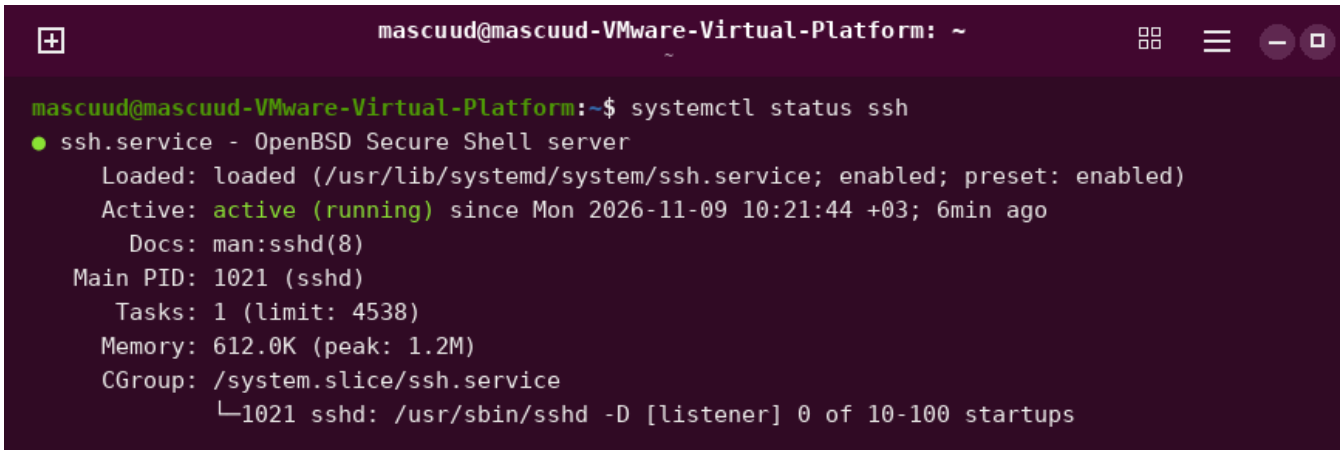
`systemctl status cron`

Shows if the cron service is running.



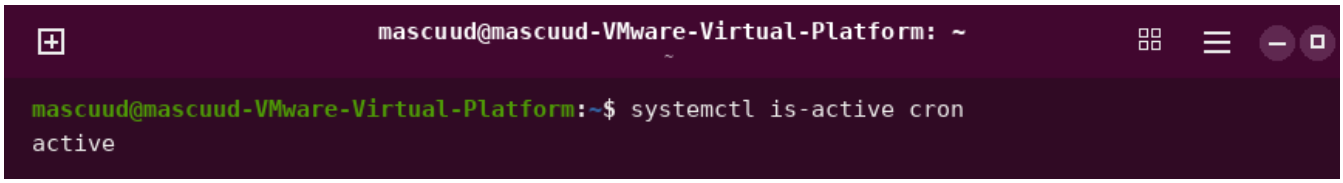
`systemctl status ssh`

Shows if the ssh service is running.



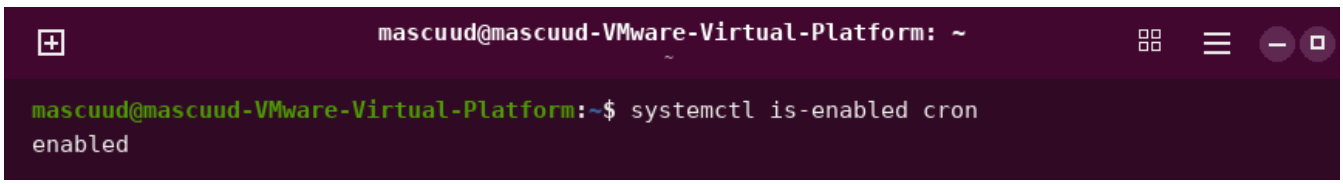
`systemctl is-active cron`

Shows if the service is active.



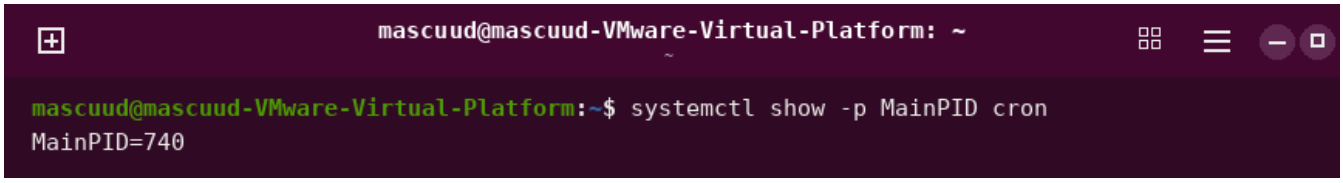
`systemctl is-enabled cron`

Shows if the service starts at boot.



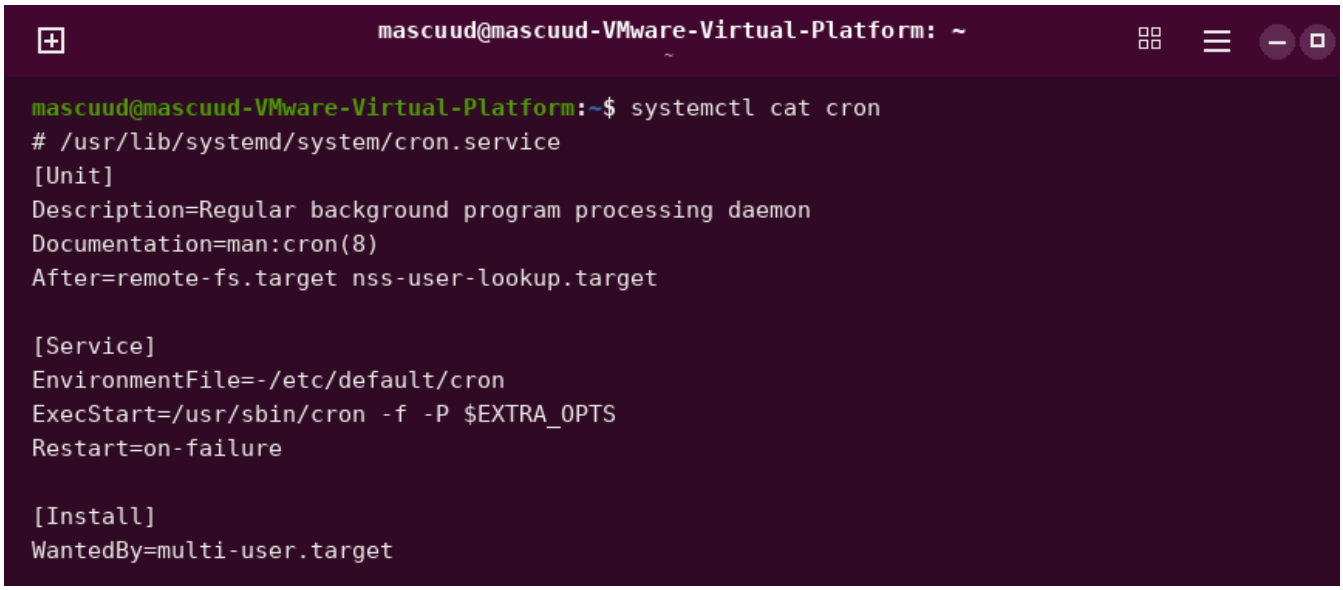
`systemctl show -p MainPID cron`

Shows the main process number of the service.



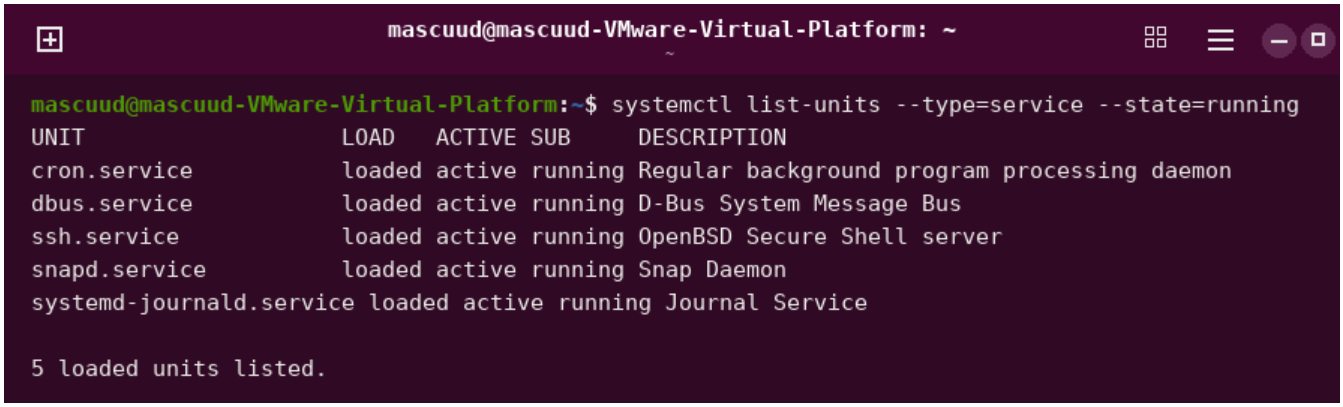
`systemctl cat cron`

Shows the file that starts the service.



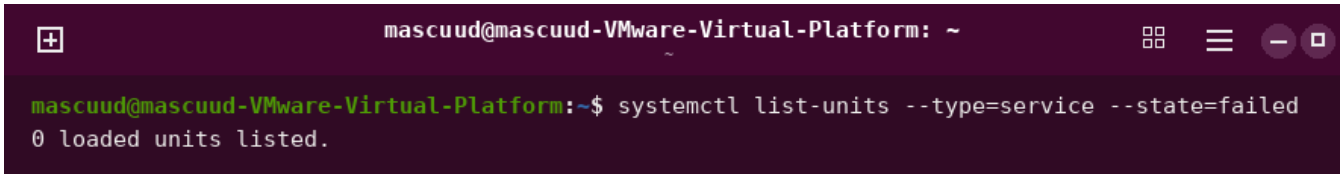
`systemctl list-units --type=service --state=running`

Lists the services that are running.



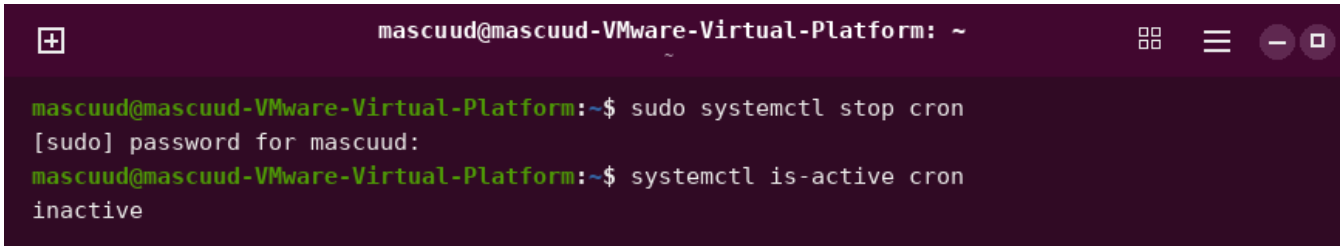
`systemctl list-units --type=service --state=failed`

Lists the services that failed.



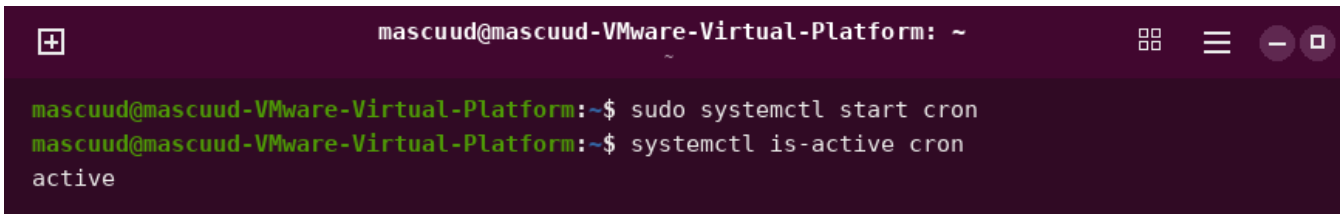
sudo systemctl stop cron

Stops the service now.



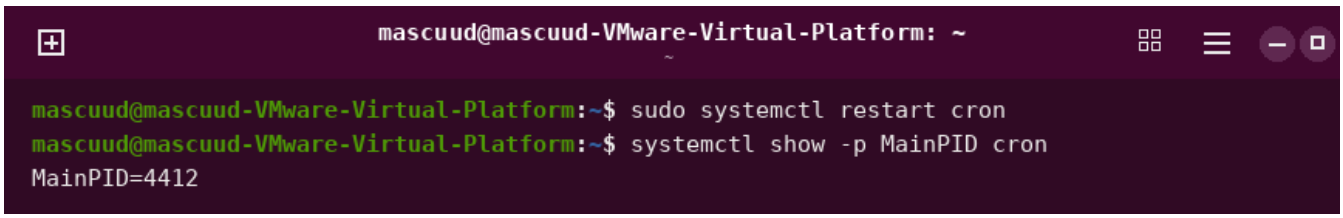
sudo systemctl start cron

Starts the service now.



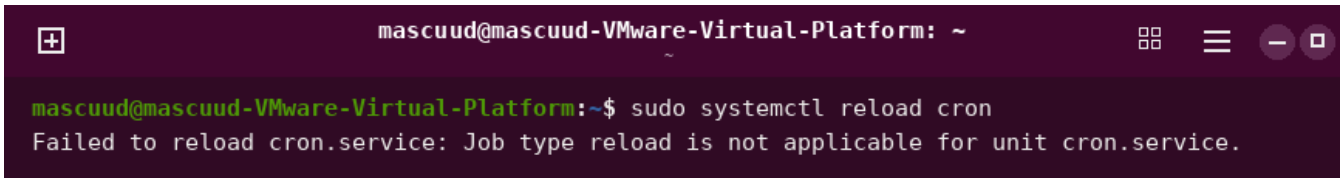
sudo systemctl restart cron

Stops the service and starts it again.



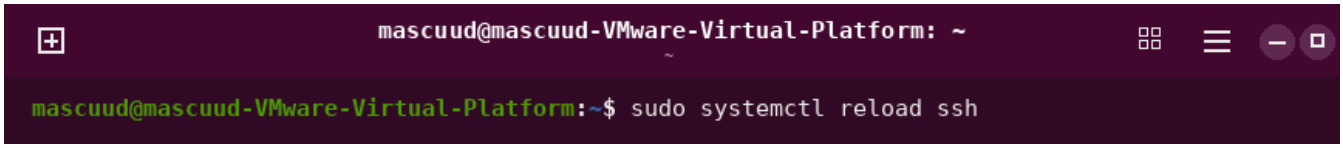
sudo systemctl reload cron

Makes the service read its settings again.



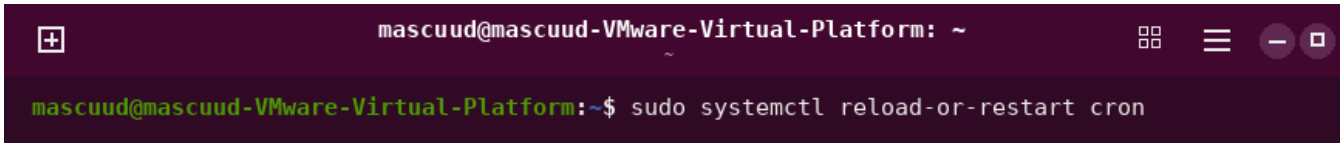
sudo systemctl reload ssh

Makes the ssh service read its settings again.



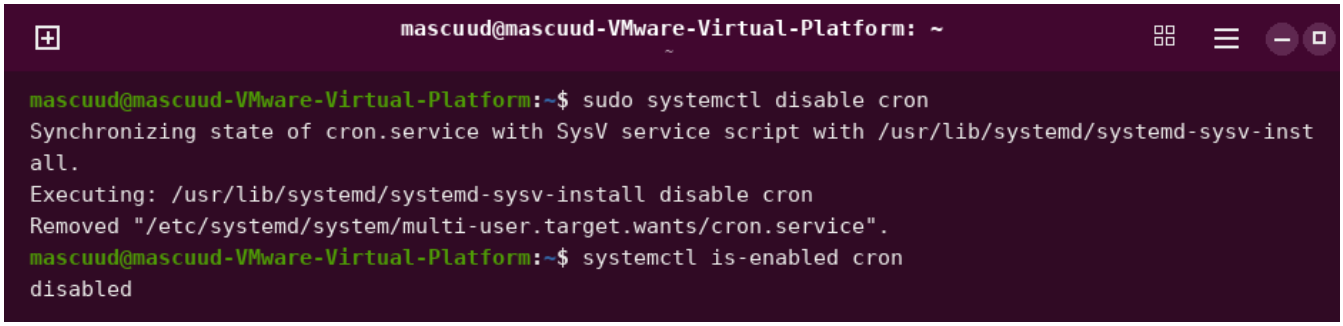
sudo systemctl reload-or-restart cron

Reloads the service, or restarts it if it cannot reload.



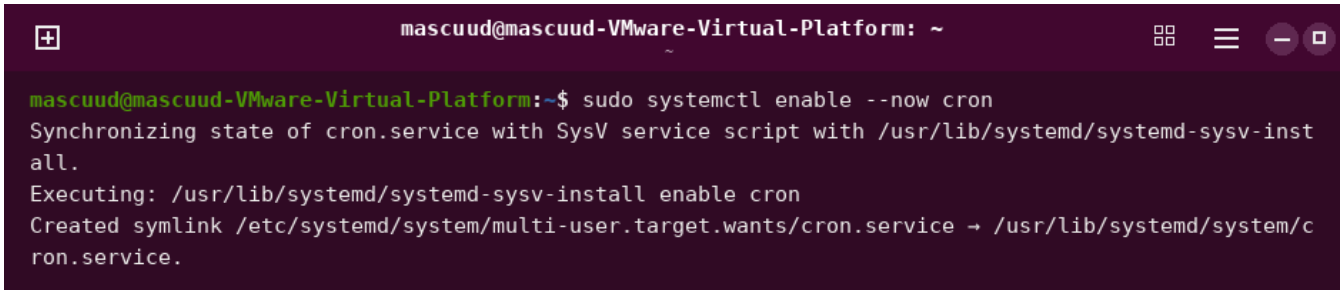
`sudo systemctl disable cron`

Stops the service from starting at boot.



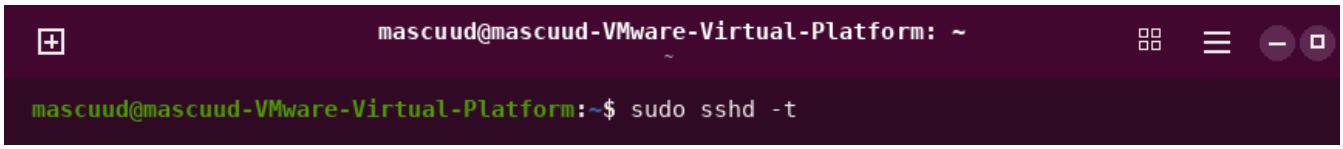
`sudo systemctl enable --now cron`

Starts the service at boot and now.



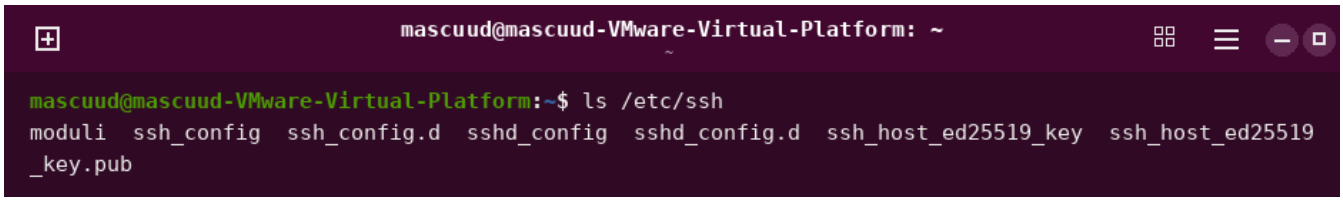
`sudo sshd -t`

Checks the ssh settings for mistakes.



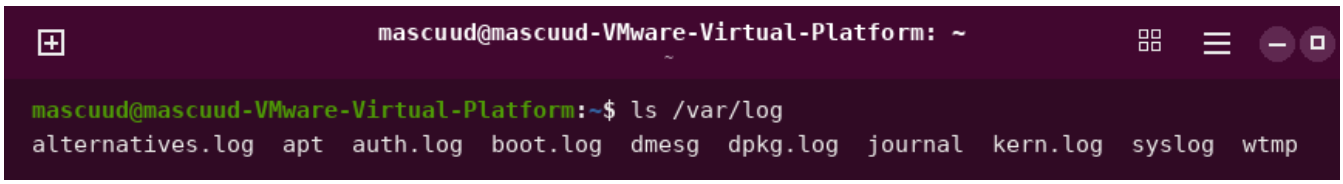
`ls /etc/ssh`

Shows the ssh settings files.



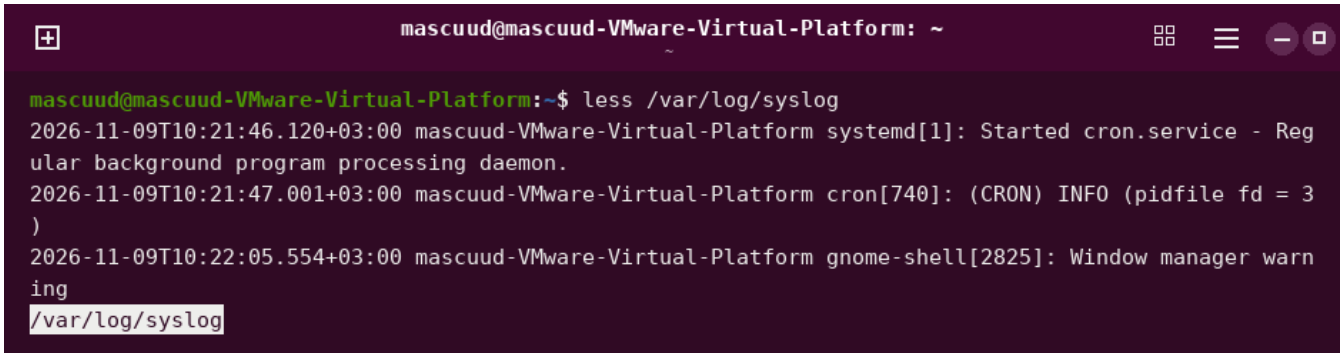
`ls /var/log`

Shows the folder with the log files.



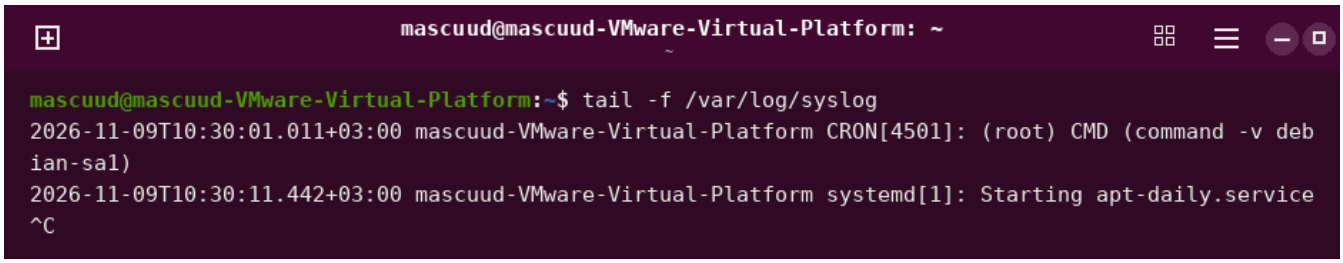
`less /var/log/syslog`

Shows the main log, one page at a time.



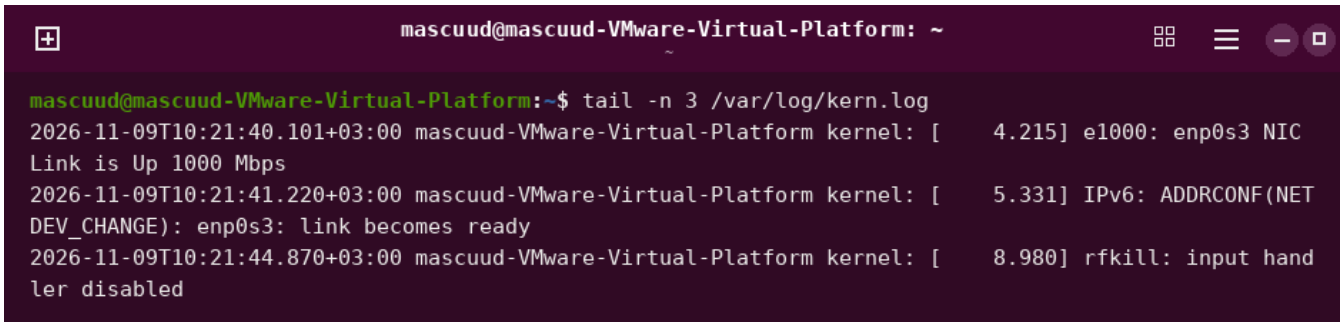
`tail -f /var/log/syslog`

Shows new log lines as they come. Press Ctrl+C to stop.



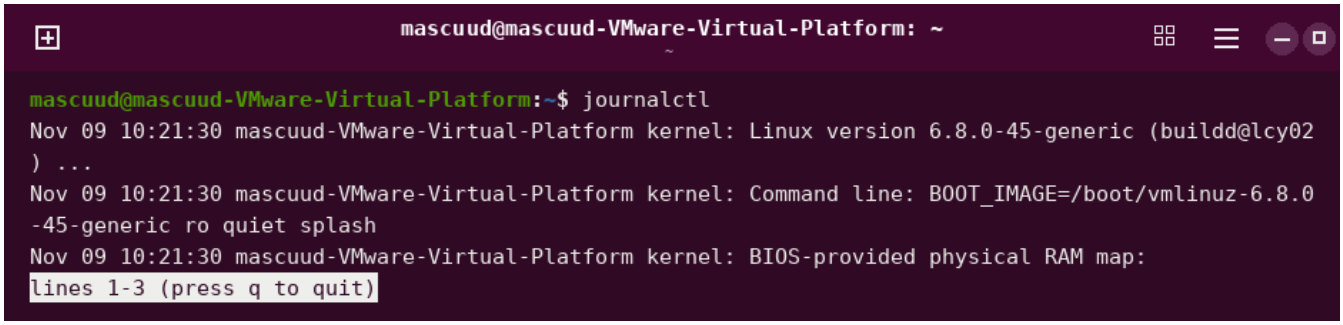
`tail -n 3 /var/log/kern.log`

Shows the last 3 lines of the kernel log.



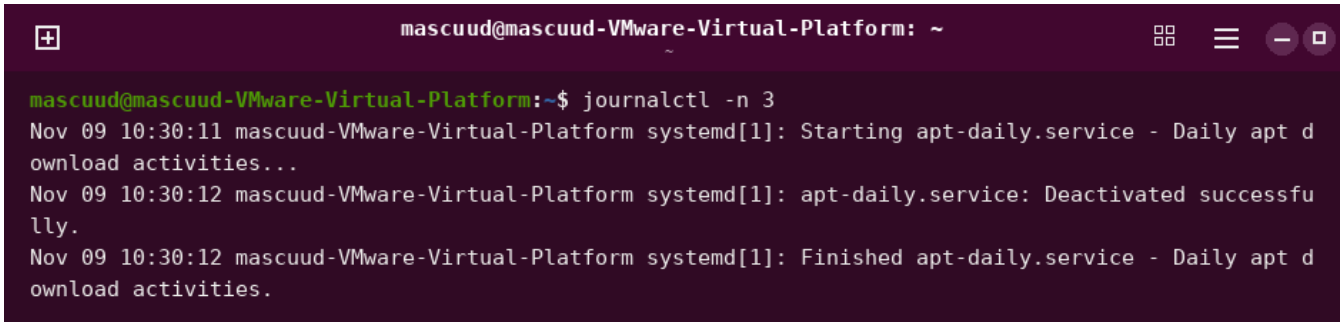
`journalctl`

Shows the system log.



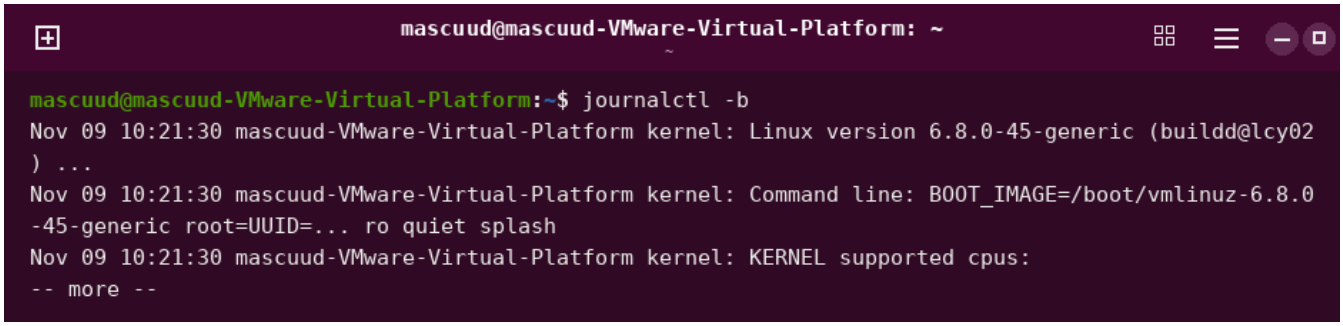
`journalctl -n 3`

Shows the last 3 lines of the log.



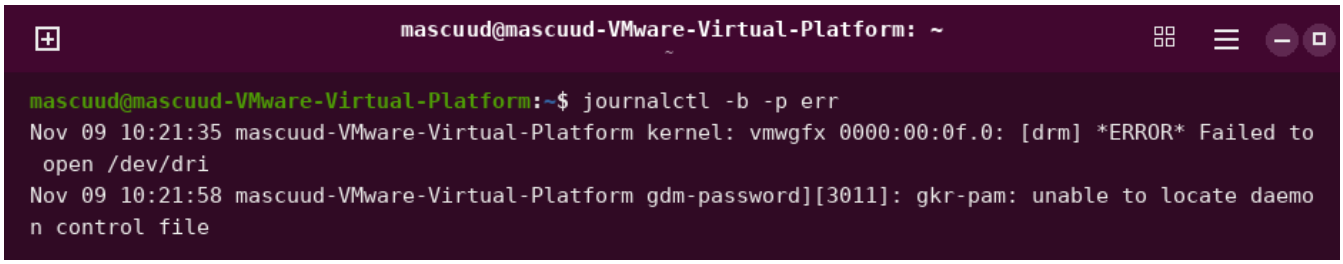
`journalctl -b`

Shows the log of this boot.



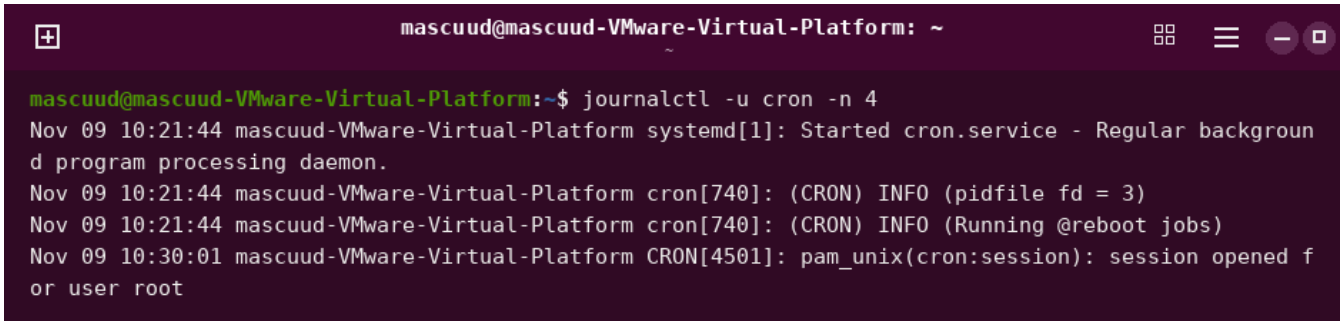
`journalctl -b -p err`

Shows only the errors of this boot.



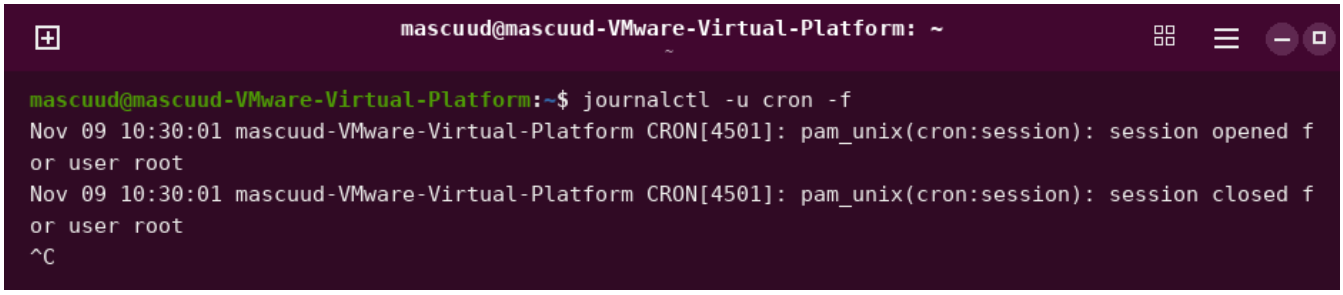
`journalctl -u cron -n 4`

Shows the last 4 log lines of the cron service.



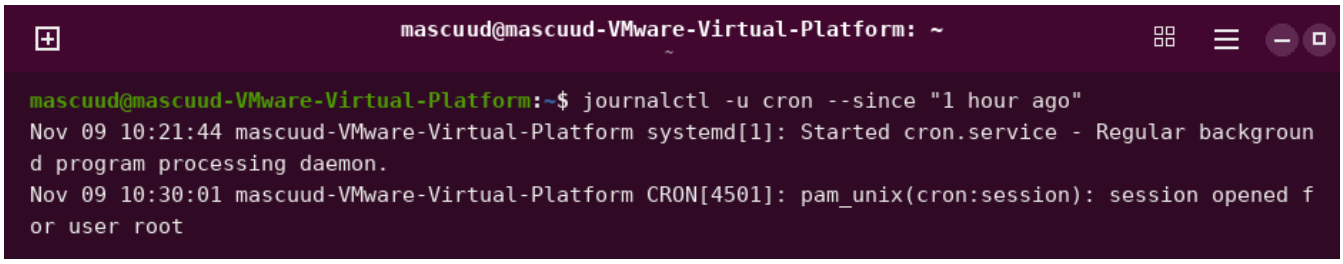
journalctl -u cron -f

Shows new log lines of the cron service.



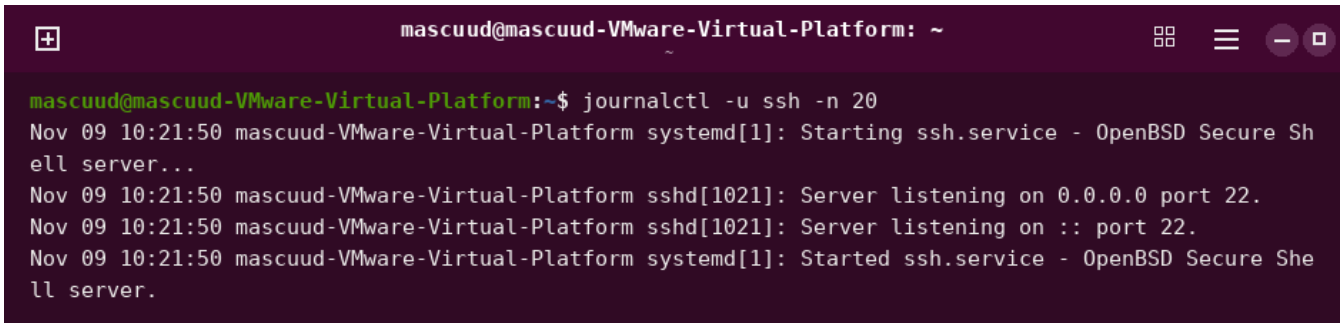
journalctl -u cron --since "1 hour ago"

Shows the cron log of the last hour.



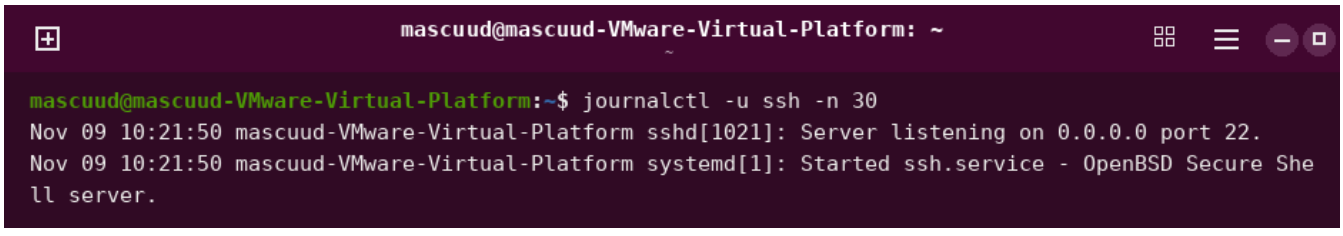
journalctl -u ssh -n 20

Shows the last 20 log lines of the ssh service.



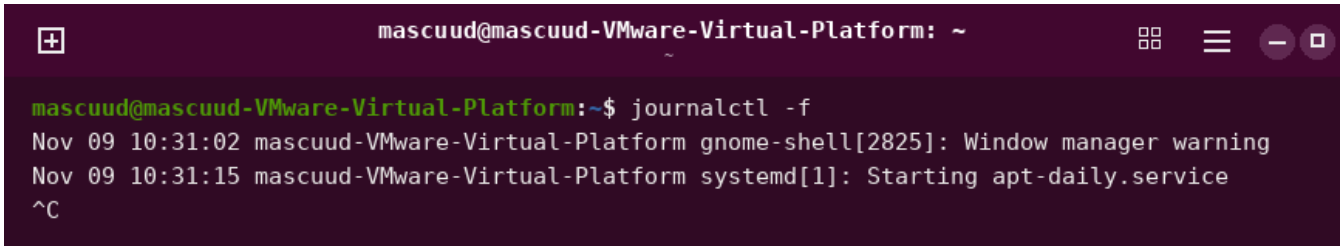
journalctl -u ssh -n 30

Shows the last 30 log lines of the ssh service.



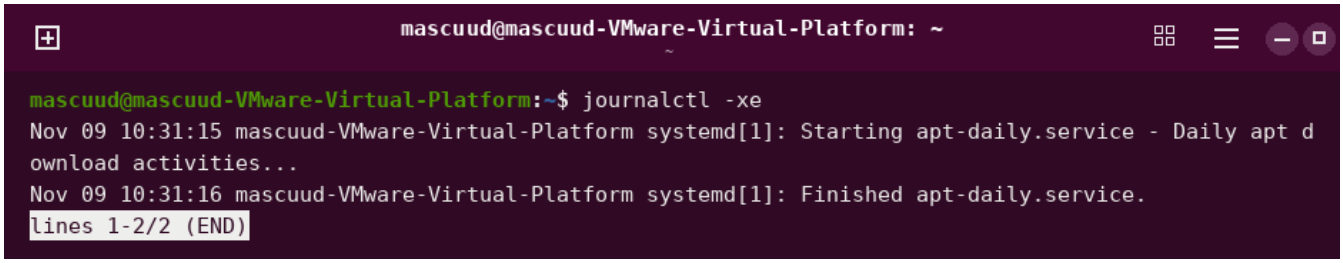
journalctl -f

Shows new log lines of the whole system.



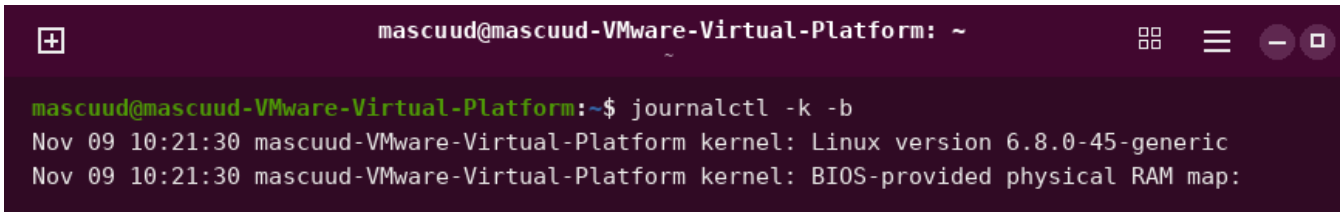
journalctl -xe

Shows the end of the log with help text.



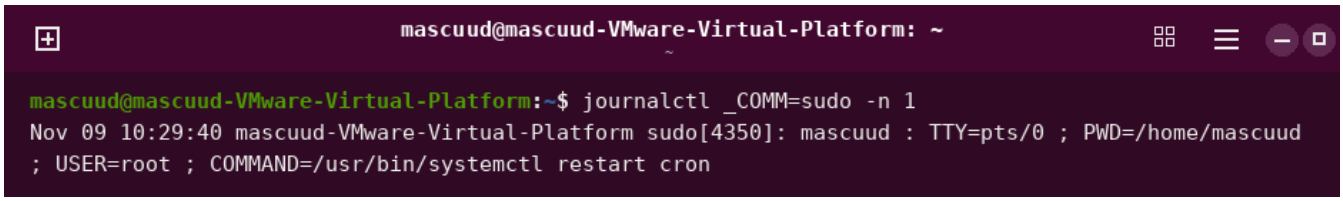
journalctl -k -b

Shows the kernel messages of this boot.



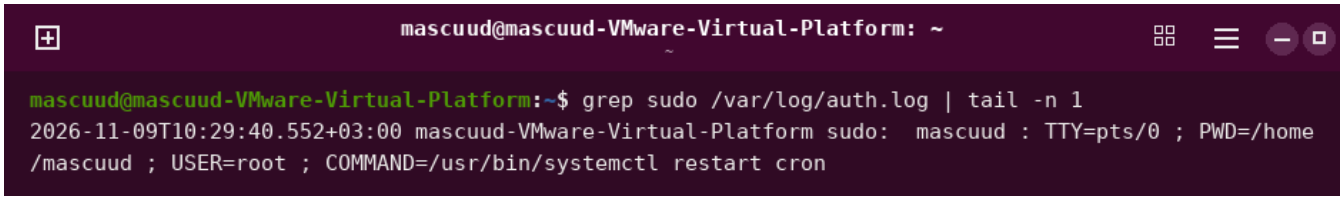
journalctl _COMM=sudo -n 1

Shows the last message from sudo.



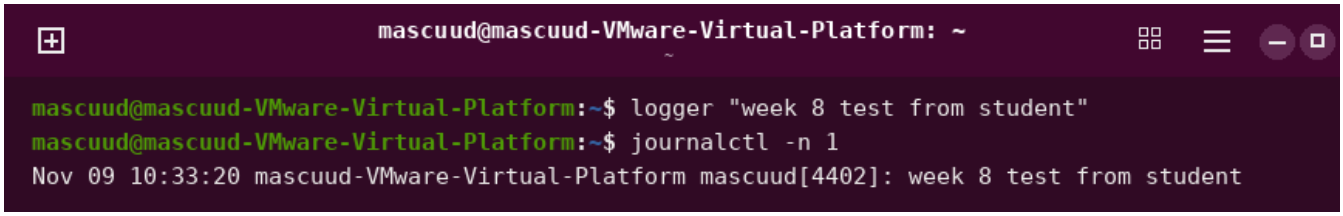
grep sudo /var/log/auth.log | tail -n 1

Shows the last sudo line in the login log.



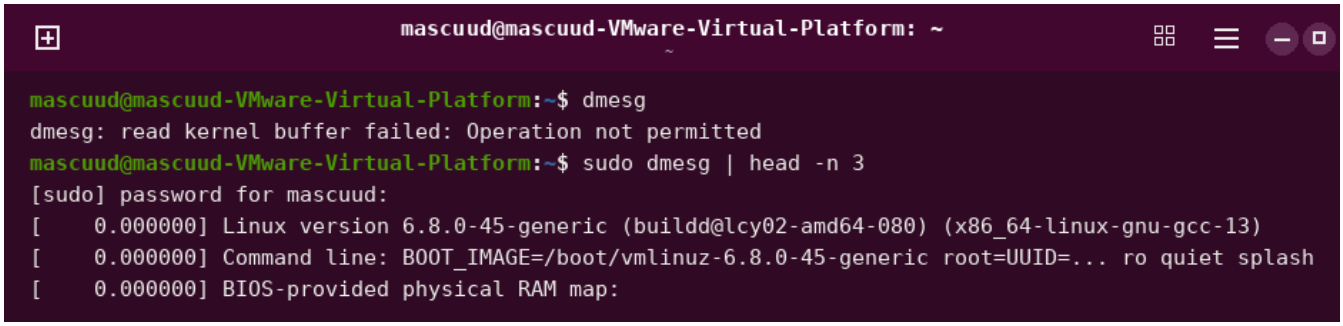
logger "week 8 test from student"

Writes your own message in the log.



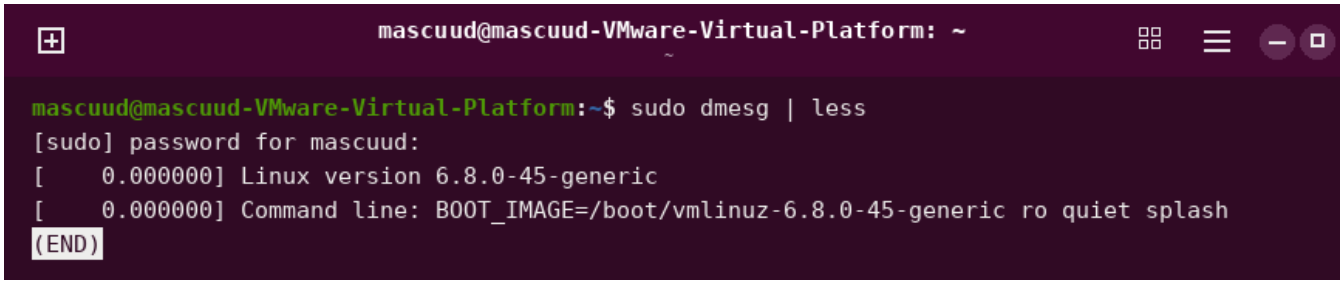
dmesg

Shows the kernel messages. You may need sudo.



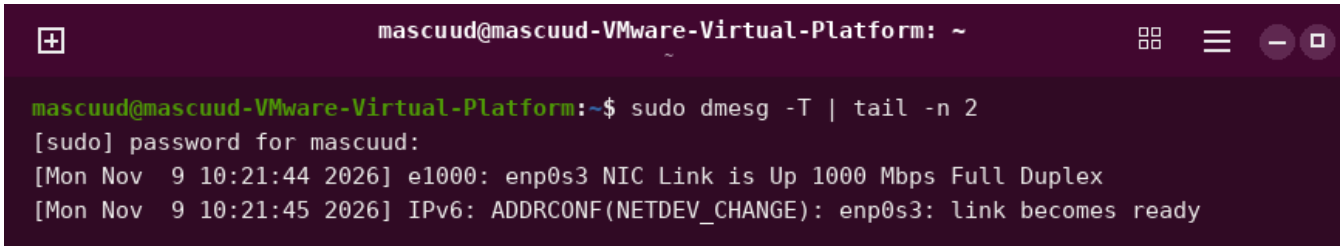
sudo dmesg | less

Shows the kernel messages, one page at a time.



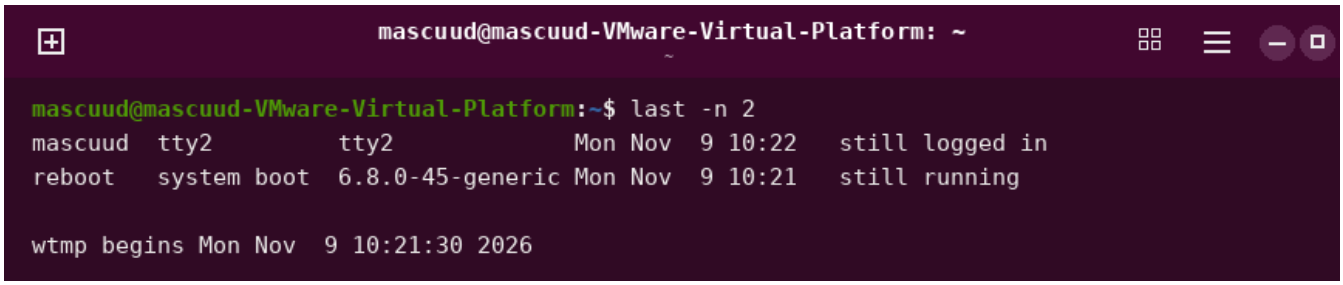
sudo dmesg -T | tail -n 2

Shows the kernel messages with real time.



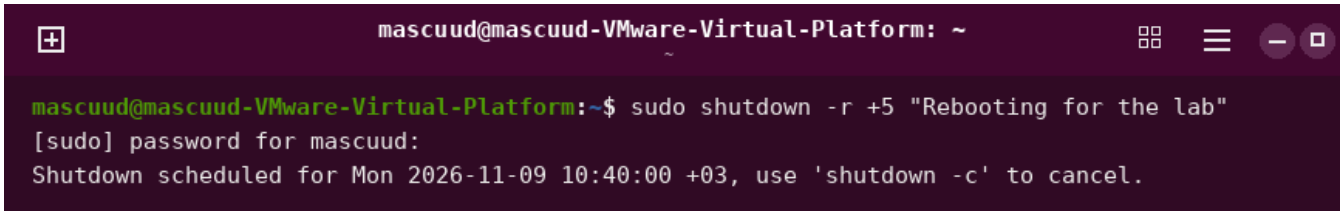
last -n 2

Shows the last 2 logins.



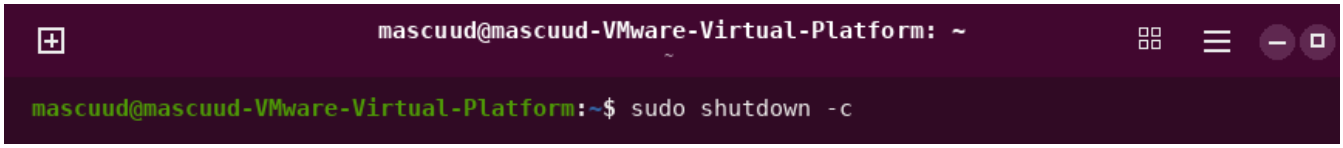
sudo shutdown -r +5 "Rebooting for the lab"

Restarts the computer after 5 minutes.



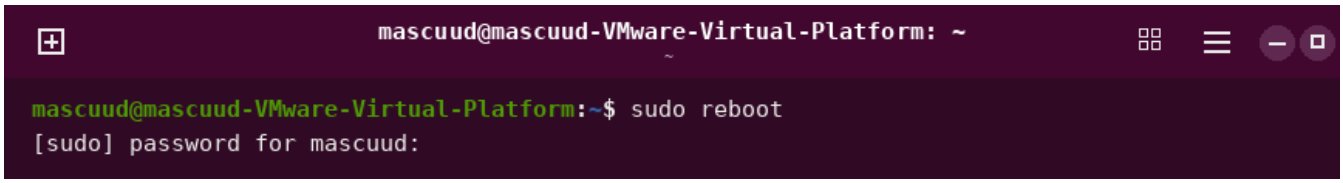
`sudo shutdown -c`

Cancels a planned shutdown.



`sudo reboot`

Restarts the computer now.



`sudo poweroff`

Turns off the computer now.

